



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Superior d'Enginyeries Industrial,
Aeroespacial i Audiovisual de Terrassa

3D Minimum-Time Trajectory Planning for Aircraft Climb, Cruise, and Descent with Accelerated Flight Profiles in a Mission Management System.

Document:

Project Charter

Author:

Pablo Urioste Alarcón

Director/Co-director:

Alex Ferrer Ferre

Degree:

Bachelor in Aerospace Technologies Engineering

Examination session:

Spring 2026

BACHELOR FINAL THESIS



Abstract

Declaration of Use of Generative Artificial Intelligence Tools

In accordance with the academic integrity policy of the UPC-ESEIAAT, the author declares the following use of generative artificial intelligence (AI) tools in the preparation of this thesis.

Models Used

- **Google Gemini Pro** (accessed via the web interface, 2024–2026)
- **Claude Code by Anthropic** (claude-sonnet-4-6, accessed via CLI, and integrated assistants in the Cursor IDE, 2025–2026)

Detail of the use of each tool

Google Gemini

- **Literature and concept exploration.** Broad scans of topics (trajectory optimisation, direct collocation, aircraft dynamics notation), suggested search directions, and short summaries of *public* articles or documentation to narrow reading lists. All citations, equations, and claims in this thesis were verified against primary sources (PDFs, textbooks, or official documentation) by the author; Gemini output was not treated as authoritative.
- **Occasional phrasing and clarity.** Alternative wordings for individual sentences or short paragraphs when the author already had a draft in place. The author revised every suggestion for technical accuracy and tone.

Claude (Claude Code / Cursor)

- **Software implementation support.** Debugging compiler and runtime errors, refactoring for readability, tracing numerical issues in the OptiFlightC codebase, and suggesting optimisa-

tions (e.g. avoiding redundant allocations, clarifying control flow). The author retained full responsibility for algorithm choice, physical models, units, and validation of results.

- **LaTeX and document structure.** Equation environments, table layouts, cross-references, and consistency checks across `.tex` files (including the uncoupled and full-control formulation chapters and preliminaries). The mathematical content and notation were fixed by the author; the assistant helped with formatting and catching typographical or structural issues.
- **Targeted technical edits.** When the author supplied a concrete goal (e.g. extend the aerodynamic coefficient block to include full stability-derivative notation, or align the thesis text with named parameters in code), the assistant proposed concrete edits to existing files. No chapter was accepted without author review.

Author's Responsibility

As explained and detailed above, disclosing the use of each generative model, the main aspects in which this thesis has been aided by AI can be summarized as:

- LaTeX formatting and readability correction
- Code debugging and concept organization
- Review and structural refining, ensuring all core simulations, figures, and scientific conclusions remain autonomously generated by the author.

Under no circumstances has generative AI been treated as a co-author of this thesis, nor has it actively shaped the final results and conclusions. Every chapter, theoretical deduction, decision, and final outcome was independently conceived, analyzed, and processed by the author. Generative AI was utilized strictly as an assistive tool to support the development of this work.

Contents

Declaration of Use of Generative Artificial Intelligence Tools	ii
List of Abbreviations and Mathematical Notation	viii
1 Introduction	1
1.1 Objectives	1
1.2 Project Scope	2
1.2.1 In-Scope: Detailed Developments	2
1.2.2 Out-of-Scope: Project Boundaries	3
1.2.3 Flow Architecture and Work Packages	4
1.3 Requirements	4
1.3.1 Functional and Dynamic Requirements	5
1.3.2 Software and Interface Requirements	5
1.3.3 Numerical and Performance Requirements	6
1.3.4 Safety and Robustness Requirements	6
1.4 Justification	6
1.5 Planning and Scheduling	8
2 State of the art	12
2.1 Geometric and Graph-Based Planning	12
2.2 Numerical Trajectory Optimization	14
2.3 Acceleration and Seeding Techniques	16
2.3.1 Numerical Structure and Gradient Information	17
2.4 Synthesis and Research Gap	18
3 Methodology	19
3.1 The challenge of trajectory optimisation with IPOPT	19

3.2	Three-tier incremental development strategy	20
3.2.1	Tier 1 — Analytical and semi-analytical exploration	20
3.2.2	Tier 2 — Benchmark optimal-control problem with minimal constraints	21
3.2.3	Tier 3 — Fully developed problem with external constraints and warm start	22
3.3	Validation procedure	22
3.4	Warm-start performance assessment	23
4	Alternative Solutions and Decision	26
4.1	Overview of Candidate Approaches	26
4.1.1	Geometric path planner (warm-start producer)	26
4.1.2	Trajectory optimisation transcription	27
4.1.3	Aircraft dynamic model used inside the optimiser	28
4.2	Decision Summary and Trade-offs	28
5	Theoretical background	31
5.1	Problem Statement	31
5.1.1	The Continuous-Time Optimal Control Problem (OCP)	31
5.1.2	Numerical Transcription to Nonlinear Programming (NLP)	33
5.1.3	Constraints and the Lagrangian Formulation	35
5.1.4	Karush-Kuhn-Tucker (KKT) Optimality Conditions	35
5.2	Uncoupled Trajectory Dynamics	36
5.2.1	Reference Frames and Coordinate Systems	36
5.2.2	State and Control Vectors	37
5.2.3	Equations of Motion	38
5.2.4	Aerodynamic Model	40
5.2.5	Summary of the Governing Equations	41
5.3	Full Control Formulation	41
5.3.1	State and Control Vectors	42
5.3.2	The Full-Control Coupled Resolution	43
5.3.3	External Wind and Aerodynamic Angles	44
5.3.4	Aerodynamic Coefficient Model	45
5.3.5	Rotation of Forces and Moments to Body Axes	49
5.3.6	Translational Dynamics	49
5.3.7	Rotational Dynamics	50
5.3.8	Euler Angle Kinematics	50
5.3.9	Position Kinematics	51

5.3.10	Mass and Energy Evolution	51
5.3.11	Summary of the Governing Equations	52
5.3.12	Comparison of the Two Formulations	52
6	Numerical Implementation	54
6.1	Numerical Transcription of the OCP	55
6.1.1	Decision-Vector Layout	55
6.1.2	Trapezoidal Direct Collocation	55
6.1.3	Residual Normalisation	56
6.2	Constraint Architecture	56
6.2.1	Domain and Environment Constraints	58
6.2.2	Collocation and Dynamic Consistency Constraints	61
6.2.3	Boundary Equality Constraints	61
6.2.4	Performance Constraints	62
6.2.5	Summary of the Constraint Architecture	63
6.3	Cost Function Definitions	64
6.3.1	Objective Function: The Time–Fuel Trade-off	64
6.3.2	Cost regularisation	66
6.4	Validation Results	70
6.4.1	Test Scenarios for Numerical Validation	70
6.4.2	Scenario Definitions	70
6.4.3	Validation Metrics	71
6.4.4	Trajectory Visualisation	71
6.4.5	Constraint Satisfaction Verification	75
6.4.6	Min-Time vs Min-Fuel Trade-off	81
6.5	Impact of cost regularisation on the optimum	85
6.5.1	Headline Metrics	85
6.5.2	Qualitative impact on the trajectory	86
6.6	Warm-Start Acceleration	91
6.6.1	Role of the Initial Guess in Interior-Point Optimisation	91
6.6.2	Hybrid A* Path Planner	92
6.6.3	Inverse-Dynamics Trajectory Reconstruction	93
6.6.4	UF → FC State Projection	94
6.6.5	Numerical Pipeline Overview	94
6.6.6	Warm-Start Performance Results	95

7	Conclusions	105
7.1	Summary of Contributions	105
7.2	Fulfilment of Requirements	105
7.3	Limitations	105
7.4	Future Work	105
	References	106
	Appendices	113
	Appendix A Warm-start benchmark: minimum-fuel detail	113
A.1	Minimum-fuel criterion: per-scenario detail	113

List of Abbreviations and Mathematical Notation

Abbreviations

Abbreviation	Full term or meaning
3-DOF	Three degrees of freedom
6-DOF	Six degrees of freedom
AoA	Angle of attack
BADA	Base of Aircraft Data (EUROCONTROL)
Bolza problem	Optimal control with Mayer (terminal) plus Lagrange (running) cost
CAGR	Compound annual growth rate
CFD	Computational fluid dynamics
CIMNE	International Centre for Numerical Methods in Engineering
DEM	Digital elevation model
DSM	Dynamic safety margin
ECEF	Earth-centred Earth-fixed (Cartesian frame)
ECTS	European Credit Transfer and Accumulation System
ERG	Explicit reference governor
FC	Full-control formulation (body-axis dynamics with control surfaces)
FMS	Flight management system
GNSS	Global navigation satellite system
IPM	Interior point method
Ipopt	Interior Point Optimizer (open-source NLP solver)
ISA	International Standard Atmosphere
KKT	Karush–Kuhn–Tucker (optimality conditions)
MMS	Mission management system
NED	North–East–Down (local geodetic frame)
NFZ	No-fly zone

Abbreviation	Full term or meaning
NLP	Nonlinear programming
NMPC	Nonlinear model predictive control
OCP	Optimal control problem
ODE	Ordinary differential equation
PRM	Probabilistic roadmap
RRT	Rapidly-exploring random tree
SFC	<i>Context-dependent</i> : (1) specific fuel consumption in fuel-burn models; (2) safe flight corridor in path planning
SiL	Software-in-the-loop
SLAM	Simultaneous Localization and Mapping
SoC	System-on-chip (state of charge when used for batteries)
SOTA	State of the art
TAS	True airspeed
TFG	Bachelor's thesis (<i>Treball de Final de Grau</i>)
UAM	Urban air mobility
UAV	Unmanned aerial vehicle
UC	Uncoupled formulation (wind-axis point mass, C_L as control)
UPC	Universitat Politècnica de Catalunya
WGS84	World Geodetic System 1984
WP	Work package

Mathematical notation

Symbol	Description
Optimal control problem and NLP transcription	
$\mathbf{x}$	State vector of the generic continuous-time OCP ($\mathbb{R}^n$)
$\mathbf{u}$	Control vector of the generic continuous-time OCP ($\mathbb{R}^m$)
J	Scalar cost functional (Bolza form)
Φ	Mayer (terminal) cost
$\mathcal{L}$	Lagrange (running) integrand; also used for rolling moment in <i>wind</i> axes (see Aerodynamics below)
t_0, t_f	Initial and final time; t_f may be a free optimisation parameter
$\mathbf{f}$	Right-hand side of state ODEs, $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$
Ψ	Terminal constraint vector
$\mathbf{h}, \mathbf{h}_{\min}, \mathbf{h}_{\max}$	Path constraint functions and bounds

Symbol	Description
$\mathbf{z}$	Stacked decision vector after transcription (NLP); also UC/FC state vector in the thesis
F, g, h	Discrete objective and constraint functions in NLP notation
λ, ν	Lagrange multipliers (equalities / inequalities)
N	Number of discretisation intervals in direct transcription

Frames, positions, and kinematic angles

$\mathcal{F}_{\text{NED}}, \mathcal{F}_W$	Local North–East–Down frame; wind-axis frame
φ, λ, h	Geodetic latitude, longitude, ellipsoidal height
x, y, z	NED position components (UC); altitude $h = -z$
x_e, y_e, z_e	NED position (FC notation)
u, v, w	Body-axis velocity components (FC)
u_r, v_r, w_r	Wind-relative body velocities
$V_{wb,1}, V_{wb,2}, V_{wb,3}$	Components of wind in body axes
$\mathbf{R}_{bn}, \mathbf{R}_{nb}$	Direction-cosine matrices (body $\leftrightarrow$ NED)
ϕ, θ, ψ	Euler roll, pitch, yaw (FC); ψ replaces heading ξ of UC
γ, ξ, μ	Flight-path angle, heading angle, bank angle (UC)
V	True airspeed (UC)
V_a	Aerodynamic speed (FC), $V_a = \sqrt{u_r^2 + v_r^2 + w_r^2}$
α, β	Angle of attack and sideslip (algebraic in FC)

Uncoupled formulation: states, controls, and drag polar

$\mathbf{z}$	$[x, y, z, V, \mu, \gamma, \xi, m, C]^T$ (state vector)
$\mathbf{u}$	$[T, C_L, p]^T$ (thrust, lift coefficient, roll rate command)
T	thrust [N]
C_L	Lift coefficient (direct control input in UC)
p	Roll rate command (bank kinematics) [rad s^{-1}]
m	Mass [kg]; m_0, m_{dry} initial and dry masses
C	Battery charge (electric aircraft)
q	Dynamic pressure $q = \frac{1}{2}\rho V^2$ (UC)
S	Wing reference area
ρ	Air density (ISA)
η_p	Propulsive efficiency
P	Shaft power [W]
c_{sfc}	Specific fuel consumption [$\text{g kW}^{-1} \text{h}^{-1}$]
$C_{D_0}, k, e, \mathcal{A}$	Parasite drag, induced-drag factor, Oswald factor, aspect ratio; $k = 1/(\pi e \mathcal{A})$

Full-control formulation: states, controls, and non-dimensional rates

Symbol	Description
$\mathbf{z}$	$[x_e, y_e, z_e, m, u, v, w, \phi, \theta, \psi, p, q, r, C]^T$
$\mathbf{u}$	$[T, \delta_e, \delta_a, \delta_r]^T$
$\delta_e, \delta_a, \delta_r$	Elevator, aileron, rudder deflections [rad]
p, q, r	Body-axis angular rates [rad s^{-1}] (not UC roll-rate control)
q_S	Dynamic pressure $\times$ area, $q_S = \frac{1}{2}\rho(z_e) V_a^2 S_w$
S_w	Wing reference area in FC notation
b	Wing span
$\bar{c}$	Mean aerodynamic chord
$\hat{p}, \hat{q}, \hat{r}, \hat{u}, \hat{\alpha}$	Non-dimensional rates: $\hat{q} = \bar{c}q/(2V_a)$, etc.; $\hat{u} = u_r/V_a$
$I_{xx}, I_{yy}, I_{zz}, I_{xz}$	Moments and product of inertia (including roll–yaw coupling)
d_{Tz}	Thrust arm for pitch moment from thrust offset
Aerodynamics: coefficients, forces, moments, and stability derivatives	
C_L, C_D, C_Y	Lift, drag, side force coefficients
C_l, C_m, C_n	Rolling, pitching, yawing moment coefficients (bold C_l vs. lift C_L)
L, D, Y	Lift, drag, side force [N]
$\mathcal{L}_W, \mathcal{M}_W, \mathcal{N}_W$	Rolling, pitching, yawing moments in wind axes [N m]
L_b, M_b, N_b	Body-axis moment components
$F_{x,a}, F_{y,a}, F_{z,a}$	Body-axis aerodynamic force components
$C_{L_0}, C_{L_\alpha}, C_{L_{\delta_e}}, C_{L_q}$	Lift: base, α , δ_e , and q effects.
$C_{D_0}, C_{D_{k1}}, C_{D_{k2}}, C_{D_q}, C_{D_{CT}}$	Drag: parasitic, induced, q , and thrust (C_T).
$C_{Y_0}, C_{Y_\beta}, C_{Y_{\delta_a}}, C_{Y_{\delta_r}}, C_{Y_p}, C_{Y_r}$	Side force: β , δ_a , δ_r , p , and r .
$C_{l_0}, C_{l_\beta}, C_{l_{\delta_a}}, C_{l_{\delta_r}}, C_{l_p}, C_{l_r}$	Rolling moment: β , δ_a , δ_r , p , and r .
$C_{m_0}, C_{m_\alpha}, C_{m_{\delta_e}}, C_{m_q}$	Pitching-moment derivatives: trim offset, static stability with angle of attack, elevator control effectiveness, and pitch damping associated with the non-dimensional rate $\hat{q}$. Higher-order unsteady and speed-stability terms (commonly written as $C_{m_{\dot{\alpha}}}$ and C_{m_u}) are acknowledged in the literature but are set to zero in the numerical implementation used in Section 6.4.
$C_{n_0}, C_{n_\beta}, C_{n_{\delta_a}}, C_{n_{\delta_r}}, C_{n_p}, C_{n_r}$	Yawing moment: β , δ_a , δ_r , p , and r .

List of Figures

1.1	Bachelor thesis and software development Gantt chart by work package	11
2.1	Dijkstra shortest-path search on a 2D grid with static NFZ constraints	13
2.2	Sampling-based RRT variants: comparative path-planning results	13
5.1	Continuous-time OCP: dynamics, cost, boundary and path constraints	32
5.2	Direct single shooting: discrete controls and forward-integrated states	34
5.3	ECEF frame, geodetic coordinates, and construction of local NED	37
5.4	FC algorithm: sequential right-hand-side evaluation	44
6.1	Trajectory overview — all criteria and formulations	72
6.2	UF altitude profiles per scenario and criterion	73
6.3	FC altitude profiles per scenario and criterion	74
6.4	UF worst-case constraint margins, all scenarios	77
6.5	FC worst-case constraint margins, all scenarios	80
6.6	Min-time vs min-fuel comparison, UF	81
6.7	Min-time vs min-fuel comparison, FC	82
6.8	UF vs FC altitude overlay, SC-00	84
6.9	UF vs FC altitude overlay, SC-01	84
6.10	UF vs FC altitude overlay, SC-02	84
6.11	UF vs FC altitude overlay, SC-03	85
6.12	UF regularisation dashboard for SC-01, min-fuel	87
6.13	FC regularisation dashboard for SC-01, min-fuel	88
6.14	FC control evolution comparison for SC-01, min-fuel	89
6.15	FC control-surface deflections, min-fuel	89
6.16	FC control-surface deflections, min-time	90
6.17	Numerical pipeline overview	95
6.18	UF warm- vs cold-start trajectories, scenario 8	99
6.19	UF warm- vs cold-start trajectories, scenario 5	100

List of Tables

1.1	Work packages and milestone structure	4
1.3	Tasks and estimated hours by work package	8
2.1	Path-planning methods: algorithms and trade-offs	13
2.2	Trajectory derivatives: naming and physical interpretation	15
3.1	Three-tier methodology: scope and role of each formulation	20
4.1	Design decisions	29
5.1	Uncoupled formulation: state variables	38
5.2	Uncoupled formulation: controls and terminal time	38
5.3	Uncoupled formulation: parabolic polar parameters	40
5.4	Uncoupled formulation: nine equations by physical category	41
5.5	Full Control formulation: state variables	42
5.6	Full Control formulation: controls and terminal time	43
5.7	FC model: stability and control derivatives	48
5.9	Full Control formulation: fourteen governing differential equations	52
5.10	Uncoupled vs. Full Control: structural comparison	53
6.1	NLP size of the two formulations	55
6.2	NLP reference scaling factors	57
6.3	Performance constraints: UF vs. FC	63
6.5	Cost weight α regimes	65
6.6	Validation benchmark scenarios	70
6.7	SC-01 cross-mode headline metrics	85
6.8	Aggregate means across all scenarios. UF rows: warm-start vs. no-warm-start. FC rows: WS→UF→FC vs. WS→FC (positive $\Rightarrow$ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	96
6.9	Computational and iteration overview (UF, min_time): warm-start (WS) vs. no-warm-start (No-WS). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	97

6.10	Computational and iteration overview (FC, min_time): WS→UF→FC (baseline) vs. WS→FC (skip the UF solve). Iteration counts are whole-pipeline totals; WS→UF→FC sums the UF and FC solves. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	101
6.11	Per-scenario relative differences (min_time). UF: warm-start speedup / iteration reduction (positive ⇒ warm start cheaper). FC: WS→UF→FC vs. WS→FC (positive ⇒ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	102
6.12	Trajectory flight time [s] (min_time). UF: warm-start (WS) vs. no-warm-start (No-WS). FC: WS→UF→FC vs. WS→FC. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	103
A.1	Computational and iteration overview (UF, min_fuel): warm-start (WS) vs. no-warm-start (No-WS). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	114
A.2	Computational and iteration overview (FC, min_fuel): WS→UF→FC (baseline) vs. WS→FC (skip the UF solve). Iteration counts are whole-pipeline totals; WS→UF→FC sums the UF and FC solves. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	114
A.3	Per-scenario relative differences (min_fuel). UF: warm-start speedup / iteration reduction (positive ⇒ warm start cheaper). FC: WS→UF→FC vs. WS→FC (positive ⇒ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	115
A.4	Trajectory end mass [kg] (min_fuel). UF: warm-start (WS) vs. no-warm-start (No-WS). FC: WS→UF→FC vs. WS→FC. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.	115

Chapter 1

Introduction

A fundamental pillar of aircraft trajectory planning is the preliminary analysis of feasible flight paths. Beyond simple pathfinding, modern aviation requires robust algorithms capable of optimizing routes from an initial point A to a destination B. In the context of Unmanned Aerial Vehicles (UAVs), this becomes even more critical due to the need for precise guidance through complex sequences of waypoints. Furthermore, should the onboard systems detect any environmental interference or operational constraints—such as obstacles, forbidden zones, or potential collision threats—the algorithm must be capable of dynamically computing an alternative feasible trajectory in real-time.

By leveraging differential equations and advanced optimization mathematics, this study develops a responsive framework within the field of flight mechanics. The proposed approach implements optimal routes that strictly account for control factors and the aircraft's specific performance parameters. Modern trajectory planning has evolved beyond manual drafting; it is now an intelligent process that integrates precise performance limitations and aerodynamic constraints, which ultimately dictate the trajectory and velocity of various flight maneuvers.

1.1 Objectives

The primary goal of this thesis is the design and implementation of a high-performance 3D trajectory solver integrated into a Linux-based Mission Management System (MMS). This software is intended to compute optimal flight paths for a generic aircraft, moving beyond traditional constant-velocity models by incorporating accelerated flight profiles during climb, cruise, and descent. A key functional requirement is the seamless integration of environmental and operational constraints, specifically utilizing Digital Elevation Models (DEM) [1] for terrain clearance and strict logic for No-Fly Zone (NFZ) [2] avoidance, ensuring that the resulting trajectories are not only time-optimal but also safety-compliant.

Central to this objective is the pursuit of computational efficiency to meet a strict real-time operational threshold of under 4 seconds. Consequently, the thesis focuses on the mathematical simplification of the Optimal Control Problem (OCP) to facilitate rapid convergence. A significant portion of the research is dedicated to the development of a hierarchical optimization strategy that utilizes semi-analytical solutions and geometric path planning. These simplified models serve as a robust “warm start” for the high-fidelity solver, providing a high-quality initial guess that significantly reduces the search space and avoids the pitfalls of random initialization.

Furthermore, this work aims to exploit an Inverse Dynamics [3] problem formulation as a core acceleration technique. By mapping desired spatial trajectories directly to the required control inputs, the system bypasses the heavy computational burden associated with traditional forward integration during the seeding phase. The final objective is to validate this dual-layer approach, ensuring that the transition from simplified analytical guesses to full-scale numerical optimization maintains physical consistency with the aircraft’s flight envelope and performance parameters .

1.2 Project Scope

1.2.1 In-Scope: Detailed Developments

The following areas constitute the core technical contributions of this research and are developed in full:

- Development of a fully coupled state-control architecture integrating the aircraft equations of motion and dynamic transitions within a Linux-based Software environment.
- Implementation of high-fidelity performance models accounting for variable mass from fuel consumption and the influence of wind vectors on ground-speed relationships.
- Design of a modular numerical optimization framework that utilizes both direct collocation and multiple shooting transcription methods for simultaneous state and control optimization.
- Implementation of a bidirectional coordinate transformation layer between internal analytical frames and the North-East-Down (NED) geodetic system to ensure full compatibility with standard autopilot waypoint protocols.
- Formulation of a geospatial constraint engine that translates digital elevation models and no-fly zone polygons into mathematical inequality constraints for terrain clearance and geofencing.
- Research into acceleration algorithms and semi-analytical techniques to optimize the computational pipeline for real-time compliance within a sub-4-second threshold.
- Integration of heuristic seeding methods, specifically using A* pathfinding and Dubins curves,

to provide the non-linear solver with a robust warm start and improved convergence.

- Migration of the core optimization logic to C++ to achieve a production-grade deliverable capable of handling high-frequency trajectory updates.
- Execution of a systematic analysis across several representative usecases to validate the model's reliability in diverse mission profiles and varying environmental conditions.
- Comparison of performance metrics and time optimization results between different solvers, evaluating the computational gains of warm start versus cold start strategies.
- Development of analytical comparisons to verify the numerical solver's output against theoretical aircraft performance limits and established flight envelopes.
- Generation of a technical report assessing the environmental impact benefits, specifically focusing on fuel-burn reduction and emission optimization through efficient trajectory design.
- Assessment of the technology's scalability and market potential, exploring the possibilities for massive deployment within the U-Space and urban air mobility sectors.

1.2.2 Out-of-Scope: Project Boundaries

To ensure the technical depth remains focused on the optimization framework within the allocated ECTS timeframe, the following areas are excluded:

- The SiL (Software-in-the-Loop) analysis and model validation are performed by Singular Aircraft; therefore, the execution of these tests and their primary results are outside the scope of this project. While the author will incorporate external feedback into the system design, the formal validation process will not be personally conducted.
- The framework assumes pre-processed and clean data inputs for state estimation and mapping, omitting the development of SLAM, LiDAR processing, or computer vision algorithms.
- Aerodynamic modeling is restricted to established performance databases and simplified coefficient models, excluding the execution of new CFD simulations or wind tunnel testing.
- The project focuses strictly on the backend solver architecture and Mission Management System logic, excluding the design and implementation of high-fidelity graphical user interfaces.
- While the solver incorporates Digital Elevation Models and No-Fly Zone polygons as operational constraints, the project does not include a comprehensive assessment of the accuracy or quality of these geospatial datasets.

1.2.3 Flow Architecture and Work Packages

The thesis is organized into six Work Packages (WPs), representing a workflow where each phase builds the necessary infrastructure to reach the next milestone.

Table 1.1: Structure of Work Packages and Project Milestones.

WP	Technical Description	Milestone Achievement
WP1	Development of uncoupled kinematic models and point-force formulations. Focuses on non-transient optimization and familiarization with NLP solvers.	Capability to generate simplified 3D trajectories using uncoupled point-mass dynamics in static test cases.
WP2	Implementation of fully coupled 6-DOF dynamics utilizing a full trapezoidal integrator. Integration of No-Fly Zones (NFZ) and environmental constraints into the solver.	Convergence of high-fidelity trajectories that respect simultaneous translational and rotational constraints.
WP3	Verification of all numerical constraints and optimization of the source code for performance, focusing on computational speed and stability.	Validation of the solver's numerical consistency and achievement of improved execution latency.
WP4	Development of warm-start logic utilizing Dubins curves and A* pathfinding to generate high-quality initial seeds for the optimization layer.	Significant reduction in solver iterations and prevention of local minima traps during initialization.
WP5	Development of inverse dynamics to mesh warm-start trajectories with the solver, enabling the synthesis of full state and control vectors from geometric seeds.	Successful reconstruction of feasible state-space profiles from kinematic path primitives.
WP6	Synthesis of experimental results, formal validation of the mission management framework through and external party, and completion of the thesis manuscript.	Formal verification of the system's operational envelope and final submission of the research.

1.3 Requirements

The following technical requirements define the operational envelope, data interfaces, and functional constraints of the proposed Mission Management System (MMS).

1.3.1 Functional and Dynamic Requirements

- The MMS shall solve simultaneous translational and rotational equations of motion using a fully coupled 6-DOF formulation to optimize control inputs and vehicle dynamics as a single system.
- The optimization solver shall be decoupled from the specific vehicle plant, accepting external datasets for aircraft performance envelopes, mass properties, and aerodynamic coefficients.
- The solver shall enforce state and control constraints, including maximum actuator deflections (δ) and rate limits ($\dot{\delta}$), as defined by the vehicle input data.
- The equations of motion shall incorporate a dynamic wind-vector field to ensure trajectory accuracy and energy-state reliability under non-zero ambient flow conditions.
- The solver shall enforce environmental constraints—including Digital Elevation Models (DEM), No-Fly Zones (NFZ), and Geofencing—as formal boundaries within the optimization problem, in addition to vehicle-specific state and control variable limitations.
- The framework shall implement multiple *warm start* techniques. The system must be capable of dynamically selecting and activating specific initialization strategies based on the operational use case to ensure high-speed convergence.
- The optimization solver shall be designed to function independently of the transcription method. It must support both *Direct Collocation* and *Multiple Shooting* formulations to allow for varied levels of numerical robustness and problem sparsity.
- The architecture shall support multiple numerical integration schemes for state propagation, including *Forward Euler*, *Runge-Kutta 4 (RK4)*, and *Trapezoidal* rules, allowing for configurable precision-to-performance ratios.
- For initial mission activation or recovery from infeasible states, the system shall generate an analytical trajectory seed using kinematic path primitives (e.g., Dubins curves or straight-line segments) to initialize the solver.

1.3.2 Software and Interface Requirements

- The system shall utilize a standardized JSON-based input architecture for parsing mission parameters, environmental constraints, and geodetic waypoints.
- The system shall export 3D trajectories in geodetic coordinates compliant with the WGS84 reference frame for integration with industrial GNSS and autopilots.
- The core solver and mission management logic shall be developed to execute natively within a

Linux-based SiL environment. The architecture must ensure seamless data exchange between the optimization framework and the simulated aircraft plant to allow for industrial validation of the 6-DOF control laws before deployment.

1.3.3 Numerical and Performance Requirements

- The solver shall converge on a feasible trajectory solution within a maximum latency of 4.0 seconds per tactical look-ahead window.
- The optimization engine shall achieve a solution with a fixed convergence tolerance of 10^{-4} , where results exceeding this residual error are discarded as non-feasible.
- The solver shall utilize analytical Jacobians for constraints and cost functions, maintaining a precision error relative to finite-difference checks of less than 10^{-6} .
- The C++ production core shall utilize static or pre-allocated memory structures for all optimization-related matrices, with no dynamic heap allocation permitted during the solver loop.

1.3.4 Safety and Robustness Requirements

- The system shall perform automated range-checking on all JSON-parsed inputs, rejecting data that exceeds defined structural or geodetic boundaries.
- In the event of solver non-convergence or a timeout exceeding 4.0 seconds, the MMS shall output the last valid trajectory segment or the analytical backup seed.
- The output trajectory shall maintain C^2 continuity to ensure the path is physically achievable by the aircraft's actuators without inducing structural jitter.
- The system shall log the solver's internal performance metrics, including iteration count and residual error, for every optimization cycle.

1.4 Justification

Aerospace engineering is currently undergoing a paradigm shift driven by the development of **Mission Management Systems (MMS)**, which transition unmanned platforms from simple navigational aids to autonomous decision-making structures. As Unmanned Aerial Vehicles (UAVs) integrate into commercial logistics and Urban Air Mobility (UAM) networks, the aviation industry is experiencing an unprecedented industrial boom, with global market forecasts indicating a Compound Annual Growth Rate (CAGR) exceeding 25% [4], [5]. Consequently, the industry is moving away from basic heuristic waypoint navigation toward complex real-time trajectory optimization algorithms capable of functioning in crowded, non-segregated airspace.

Despite these advancements, a significant technological rift persists between idealized geometric planning and the rigorous physical realities of flight dynamics. Current industrial standards often favor “kinematic decoupling,” adhering to simplified 3-DOF (Three Degrees of Freedom) point-mass models such as the EUROCONTROL Base of Aircraft Data (BADA) [6]. Historically, this abstraction has been a necessary concession to the limited computational throughput of flight-certified hardware and the requirements of safety-critical software certification. Because solving a fully coupled 6-DOF optimization in real-time is a non-convex, intensive task, most commercial systems utilize high-speed search algorithms (such as A* [7] or RRT* [8]) that essentially defer the burden of aerodynamic complexity to the lower-level autopilot.

This reliance on simplified models is increasingly inadequate for high-performance or heavy-payload platforms, such as the amphibious *Flyox I* [9]. By ignoring the high-frequency coupling between control surface deflections and aerodynamic stability, solvers frequently output geometric paths under the risky assumption that the autopilot possesses the control authority to compensate for dynamic residuals. This can lead to *tracking divergence*¹, occurring when a trajectory is mathematically plotted but physically unachievable due to actuator saturation, shifting centers of gravity, or severe cross-coupled moments.

The origins of this thesis are fundamentally rooted in a professional collaboration with the *International Centre for Numerical Methods in Engineering (CIMNE)* through a series of curricular and extracurricular internships. The primary objective of this engagement was the development of a robust Mission Management System for *Singular Aircraft* [10]. The development of the core algorithms and codebase was driven by industrial R&D requirements, necessitating an early start prior to the official academic term in March. This chronological overlap was essential to ensure the depth and quality of the technical work, positioning the thesis as the final developmental phase of a project already integrated into a professional aerospace pipeline.

Ultimately, this thesis serves as a technical booster for the existing MMS architecture by developing a fully coupled state-control optimization framework [11]. By formulating the trajectory generation as a *Nonlinear Optimal Control Problem (OCP)* and targeting high-performance embedded Systems-on-Chip (SoC), the proposed system embeds the aircraft’s true attitude and actuator constraints directly into the planning phase. While the current work focuses on solving these OCPs, the framework is architected for eventual deployment as a *Nonlinear Model Predictive Control (NMPC)* scheme [12], intended for instantaneous planning and continuous updates via onboard sensors. To ensure real-time feasibility for such future closed-loop applications, the research implements specific acceleration techniques, most notably *Warm Start* features and *Inverse Dynamics*. These features allow the solver to handle the complexity of the 6-DOF model while remaining computationally agile, ensuring that autonomous operations are strictly bounded by the true aerodynamic and structural limits of the airframe.

¹**Tracking divergence:** A state in which the error between the commanded trajectory and the actual dynamic state increases monotonically over time, preventing asymptotic stability.

1.5 Planning and Scheduling

Given that developing a 3D trajectory optimizer is a highly complex and non-trivial undertaking, the project delivered to Singular officially commenced in September 2025. This early start was essential to establish the core algorithmic framework required for the subsequent analysis.

The following schedule integrates the code development process conducted during the autumn semester, highlighting the direct relationship between these foundational tasks and the final objectives of this Bachelor's Thesis (TFG). The initial phase focused on defining the optimality problems and building the robust software architecture necessary to support the advanced computations presented in the latter stages of the study.

Table 1.3: Detail of tasks classified in each work package of the thesis.

ID	Type	Detailed Description and Technical Objectives	Hours
WP1: Uncoupled Formulation			
T-001	Theory	State of the Art: Research on transcription methods (Direct Collocation) and parabolic trajectory discretization.	8
C-001	Coding	Aircraft Class Composition: OOP architecture for aerodynamic and mechanical parameters.	3
C-002	Coding	Analytical Non-Opt: Implementation of constant speed use-cases without optimization.	12
C-003	Coding	Analytical Opt: min Time/Fuel optimization for uncoupled systems.	12
C-004	Coding	3D Analytical Extension: Integration of coordinate turns into the analytical model.	4
T-002	Theory	Optimization Formulation: Definition of cost functions and study of NLP solvers.	2
C-005	Coding	Numerical Opt (Scalar): Solver for $V = \text{const}$ uncoupled formulations.	4
C-006	Coding	Numerical Opt (Vector): Solver for transient states ($V = \text{variable}$).	12
C-007	Coding	Horizontal Simplification: Acceleration/deceleration logic validation.	5
C-008	Coding	3D Transient Uncoupled: Complex 3D problem for min Time/Fuel.	6
T-003	Theory	NFZ Approach: Mathematical formulation of geofencing constraints.	4

Continues on next page...

ID	Type	Detailed Description and Technical Objectives	Hours
C-010	Coding	NFZ Integration: Inequality constraints implementation in the solver.	8
WP2: Coupled Formulation			
T-004	Theory	Coupled Dynamics: Theory of 6-DOF stability and aircraft coupling.	12
C-011	Coding	Analytical Non-Opt (Coupled): Non-linear solving with fsolve.	6
C-012	Coding	Wind Integration: Equation modification for aerodynamic speed vs ground speed.	8
C-013	Coding	Mass Loss Integration: Power-based fuel consumption iterative loop.	4
C-014	Coding	Analytical Opt (Coupled): Static optimization with full performance restrictions.	12
C-015	Coding	Transient Opt (Coupled): Full vector optimization for coupled dynamics.	20
C-016	Coding	NFZ in Coupled: Validation of geofencing in the 6-DOF environment.	14
WP3: Performance Study & Optimization Core			
C-022	Coding	Performance Study: Assessment of solver robustness and accuracy.	10
C-023	Coding	C++ Migration: Porting Matlab core to C++ for computational speedup.	40
C-024	Coding	Adjoint & Gradient Formulation: Analytical sensitivity analysis to improve convergence.	35
WP4: Warm Start			
C-017	Coding	Dubins Trajectories: Geometrically feasible analytical paths.	40
C-018	Coding	Warm Start (A*): Heuristic trajectory proposal as initial guess for NLP.	35
WP5: Inverse Dynamics			
C-019	Coding	Inverse Dynamics: Control vector computation from trajectory discretization.	20
C-020	Coding	Orbit Here: Target entry point trajectory computation.	10
C-021	Coding	Discretization Output: Resolution criteria for autopilot performance.	6
WP6: TFG Writing & Documentation			

Continues on next page...

ID	Type	Detailed Description and Technical Objectives	Hours
TFG-01	Mgmt	Project Charter & Planning: Scope definition and ECTS scheduling.	10
TFG-02	Writing	Technical Body I: Documentation of dynamics and uncoupled/coupled models.	15
TFG-03	Writing	Technical Body II: Adjoint methods, C++ architecture, and sensitivity theory.	15
TFG-04	Writing	Results & Analysis: Discussion on Warm Starts and performance metrics.	14
TFG-05	Writing	Final Review: Formatting, bibliography, and abstract preparation.	10
TOTAL ESTIMATED HOURS (12 ECTS)			415*

*The total estimated hours include a preliminary stage of the project which took place between September 2025 and February 2026. During this period, several activities and code development tasks were completed (approximately until the completion of Work Package 2 and the beginning of Work Package 3). This early start was necessary due to the extensive work required to develop and optimize the algorithm and codebase. Through an internship at CIMNE, these tasks were carried out prior to the official Bachelor's thesis period. The remaining tasks have been scheduled to satisfy the 360 hours corresponding to the 12 ECTS credits.

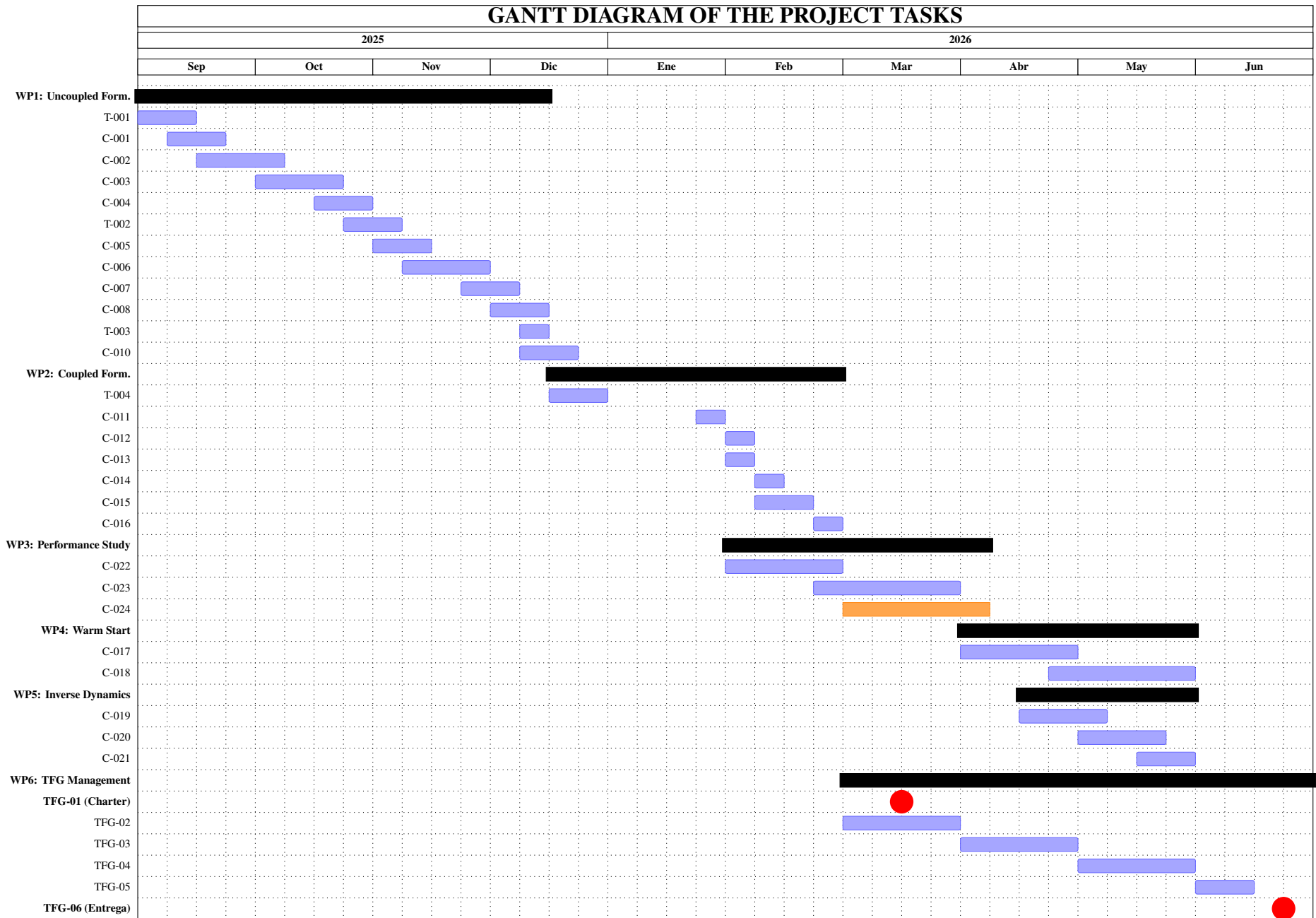


Figure 1.1: Project schedule for the bachelor's thesis and code development, categorized by Work Packages (WPs). *(Own Work)*

Chapter 2

State of the art

The transition of aviation from human-piloted systems to fully autonomous Unmanned Aerial Vehicles (UAVs) has highlighted a fundamental tension between computational geometry and aeronautical reality. In traditional flight management systems, the challenge of navigating an aircraft through three-dimensional space was largely solved through the lens of obstacle avoidance and topological connectivity [13]. However, as the operational envelope of UAVs expands to include aggressive maneuvers, dense urban environments, and complex multi-agent coordination, the limitations of purely geometric and graph-based planning have become critical points of failure. The core of the issue lies in the fact that most ubiquitous path planning protocols rely on simplified kinematic models that treat the aircraft as a point mass or a unicycle-like vehicle, effectively ignoring the transient dynamics, actuator saturation limits, and aerodynamic forces that dictate the actual flight path [14].

2.1 Geometric and Graph-Based Planning

Currently, the vast majority of commercial and proprietary flight management systems (FMS) utilize a hierarchical approach where a high-level planner generates a baseline topological route. These planners typically operate on a discretized representation of the airspace, such as a 3D raster or a visibility graph, to find a collision-free path from an origin to a destination.

The Dijkstra algorithm remains a foundational technique in this domain, providing a single-source shortest-path solution on weighted graphs. By exploring the state space in order of increasing cost, Dijkstra's algorithm guarantees a globally optimal solution in static environments. Nevertheless, the exhaustive nature of the search leads to significant computational overhead, especially in large-scale maps where memory consumption and processing time scale poorly with resolution [13].

To mitigate these computational challenges, the A* algorithm is frequently employed. By integrating a

heuristic function that estimates the remaining cost to the goal, A* can reduce the search space by several orders of magnitude while maintaining optimality guarantees [7]. Despite its efficiency, A* inherits the fundamental limitation of grid-based methods: the resulting path is inherently piecewise-linear and restricted to the connectivity of the graph. In a 3D environment, this often results in a "jagged" trajectory with sharp corners that require infinite acceleration to follow exactly—a physical impossibility for any aircraft. Consequently, while the path is geometrically feasible, it is not "flyable" in its raw state, necessitating a post-processing or smoothing phase that may compromise the original collision-free guarantees [15].

In high-dimensional configuration spaces where grid-based discretization becomes intractable, sampling-based algorithms such as Rapidly-exploring Random Trees (RRT) [8] and Probabilistic Roadmaps (PRM) [16] have gained prominence. RRT algorithms are particularly effective for aircraft because they can integrate differential constraints directly into the tree expansion process, allowing for the discovery of paths that respect minimum turning radii or climb rates. Variants such as RRT* and Informed RRT* focus on refining these paths toward an optimal solution, yet they still struggle with "noise" in the trajectory and the need for expensive post-optimization to eliminate sharp turns and random oscillations.

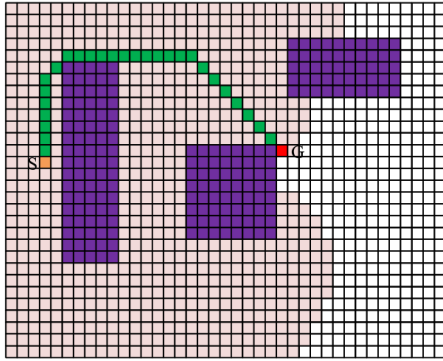


Figure 2.1: Dijkstra algorithm example for NFZ avoidance using 2D grid discretization [13].

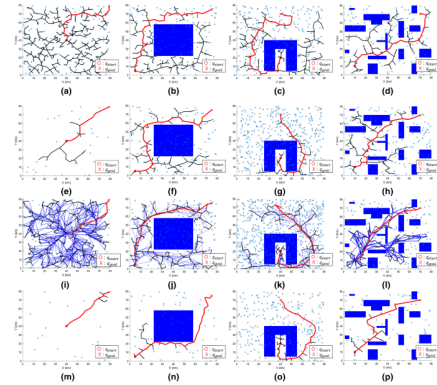


Figure 2.2: Comparative results of various sampling-based RRT algorithms [17].

The differences and trade-offs of these methods are shown in Table 2.1.

Table 2.1: Comparative Analysis of Path Planning Methodologies

Algorithm	Search Mechanism	Optimality	Constraint Handling	Primary Limitation
Dijkstra	Breadth-first / Weighted	Global	Static Obstacles	Computational Complexity
A*	Heuristic-guided	Global (Admissible)	Static Obstacles	Jagged Piecewise-Linear
RRT	Random Sampling	Probabilistic	Nonholonomic	Suboptimal / Randomness
RRT*	Rewiring Random Tree	Asymptotic	Nonholonomic	High Iteration Count
PRM	Probabilistic Roadmap	Probabilistic	Multi-query	Static Map Dependence

2.2 Numerical Trajectory Optimization

While geometric and probabilistic methods provide feasible routes, they often fail to capture the complex kinodynamic constraints of the aircraft, particularly during transient phases of flight where speed and heading change rapidly. To address this limitation, modern Mission Management Systems are increasingly adopting Numerical Trajectory Optimization. This approach formulates the path planning task as an Optimal Control Problem (OCP), which is subsequently transcribed into a finite-dimensional Nonlinear Programming (NLP) problem using direct methods, such as direct collocation or multiple shooting [18].

By utilizing robust Non-Linear Programming (NLP) solvers, it is possible to optimize continuous trajectories while strictly adhering to vehicle dynamics, fuel capacity, and complex spatial constraints, such as dynamic no-fly zones. Among state-of-the-art numerical solvers, Interior Point Methods (IPM) have proven highly effective for large-scale, heavily constrained aerospace applications [19]. Specifically, the open-source solver Ipopt (Interior Point Optimizer) is widely recognized in the literature for its computational efficiency and reliability [20]. To interface with such solvers, modern frameworks often utilize CasADi, an algorithmic differentiation (AD) tool that automates the transcription of Optimal Control Problems (OCPs) into NLP form by computing exact Jacobians and Hessians [21].

However, while CasADi is indispensable for rapid prototyping, it introduces several limitations for high-performance, real-time aerospace applications. Its reliance on symbolic expression trees¹ and an internal virtual machine can lead to significant memory overhead and increased latency compared to hand-optimized, compiled code. Furthermore, in safety-critical environments, the complexity of CasADi's memory management can be a hurdle for deterministic execution. To overcome this, integrating Ipopt directly within a compiled C++ architecture allows the Mission Management System (MMS) to manage memory efficiently and achieve the low-latency performance required for real-time or near real-time mission recalculations [23], offering a significant operational advantage over the semi-interpreted or graph-based environments traditionally used in preliminary academic research.

The implementation of the trajectory optimizer is modular, separating the numerical engine from the vehicle-specific physics. The NLP formulation is generally composed of the following core components:

- **Numerical Solver (IPOPT):** At the heart of the system is the *Interior Point Optimizer*. It utilizes a primal-dual interior-point algorithm to solve large-scale non-linear problems. By leveraging analytical Jacobians and Hessians, it navigates the high-dimensional state space to find a solution

¹A Symbolic Expression Tree is a hierarchical data structure where internal nodes represent mathematical operators (e.g., +, ×, sin) and leaf nodes represent variables or constants. While this allows for Exact Algorithmic Differentiation (AD), navigating these pointer-based structures during function evaluation introduces significant computational overhead compared to the direct execution of compiled, hard-coded arithmetic [22].

that satisfies all constraints while minimizing the objective functional.

- **System Dynamics:** These represent the fundamental "rules of flight" expressed as differential equations (ODEs). In a direct collocation framework, these equations are discretized into equality constraints, ensuring that the transition between state x_k and x_{k+1} is physically consistent with the aircraft's motion model.
- **Environmental Constraints:** These define the "safe" configuration space. They include No-Fly Zones (NFZs), terrain elevation models, and dynamic obstacles. To ensure robustness, these are often modeled as inequality constraints that keep the vehicle within a safe distance from prohibited volumes.
- **Aircraft Performance and Physics:** Beyond basic motion, the solver must respect the structural and aerodynamic limits of the airframe. This includes constraints on maximum thrust, stall speeds, load factor limits (g -load), and maximum bank angles to prevent loss of control or structural failure.
- **Cost Function (Objective):** This defines the "quality" of the trajectory. While many solvers aim for minimum time or minimum fuel, modern flight management systems often prioritize smoothness to protect avionics and actuators. Optimization criteria depend on the specific mission priorities; however, common physical objectives are detailed in Table 2.2.

In practice, a trajectory is rarely optimized for a single criterion. Instead, a composite cost function is constructed as a weighted sum of various objectives, penalties, and soft constraints. For example, a typical objective functional J might take the following form:

$$J = w_t \int_{t_0}^{t_f} 1 \, dt + w_s \int_{t_0}^{t_f} \|\sigma^{(4)}(t)\|^2 \, dt$$

Where:

- w_s and w_t are weight coefficients for **Snap** (smoothness) and **Time** (efficiency if the objective of the simulation is to obtain a *min-time* solution).

Table 2.2: Physical Significance of Trajectory Derivatives in Aerial Planning

Derivative	Name	Physical Significance
$\sigma^{(0)}$	Position	Obstacle clearance and waypoint tracking
$\sigma^{(1)}$	Velocity	Kinetic energy and total travel time
$\sigma^{(2)}$	Acceleration	Required thrust magnitude and lift forces
$\sigma^{(3)}$	Jerk	Rate of change of acceleration (stress on airframe)
$\sigma^{(4)}$	Snap	Angular rates and attitude smoothness

Adopting this approach, though, significantly escalates the problem's difficulty, introducing a higher degree of complexity and a vast increase in the degrees of freedom. Unlike geometric path-finding, the simultaneous integration of dynamic equations and state variables is computationally intensive. Given that Non-Linear Programming (NLP) solvers are hypersensitive to the initial guess, implementing robust seeding and acceleration techniques is essential to meeting the strict convergence windows required for real-time applications.

2.3 Acceleration and Seeding Techniques

While numerical solvers like Ipopt are mathematically robust, they operate on the principle of local descent, making them highly sensitive to the provided initial state. In non-convex environments—specifically those cluttered with No-Fly Zones (NFZs) or terrain obstacles—a "cold start" (such as a simple linear interpolation between waypoints) often causes the solver to converge to a poor local minimum or fail entirely due to a "locally infeasible" solution [24], [25]. To bridge the gap between global discrete planning and local continuous optimization, a hierarchical three-stage pipeline is typically employed:

1. **Global Skeleton Generation:** A graph-based algorithm (e.g., A^* , Dijkstra, or Theta*) is used to compute an obstacle-free spatial skeleton. This ensures that the high-level routing is topologically correct and avoids all static constraints.
2. **Kinematic Refinement:** To prevent the optimizer from wasting iterations smoothing the "jagged" piecewise-linear segments of the skeleton, the path is refined using analytical curves such as *Dubins curves* [26], *Reeds-Shepp paths* [27] or *Clothoids* [28]. By enforcing the minimum turning radius (R_{min}) of the aircraft, this stage transforms the skeleton into a G^1 or G^2 continuous path that respects the vehicle's basic maneuverability.
3. **Inverse Dynamics Mapping:** To bridge the gap between a 3-DOF kinematic guess and a full 6-DOF dynamic solution, *Inverse Dynamics* are employed to estimate initial control surface deflections and thrust requirements. By mapping desired accelerations and curvatures back to the aircraft's actuator space, the solver avoids "tracking divergence" caused by initial attitude (ϕ, θ, ψ) inconsistencies.
4. **NLP Warm-Starting:** The refined kinematic path is sampled to provide the continuous solver with a "warm start." This involves initializing the primal variables—position $\mathbf{p}(t)$, velocity $\mathbf{v}(t)$, and control inputs $\mathbf{u}(t)$ —near their expected optimal values. Furthermore, advanced implementations estimate the dual variables (Lagrange multipliers) associated with the system's constraints, facilitating a rapid satisfaction of the Karush-Kuhn-Tucker (KKT)² optimality

²The KKT conditions represent the mathematical requirements for a point to be considered an optimal solution in nonlinear programming. Beyond ensuring that all physical limits are respected (*Primal Feasibility*), they require

conditions.

This synergy between global geometric seeding and local NLP warm-starting drastically reduces the iterations required for convergence. Within a C++ Ipopt framework, this approach allows for the generation of strictly optimal, flyable profiles within the millisecond-range timeframes required for real-time autonomous missions.

2.3.1 Numerical Structure and Gradient Information

Beyond providing a high-quality initial guess, the convergence rate of a Nonlinear Program (NLP) is fundamentally governed by the accuracy and sparsity of the problem's derivatives. Relying on the solver to estimate gradients via finite differences is computationally expensive and numerically unstable, particularly in high-speed flight regimes where small perturbations in control inputs $\mathbf{u}(t)$ can lead to divergent state trajectories. Therefore, a critical acceleration technique involves providing the analytical Jacobian of the constraints and the Gradient of the cost function.

In the context of **Direct Collocation**, the state and control trajectories are discretized into a large-scale but sparse NLP. By providing the exact sparsity pattern of the Jacobian $\nabla \mathbf{h}(\mathbf{x})$, solvers such as Ipopt can leverage sparse linear algebra to evaluate the KKT conditions efficiently [29]. This sparsity is a direct result of the local nature of collocation polynomials, where a change in a single collocation point only affects neighboring constraints [30].

Conversely, in **Single Shooting** formulations—where only control inputs are decision variables—the resulting numerical problem is smaller but dense [30]. To maintain scalability in these cases, the **Adjoint Method** is frequently employed. By solving the adjoint equations backward in time, the sensitivity of the cost function with respect to all parameters can be computed with a computational cost independent of the number of decision variables. This offers a significant advantage for high-dimensional problems where the mission duration or the number of obstacles would otherwise lead to the "curse of dimensionality." [31], [32]

Furthermore, to handle the "saddle-point" nature of constrained optimization, techniques such as the Uzawa method [33] or augmented Lagrangian iterations are utilized to decouple the primal and dual updates. These methods ensure that the dual variables (Lagrange multipliers) evolve alongside the primal trajectory, facilitating the satisfaction of *Complementary Slackness*—whereby multipliers for inactive constraints are driven to zero [29]. When combined with adjoint-based gradient computation and a warm-start initialization, these numerical strategies ensure that the flight management system remains responsive and capable of real-time trajectory re-planning.

the gradient of the objective function to be perfectly balanced by the gradients of the constraints (*Stationarity*), while ensuring that only active constraints influence the solution (*Complementary Slackness*). Formally: (i) **Stationarity**: $\nabla f(x) + \sum \mu_i \nabla g_i(x) + \sum \lambda_j \nabla h_j(x) = 0$; (ii) **Primal Feasibility**: $g_i(x) \leq 0, h_j(x) = 0$; (iii) **Dual Feasibility**: $\mu_i \geq 0$; and (iv) **Complementary Slackness**: $\mu_i g_i(x) = 0$. [29]

2.4 Synthesis and Research Gap

The progression from discrete graph-based search to continuous numerical optimization reflects the increasing demand for UAV trajectories that are not merely "collision-free," but dynamically feasible and energy-optimal. As established in this review, while global planners like A* and RRT* provide topological robustness, they lack the high-fidelity awareness of aircraft limits such as load factor ($\sigma^{(2)}$) and attitude smoothness ($\sigma^{(4)}$). Conversely, while Nonlinear Programming (NLP) solvers like Ipopt offer the precision required for these "accelerated flight profiles," their heavy dependence on high-quality initial guesses and analytical gradients remains a significant barrier to real-time, deterministic execution in safety-critical environments.

The current literature suggests a clear hierarchy for path planning, yet a critical gap remains in the seamless integration of these layers for high-performance maneuvers. Specifically, the transition from a "jagged" geometric skeleton to a "warm-started" NLP often introduces numerical artifacts or excessive latencies that can render a system unresponsive during dynamic re-planning.

This thesis addresses this gap by developing a high-performance, unified C++ framework that integrates a complete aircraft dynamic model within the Ipopt ecosystem. By moving away from semi-interpreted environments and leveraging direct memory management, the proposed system facilitates the use of advanced warm-starting techniques—specifically the seeding of primal-dual variables derived from kinematically refined skeletons. By optimizing the numerical structure through analytical Jacobians and leveraging the adjoint method for gradient computation, this research demonstrates that the tension between computational geometry and aeronautical reality can be resolved. The result is a mission management architecture capable of generating complex, high-speed trajectories with mathematical certainty within the strict temporal windows required for autonomous operations.

Chapter 3

Methodology

3.1 The challenge of trajectory optimisation with IPOPT

Building an aircraft-trajectory optimiser around a general-purpose nonlinear-programming solver such as IPOPT is not a one-shot procedure [19], [20]. Direct collocation transforms the original continuous-time Optimal Control Problem (OCP) into a large sparse Nonlinear Program (NLP) whose decision vector simultaneously encodes states and controls at every collocation node, whose equality constraints encode the discretised dynamics, and whose inequality constraints encode the entire flight envelope together with the spatial mission constraints [34], [35]. For a realistic aircraft mission the resulting NLP contains thousands of decision variables and constraints and is strongly non-convex¹ [36], [37], with a curvature that is dominated by the aerodynamic model and by the trigonometry of the body-to-wind rotations. IPOPT is a primal-dual interior-point algorithm and is only locally convergent [38], so whether it reaches the global solution, converges to a poor local minimum or simply fails depends almost entirely on how the problem is presented to it—which is precisely the reason why an informed warm-start strategy becomes a first-class design concern [39], [40].

Three properties of the formulation control this outcome and they interact strongly with each other. The first is scaling: the decision variables span many orders of magnitude, from mass measured in tons to control-surface deflections measured in fractions of a radian and from positions measured in kilometres to angles measured in radians, and without a careful non-dimensionalisation of both variables and residuals the interior-point barrier sub-problem becomes ill-conditioned and the quasi-Newton Hessian approximation degrades after a handful of iterations [37], [41]. The second is the quality of the initial guess. An interior-point method spends its early iterations driving the trial point into the interior of the feasible set, and a guess that is dynamically inconsistent or that already violates several path constraints

¹An optimisation problem is non-convex if its objective function or its feasible region does not satisfy the conditions of convexity; in a non-convex NLP a local optimum is not guaranteed to be the global one.

will force the solver into a long detour through infeasibility and very often into a non-physical local minimum, with symptoms such as negative-thrust segments or large bank reversals [24]. The third is sparsity and the availability of exact derivatives. A dense Jacobian obtained by finite differences makes each IPOPT iteration prohibitively expensive on a thousand-variable problem, and the theoretical complexity of the algorithm is only recovered when the sparsity pattern is exposed explicitly and the derivatives are provided exactly, for example through the algorithmic differentiation that CasADi performs on its symbolic expression graph [21], [42].

Because of these three sensitivities the realistic six-degree-of-freedom trajectory-optimisation problem cannot be handed to IPOPT as a single monolithic NLP and expected to converge. The methodology followed in this thesis therefore grows the formulation in complexity one block at a time and fully understands and validates each stage before the next one is enabled. That staged construction is the subject of Section 3.2; the corresponding validation protocol and the warm-start analysis are described in Sections 3.3 and 3.4.

3.2 Three-tier incremental development strategy

The implementation pipeline is organised as a hierarchy of three increasingly faithful formulations. Each tier produces solutions that are directly usable as sanity checks, as warm starts, or as final mission-ready trajectories for the next tier, so that any departure from the expected behaviour can be isolated to the block that was most recently introduced. This block-by-block construction is the standard practice recommended for large-scale direct-collocation solvers [34]. Table 3.1 summarises the role of each tier; the rest of this section discusses them in turn.

Table 3.1: Three incremental tiers of the development methodology. Each row reflects what is solved, why it is built, and how its output feeds the next tier.

Tier	Formulation	Role in the methodology	Output used as
1	Analytical / semi-analytical	Qualitative understanding, preliminary sanity checks	Reference for tier 2
2	Benchmark NLP (UF, minimal constraints)	Solver tuning, weight calibration, debugging	Warm start for tier 3
3	Full constrained NLP (UF $\rightarrow$ FC) with warm start	Mission-ready trajectories under real environment	Final deliverable

3.2.1 Tier 1 — Analytical and semi-analytical exploration

The first tier collects closed-form and semi-analytical solutions of restricted regimes of the point-mass dynamics, namely constant-airspeed cruise, constant-mass climb, equilibrium turn at fixed bank and the Riccati-type mass evolution that arises under a quadratic-drag and quadratic-thrust polar [43],

[44]. These derivations rely on strong simplifying assumptions — no wind, frozen geometry, fixed speed — and they are not part of the final deliverable of this thesis. They are explicitly out of scope as a stand-alone result and only the adjoint outline is reproduced in the appendices for completeness (Annex ??) [32].

Their role was instead methodological. The closed-form expressions produced quasi-stationary reference trajectories against which the early optimiser runs of Tier 2 could be sanity-checked, which made it possible to recognise immediately when IPOPT had converged to a non-physical local minimum simply by comparing the predicted cruise-optimal airspeed or steady-climb angle with the one returned by the solver. The same derivations also clarified the optimality structure of the OCP through the adjoint and Hamiltonian formalism, even though the final solver is direct rather than indirect [45], and they established the ranking of forces, with drag dominating over induced drag at cruise and gravity dominating the energy balance in climb, that later informed the residual-normalisation step of Tier 2. In other words, Tier 1 acted as a scaffold rather than as a result, and its derivations remain in the working notes of the project.

3.2.2 Tier 2 — Benchmark optimal-control problem with minimal constraints

The second tier is the first genuinely numerical formulation. It applies direct trapezoidal collocation to the Uncoupled point-mass dynamics introduced in Section 5.2 [30], [35], reduces the cost function to a pure minimum-time term and keeps only the indispensable bounds active, namely positive mass, positive airspeed, bounded lift coefficient and bounded flight-path angle. The mission boundary conditions are kept deliberately simple — straight-and-level transit or single climb leg between two waypoints in empty airspace — so that the optimal answer is intuitive and can be compared term by term against the Tier 1 references.

The purpose of this tier is not to produce mission-ready trajectories but to regulate the optimiser itself. It is on these benchmark cases that the residual normalisation, the decision-vector scaling, the Jacobian sparsity pattern and the IPOPT options — barrier parameter strategy, ℓ_1 penalty for infeasibility and L-BFGS Hessian approximation — were tuned [19], [20], [41]. The cost-weight α that balances time and fuel (Section 6.3) was first calibrated here in its two pure endpoints, $\alpha = 0$ and $\alpha = 1$, precisely on cases where the qualitative shape of each Pareto extreme is known in advance and where a deviation from the expected control profile is therefore an unambiguous symptom of mis-scaling rather than of a poor solver setting [46].

The validation suite that ships with the thesis is built directly on top of this benchmark formulation: the same four reference scenarios used in Chapter 6 are the natural continuation of the Tier 2 benchmarks, with the constraint set being gradually expanded in Tier 3. The numerical results reported in Section 6.4 thus inherit by construction the solver settings that were tuned at this stage.

3.2.3 Tier 3 — Fully developed problem with external constraints and warm start

The third tier is the final formulation that is intended to be deployed and it adds three blocks of complexity on top of the Tier 2 baseline. The first block is the external environment: the path constraints corresponding to the operational environment are switched on, namely the Digital-Elevation-Model terrain clearance [1], the no-fly-zone exclusion [2], the geofence containment and the intermediate-waypoint hard constraints, all of which are detailed in Chapter 6. The second block is the performance envelope and the regularisation: the aircraft performance bounds ($V_{\max}$, V_{stall} , $C_{L,\max}$, $P_{\max}$ and $h_{\min}$) are enforced, the formulation is promoted from Uncoupled to Full Control (Section 5.3) and the smoothness penalties on control rates and actuator deflections (Section 6.3.2) are enabled [47]. The third block is the warm start: a Hybrid A* path planner [48] followed by an inverse-dynamics reconstruction [3] provides a kinematically feasible initial guess, and the converged UF solution is projected onto the FC state space to seed the rigid-body NLP. The full pipeline is described in Section 6.6 and its empirical performance is studied separately in Section 3.4.

Activating these three blocks one at a time, rather than all together, is what makes the methodology tractable. Each block is regression-tested against the Tier 2 baseline before the next one is enabled, so that any loss of convergence or any drift in the optimum can be attributed to a specific source rather than to an opaque interaction between several newly introduced features at once [49].

3.3 Validation procedure

The validation of the third-tier solver is concentrated in Chapter 6 and is built on the same benchmark scenarios that were used to tune Tier 2. The purpose of this section is to describe what is verified and along which axes; the full numerical results, the plots and the tables are deferred to the corresponding section of the numerical implementation chapter and to the appendices. The four-axis cross-checking protocol described below is the qualitative-then-quantitative discipline that is universally recommended for direct-collocation solutions [34].

Every comparison reported in the validation chapter is run on the same four reference scenarios, SC-00 to SC-03, whose geometric and environmental parameters are listed in Section 6.4.1. The scenarios were designed as a 2×2 factorial in empty airspace, contrasting straight against turning missions and level against climbing ones, precisely so that the effects of the cost function, of the dynamic model and of the constraint set can be isolated rather than mixed. Each scenario is solved twice per formulation, once under the minimum-time criterion and once under the minimum-fuel criterion, which corresponds to the two extremes $\alpha = 0$ and $\alpha = 1$ of the cost-weight introduced in Section 6.3 [46].

A core element of the methodology is that the same problem is solved with two dynamic models of increasing fidelity. The Uncoupled point-mass formulation (UF, Section 5.2) carries three translational degrees of freedom and treats the lift coefficient as a control input, while the Full-Control rigid-body

formulation (FC, Section 5.3) [50] carries the full six degrees of freedom and exposes the elevator, aileron and rudder commands explicitly. Both formulations are introduced in full in Chapter 5; here it is enough to anticipate that running them on the same scenario suite serves two complementary purposes. It cross-validates the simpler model against the higher-fidelity one, since any qualitative disagreement between UF and FC on a benchmark case is in itself a finding that has to be explained, and it quantifies the cost paid in iteration count, wall clock and final objective when the model is promoted from UF to FC. The cross-formulation comparison is reported in Section 6.4.

For each combination of scenario, formulation and criterion the verification proceeds along four redundant axes. The qualitative trajectory inspection (Section 6.4.4) overlays the top-view ground tracks and the altitude profiles of the four combinations of formulation and criterion and checks the expected qualitative signatures, such as the minimum-fuel optimum trading altitude for airspeed on climbing legs or the minimum-time solution saturating the propulsive envelope [43]. The constraint-satisfaction verdict (Section 6.4.5) computes a normalised worst-case margin for every decision variable, every hard path constraint and every collocation defect, with the full per-run numerical margins tabulated in Annex ???. The Pareto trade-off between criteria (Section 6.4.6) compares the minimum-time and minimum-fuel runs of the same formulation head-to-head on flight time, fuel burn, control effort and constraint activity, producing a formulation-specific Pareto endpoint pair. The impact of the regularisation penalties (Section 6.5) finally repeats every run with the smoothness penalties of Section 6.3.2 enabled and disabled, which isolates the cost paid for physically smooth controls and quantifies the corresponding change in the optimum.

The four axes are intentionally redundant. A failure or an unexpected behaviour visible in any one of them is cross-checked against the others before being attributed to a specific source, be it the solver settings, the model fidelity, the constraint conditioning or the penalty calibration. This redundancy is what guards the methodology against the silent failure modes catalogued for direct-collocation transcriptions in recent literature [49].

3.4 Warm-start performance assessment

Once the third-tier solver converges reliably on the validation suite, the next concern of the methodology is computational cost. Interior-point methods are notoriously sensitive to the quality of the initial guess [39], and the difference between a kinematically feasible seed and a naive linear interpolation between waypoints can be of several orders of magnitude in iteration count and in wall-clock time [24]. The warm-start pipeline introduced in Section 6.6 — Hybrid A* path planning [48], followed by inverse-dynamics reconstruction [3] and by the UF $\rightarrow$ FC state projection — is therefore evaluated against a cold start in a dedicated experiment. The cold reference is built by feeding IPOPT a straight-line interpolation between the initial and final waypoints, with constant nominal airspeed and zero attitude; it is the worst credible guess that is still admissible by the bounds, and it is the kind of

guess that a non-specialised user of the solver would supply.

Before any iteration-count comparison is performed the warm-start guess is assessed on its own merits, independently of IPOPT. The trapezoidal collocation defect of the guess — a direct measure of how close the seed already is to a feasible trajectory [35] — is computed and compared against the same defect of the cold-start guess. This step verifies that the warm-start pipeline is producing a useful seed in the first place: a guess whose defect is comparable to the cold one would not justify the additional planning and inverse-dynamics machinery, no matter how much IPOPT seemed to benefit from it downstream.

For every combination of scenario and formulation the comparison then proceeds along four quantitative metrics. The first is the initial-guess quality just described, expressed as a root-mean-square collocation defect of the seed before any IPOPT iteration. The second is the number of primal-dual iterations required to reach the same convergence tolerance under the warm and the cold guess [19]. The third is the wall-clock breakdown of the total solve time, decomposed by pipeline layer (Hybrid A*, inverse dynamics, UF NLP, UF $\rightarrow$ FC projection and FC NLP) so that the cost of the warm-start machinery itself is not hidden in the comparison. The fourth is the sensitivity to the number of collocation nodes, evaluated on a single representative scenario in order to expose the trade-off between discretisation accuracy and solve time as the problem size grows [51]. A complementary strategy comparison runs the same scenarios with several warm-start variants (a straight-line cold start, the Hybrid A* path alone without inverse-dynamics reconstruction, and the full warm-start pipeline) so that the marginal contribution of each block can be quantified rather than merely conjectured.

A point that deserves to be stated explicitly is that the workflow chosen for the production pipeline, namely Hybrid A* $\rightarrow$ inverse dynamics $\rightarrow$ UF $\rightarrow$ FC, is not necessarily the fastest one in every individual scenario. The experimental campaign reported in Chapter 6 shows that on some easy geometries a leaner warm start (for example, the Hybrid A* path used directly without inverse-dynamics reconstruction) can shave a few IPOPT iterations off the FC solve. The reason that pipeline is nevertheless retained as the default is one of robustness: across the full complexity span swept by the experiments, including obstacle-rich and NFZ-rich missions, it is the only configuration that converges in every single case [40]. In the operational context of this thesis a slightly slower but uniformly convergent solver is preferred to a faster solver that occasionally fails, because a failure in production would have to be handled by the mission planner and would erase any average speed advantage on its first occurrence.

The cases selected for the warm-start study are deliberately picked to span the full complexity envelope of the solver. The lower end of the sweep consists of the SC-00 and SC-01 baseline scenarios already used in the validation suite, where even the cold start converges and the comparison is purely about speed. The upper end adds NFZ-rich and obstacle-dense missions in which the cold start either does not converge, converges to a poor local minimum, or takes prohibitively long to do so; in those regimes the warm-start pipeline is what makes the problem tractable at all. This deliberate span of complexity

is what allows the warm-start analysis to report not a single context-free speed-up factor but the full operating range over which the pipeline remains the recommended option. The corresponding empirical results — convergence curves, timing breakdowns, sensitivity plots and strategy-comparison tables — are reported in the warm-start results section of Chapter 6.

Chapter 4

Alternative Solutions and Decision

4.1 Overview of Candidate Approaches

This chapter discusses the main technical alternatives that were considered for each of the three core decisions of the project — (i) the geometric path planner used as warm start, (ii) the trajectory optimisation transcription, and (iii) the aircraft dynamic model that the optimiser sees — and justifies the option finally chosen. The detailed mathematical development of the selected solutions is given in Chapters 5 and 6; here the focus is the decision rationale rather than the formulation itself. Throughout the chapter the reader should keep in mind that the final user of the solver is *Singular Aircraft* [10], whose engineering and delivery requirements (mature codebase, deterministic solve times on embedded hardware, certifiable behaviour at delivery) have acted as a strong filter on the design choices and, in particular, have constrained the time available for the exploration of experimental transcription methods within the scope of this thesis.

4.1.1 Geometric path planner (warm-start producer)

Three families of methods were screened for the role of geometric warm-start producer. The first family is the grid-based shortest-path approach, with Dijkstra, A* and Hybrid A* as its main representatives [7], [48]. These methods are deterministic, optimal on the discretised graph, naturally handle obstacles and no-fly zones, and —most importantly for the present application— Hybrid A* additionally embeds the minimum-turn-radius kinematic constraint, so that the resulting path is already flyable as a first guess for the NLP [48]. The second family is sampling-based planning, principally RRT/RRT* [8] and probabilistic roadmaps [16]. These methods scale better in high dimensions but produce non-deterministic paths whose quality depends on the sampling budget, which is less suited to an offline mission-planning loop that must be reproducible run-to-run. The third family groups the lightweight heuristic seeders — direct potential fields and the like — which are the cheapest at runtime but

cannot guarantee obstacle avoidance and produce un-smooth seeds that destabilise the interior-point solver downstream. Hybrid A* was selected for its determinism, its native obstacle handling and the turn-radius kinematic embedding that makes the warm start dynamically consistent before the NLP is ever called [52].

4.1.2 Trajectory optimisation transcription

Three transcription families were considered for the optimal-control problem itself. The first is the family of indirect methods, in which the Pontryagin maximum principle is used to derive the costate equations and the problem is solved as a two-point boundary-value problem, typically via shooting [32]. These methods provide rigorous first-order optimality conditions and can be very accurate, but they require deriving the costate equations in closed form for every change in the cost function or the dynamic model and are notoriously sensitive to the initial guess of the costates [45]. The second family is direct shooting and multiple shooting, where the controls are discretised and the dynamics are integrated forward in time inside the optimiser [53]. They are easy to set up but the resulting Jacobian is dense in time because every state at the end of the horizon depends on every preceding control, which hurts large-scale convergence and exposes the solver to a known set of transcription-induced failure modes [49]. The third family is direct collocation — trapezoidal, Hermite-Simpson, pseudo-spectral — which decouples state and control via collocation nodes and produces a large but very sparse NLP that is well suited to modern interior-point solvers such as IPOPT and that admits warm starts at the primal-variable level [34], [35].

Direct collocation was chosen as the transcription method for this project, in the simplest trapezoidal variant. The choice is motivated by three reasons that are widely documented in the optimal-control literature and that are particularly relevant for the mission-planning workloads delivered to Singular Aircraft. First, direct collocation produces an NLP whose constraint Jacobian is block-banded and extremely sparse, so that the cost of one IPOPT iteration scales essentially linearly in the number of nodes rather than quadratically as in dense direct shooting [21], [34]. This sparsity is what makes trapezoidal collocation competitive on the thousand-variable problem sizes considered here. Second, direct collocation is robust against discontinuous or only mildly feasible initial guesses, because every node carries its own slack and any infeasibility is absorbed locally instead of accumulating along a shooting integration [30], [51]. This property is what makes the upstream Hybrid A* and inverse-dynamics seeds usable as warm starts in the first place. Third, direct collocation is by now the state-of-the-art transcription for six-degree-of-freedom aerospace trajectory-optimisation problems of the size considered in this thesis: recent work on rocket landing, hypersonic re-entry and unmanned-aircraft mission planning all uses direct collocation in commercial or open-source NLP solvers [45], [49], and the standard reference implementations (CasADi, GPOPS-II, PSOPT) have made the transcription itself essentially routine to set up [19], [21].

A deliberate scope decision is associated with this choice. The mathematical formulation and the

procedural outline of the indirect adjoint method are reproduced in Annex ?? [31], and the direct-single-shooting transcription is summarised at the theoretical level in Chapter 5 [30]; however, *the present thesis has not yet incorporated a working implementation using either of those two formulations*. The only formulation that has been numerically realised end-to-end is direct trapezoidal collocation. The reasons for this are partly technical and partly contractual. On the technical side, single shooting would require a tightly coupled forward integrator with sensitivity propagation that does not reuse the CasADi expression graph already built for the collocation pipeline, and an adjoint formulation would require deriving and verifying the costate equations of the FC model by hand, which is a multi-week effort on its own. On the contractual side, the development of the solver has been bounded by the time and delivery constraints imposed by Singular Aircraft as the final user of the solver [9]: integration into their existing mission-management pipeline, the corresponding test campaign and the deliverable deadlines have all had higher priority than the exploration of additional transcription methods. Single shooting and adjoint methods are therefore identified explicitly as future work in Chapter 7.

4.1.3 Aircraft dynamic model used inside the optimiser

Two dynamic-model fidelities were considered for the inner loop of the optimiser. The Uncoupled point-mass model (UC) is a three-degrees-of-freedom formulation written in wind axes, in which the lift coefficient enters as a direct control input and attitude is not propagated explicitly [43]. It has a compact state vector and is fast to optimise, but it ignores the attitude dynamics and the actuator limits that the autopilot will ultimately have to enforce. The Full-Control rigid-body model (FC) is a six-degrees-of-freedom formulation written in body axes, with explicit elevator, aileron and rudder commands and with actuator-rate limits enforced as path constraints [50]. It is physically realistic and produces trajectories that an autopilot can execute directly, but the state vector and the Jacobian grow accordingly and the NLP becomes harder to converge from scratch. Rather than choose one of the two models, the project uses both in a hierarchy: the UC formulation solves first, on a smaller and better-conditioned NLP, and its converged solution is then used as a structured warm start for the FC formulation, which inherits a near-feasible iterate and only has to refine the attitude states and the actuator commands. This two-stage workflow exploits the strengths of both fidelities and is what makes the FC solve tractable in the wall-clock budgets imposed by the production pipeline [39].

4.2 Decision Summary and Trade-offs

Table 4.1 consolidates the final design decision for each block of the solver together with the dominant trade-off behind it. The chosen pipeline is therefore Hybrid A* path $\rightarrow$ inverse-dynamics seed $\rightarrow$ UC trapezoidal-collocation NLP solved by IPOPT $\rightarrow$ FC trapezoidal-collocation NLP warm-started from the UC solution [19], [35], [48]. A few remarks justify why the table reads the way it does, beyond the specific decision of each row.

Table 4.1: Summary of the alternatives considered and the chosen option for each block of the solver, together with the dominant driver of the decision. The chosen options are mutually consistent and form the pipeline described in Chapter 6.

Block	Chosen option	Main driver of the decision
Path planner / warm start	Hybrid A* + inverse dynamics	Deterministic, obstacle-aware, kinematically feasible seed; robust across the full complexity envelope of the mission set.
Optimisation method	Direct trapezoidal collocation + IPOPT	Sparse NLP, warm-start friendly, robust to discontinuous guesses; state-of-the-art transcription for 6-DOF aerospace problems.
Dynamic model	UC $\rightarrow$ FC hierarchy	Fast and well-conditioned convergence on UC; physical fidelity on FC; FC inherits a near-feasible iterate from UC.
Cost function	Bolza form (time + fuel + regularisation)	Smooth Pareto sweep between minimum-time and minimum-fuel; regularisation absorbs collocation noise without biasing the optimum.

The first remark concerns the choice of direct collocation as the transcription method. As argued in Section 4.1.2, the deciding factors were the sparsity of the resulting NLP, which scales the per-iteration cost linearly with the number of nodes, and the robustness of the method against the only mildly feasible warm-start guesses produced by the upstream Hybrid A* and inverse-dynamics layers [34], [51]. Direct collocation is also the de facto state of the art for six-degree-of-freedom trajectory-optimisation problems of comparable size in adjacent aerospace domains [49], which gives the selected transcription a substantial body of reference implementations to draw on. The thesis explicitly does not deploy direct single shooting or the indirect adjoint method, even though both are formalised in the appendices [32]; the development time was prioritised on the production-grade direct collocation pipeline because of the delivery constraints imposed by Singular Aircraft as the final user of the solver [10].

The second remark concerns the warm-start pipeline. The empirical campaign reported in Chapter 6 demonstrates that the chosen workflow — Hybrid A* $\rightarrow$ inverse dynamics $\rightarrow$ UF $\rightarrow$ FC — is not always the absolute fastest configuration on any single scenario; on the simplest geometries a stripped-down warm start (for example, the Hybrid A* path used directly without inverse-dynamics reconstruction) can save a handful of IPOPT iterations. The reason that the full pipeline is nevertheless adopted as the production default is one of robustness: it is the only configuration in the experimental set that converges in every scenario considered, including the obstacle-rich and NFZ-rich missions where the cold start and the leaner warm-start variants either diverge, converge to a poor local minimum, or hit the iteration limit [24], [40]. For an operational solver that must be handed over to an industrial end user, a slightly slower but uniformly convergent pipeline is preferred over a faster but occasionally

failing one, because in production a single failure cannot be compensated by an average speed-up.

Aspects that were deliberately left out of the present scope — real-time re-planning, MPC closure [12], stochastic wind, multi-aircraft coordination, and the indirect/adjoint and direct-single-shooting transcriptions discussed in the appendices — are identified as future work in Chapter 7.

Chapter 5

Theoretical background

5.1 Problem Statement

The development of a Trajectory Optimizer for a Mission Management System (MMS) requires a rigorous mathematical definition to transition from high-level mission requirements to a flyable numerical solution. This section formalizes the optimization task by first defining the continuous-time Optimal Control Problem (OCP) governing the vehicle's motion, and subsequently detailing its transcription into a discrete, constrained Nonlinear Programming (NLP) problem solvable by computational algorithms.

5.1.1 The Continuous-Time Optimal Control Problem (OCP)

Before introducing numerical discretization, the trajectory generation must be formulated in the continuous time domain, $t \in [t_0, t_f]$. The system's degrees of freedom are governed by two primary, time-dependent vector functions:

- **State Vector** ($\mathbf{x}(t) \in \mathbb{R}^n$): Describes the dynamic condition of the vehicle at any given instant. This typically encapsulates the 3D position, velocity vectors, attitude (e.g., Euler angles or quaternions), and angular rates.
- **Control Vector** ($\mathbf{u}(t) \in \mathbb{R}^m$): Represents the independent input variables commanded by the MMS to manipulate the state trajectory (e.g., thrust magnitude, control surface deflections).

The core objective of the continuous OCP is to determine the optimal control history $\mathbf{u}(t)$ —and potentially the final time t_f —that minimizes a predefined scalar performance metric J , while strictly adhering to the physical laws of motion and operational boundaries. Mathematically, the standard continuous OCP is formulated as a Bolza problem:

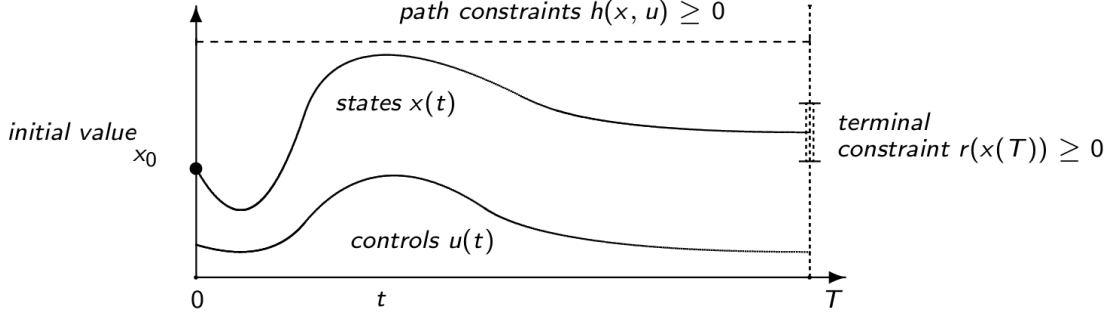


Figure 5.1: Visual representation of the continuous-time Optimal Control Problem (OCP). The optimizer seeks continuous control inputs $u(t)$ that drive the states $x(t)$ from a fixed initial value x_0 to a state satisfying the terminal constraints at time T , while continuously strictly adhering to the defined path constraints. [53]

$$\begin{aligned}
 \min_{\mathbf{u}(t), t_f} \quad & J = \underbrace{\Phi(\mathbf{x}(t_f), t_f)}_{\text{Mayer Term}} + \int_{t_0}^{t_f} \underbrace{L(\mathbf{x}(t), \mathbf{u}(t), t)}_{\text{Lagrange Term}} dt \\
 \text{subject to} \quad & \dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), t), \quad \forall t \in [t_0, t_f] \\
 & \mathbf{x}(t_0) = \mathbf{x}_0 \\
 & \Psi(\mathbf{x}(t_f), t_f) = \mathbf{0} \\
 & \mathbf{h}_{\min} \leq \mathbf{h}(\mathbf{x}(t), \mathbf{u}(t), t) \leq \mathbf{h}_{\max}, \quad \forall t \in [t_0, t_f]
 \end{aligned} \tag{5.1}$$

In this formulation, the components are defined as follows:

- **Cost Function (J):** The metric being minimized. This scalar functional evaluates the overall quality of the generated flight path and is typically expressed in the standard Bolza form, which separates the objective into boundary and continuous components:
 - **Mayer Term (Φ):** Represents the terminal cost or boundary penalty, evaluated exclusively at the final time t_f . This term is designed to optimize discrete conditions at the conclusion of the mission. Typical applications include minimizing the total time of flight (t_f) for time-critical operations, or minimizing the terminal distance/error relative to a specific target landing zone.
 - **Lagrange Term (L):** Represents the running cost, which is integrated continuously over the entire flight horizon from t_0 to t_f . This term penalizes path-dependent behaviors and accumulated costs throughout the mission. It is frequently utilized to minimize total fuel or energy consumption, as well as to penalize aggressive maneuvering (e.g., minimizing acceleration or jerk) to ensure structural safety and passenger comfort.

Then, by assigning weights to different cost function terms, the MMS balances competing mission priorities—like speed versus energy efficiency—to calculate the optimal flight path.

- **System Dynamics ($\dot{\mathbf{x}} = \mathbf{f}$):** This is the governing Ordinary Differential Equation (ODE). It represents the highly non-linear kinematics and kinetics of the aircraft. The optimizer must find a control $\mathbf{u}(t)$ whose resulting integrated state $\mathbf{x}(t)$ perfectly satisfies this ODE at every instant.
- **Boundary Conditions ($\mathbf{x}_0, \Psi$):** Equality constraints anchoring the trajectory. The initial condition ties the solution to the vehicle’s current telemetry, while the terminal condition Ψ dictates the mission’s required end state.
- **Path Constraints ($\mathbf{h}$):** Continuous inequality constraints that enforce safety and structural limits (e.g., maximum load factor, actuator saturation) and spatial boundaries (e.g., terrain avoidance, No-Fly Zones) throughout the entire flight.

5.1.2 Numerical Transcription to Nonlinear Programming (NLP)

Because numerical solvers cannot compute infinite-dimensional continuous functions, the continuous OCP expressed in Equation 5.1 must be discretized. Direct methods “transcribe” the infinite-dimensional control problem into a finite-dimensional NLP by partitioning the continuous time domain $[t_0, t_f]$ into N intervals on a discrete grid.

Depending on how the states $\mathbf{x}(t)$ and controls $\mathbf{u}(t)$ are handled during this discretization, direct methods are broadly categorized into three distinct approaches:

1. **Direct Single Shooting (Sequential Approach):** Only the control variables $\mathbf{u}(t)$ are discretized into a finite set of parameters (e.g., piecewise constant or linear). The states $\mathbf{x}(t)$ are treated as dependent variables, obtained by explicitly integrating the system dynamics ODE ($\dot{\mathbf{x}} = \mathbf{f}$) forward in time using a numerical simulator (like Runge-Kutta, Euler or Trapezoidal).
 - *Advantages:* Results in a very small NLP, as the optimizer only searches the control parameter space. It is relatively simple to implement if a reliable ODE solver is already available.
 - *Disadvantages:* The forward integration makes it highly sensitive to initial guesses, especially in unstable systems (where there is a lot of coupling between the states in different time steps). Furthermore, evaluating continuous path constraints is difficult and computationally expensive.
2. **Direct Multiple Shooting (Hybrid Approach):** The time horizon is split into multiple shorter intervals. Both states and controls are discretized at the boundaries of these intervals (nodes). The ODE is integrated independently within each short interval, and the NLP solver enforces

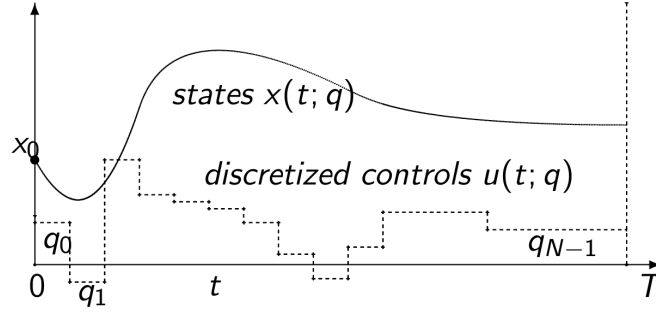


Figure 5.2: Illustration of the Direct Single Shooting transcription method. The infinite-dimensional control trajectory is discretized into a finite set of piecewise constant parameters $q = (q_0, q_1, \dots, q_{N-1})$. The state trajectory $x(t; q)$ is treated as a dependent variable, obtained by integrating the system dynamics forward in time based on the discrete control inputs.[53]

continuity between intervals using algebraic “defect” equality constraints. This limits the propagation of integration errors compared to single shooting.

3. **Direct Collocation (Simultaneous Approach):** Both the control and state histories are fully discretized over the time grid, creating a large, simultaneous parameter space. Instead of using explicit numerical forward integration, the system dynamics ODEs are approximated using polynomial interpolation (e.g., Hermite-Simpson). The governing ODEs are then enforced as algebraic equality constraints (defects) strictly at specific collocation points.

- *Advantages:* Highly robust to poor initial guesses and handles unstable dynamics well, since errors do not compound over time. It is exceptionally efficient at handling path constraints. The resulting NLP is very large but highly sparse, which modern solvers exploit for rapid computation.
- *Disadvantages:* Requires large-scale sparse NLP solvers and is mathematically more complex to formulate than shooting methods.

By applying one of these transcription methods (often Direct Collocation in modern aerospace applications), the continuous problem is reformulated into a discrete parameter optimization task. The optimizer must now find the optimal finite-dimensional **Decision Vector**, $\mathbf{z}$. In a simultaneous approach, this vector typically encompasses all discrete states, controls, and potentially the final time:

$$\mathbf{z} = \left[\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_N, \mathbf{u}_N, t_f \right]^T \in \mathbb{R}^{(n+m)(N+1)+1} \quad (5.2)$$

5.1.3 Constraints and the Lagrangian Formulation

The optimization search space for the decision vector $\mathbf{z}$ is fundamentally restricted by the physical and operational boundaries of the mission. Once discretized, these boundaries are mathematically encoded into two distinct constraint vectors:

- **Equality Constraints** ($g(\mathbf{z}) \in \mathbb{R}^p$): Typically represent the discretized vehicle dynamics (defect constraints) and the necessary initial/terminal boundary conditions.
- **Inequality Constraints** ($h(\mathbf{z}) \in \mathbb{R}^q$): Safeguard the vehicle by enforcing structural limits (e.g., maximum aerodynamic loads) and operational boundaries (e.g., collision avoidance and designated No-Fly Zones).

To solve this constrained optimization problem, Interior-Point solvers, such as Ipopt, transform it into a sequence of unconstrained sub-problems utilizing the **Lagrangian** function, $\mathcal{L}$. This function effectively internalizes the constraints by augmenting the original discrete objective function $F(\mathbf{z})$ with weighted penalty terms. These weights are the Lagrange multipliers (λ for equality constraints and ν for inequality constraints), which act as mathematical restoring forces that pull the solution back into the feasible region whenever a bound is violated.

The Lagrangian is defined as:

$$\mathcal{L}(\mathbf{z}, \lambda, \nu) = F(\mathbf{z}) + \sum_{i=1}^p \lambda_i g_i(\mathbf{z}) + \sum_{j=1}^q \nu_j h_j(\mathbf{z}) \quad (5.3)$$

The solver's primary objective is to identify a stationary point where the gradients of the Lagrangian vanish ($\nabla \mathcal{L} = \mathbf{0}$). Reaching this stationary point signifies that the system has achieved a local mathematical balance, formally described by the Karush-Kuhn-Tucker (KKT) optimality conditions.

5.1.4 Karush-Kuhn-Tucker (KKT) Optimality Conditions

For a solution to be considered a local optimum in a nonlinear programming problem, it must satisfy the KKT conditions. These conditions represent the first-order necessary criteria for optimality and consist of the following principles:

- **Primal Feasibility** ($g(\mathbf{z}) = \mathbf{0}, h(\mathbf{z}) \leq \mathbf{0}$): Ensures that the resulting trajectory is physically possible. It dictates that the solution must strictly respect all discretized environmental boundaries, structural limits, and system dynamics.
- **Dual Feasibility** ($\nu_j \geq 0, \forall j$): Requires that the Lagrange multipliers associated with the inequality constraints be non-negative. This confirms that the gradient of the cost function is properly counterbalanced by the gradients of the active constraints, preventing the optimizer

from moving the solution outside the feasible region to artificially lower the cost.

- **Complementary Slackness** ($\nu_j h_j(\mathbf{z}) = 0, \forall j$): Acts as a logical bridge between the constraints and the cost. It mathematically enforces that an inequality constraint (e.g., a minimum altitude limit) only influences the optimization cost ($\nu_j > 0$) when the vehicle is actively operating exactly on that boundary ($h_j(\mathbf{z}) = 0$). If the constraint is inactive (the vehicle is safely above minimum altitude), its corresponding multiplier must be zero ($\nu_j = 0$).

5.2 Uncoupled Trajectory Dynamics

The Uncoupled (UC) formulation models the aircraft as a point mass whose path is governed directly by the resultant aerodynamic forces, without resolving the internal attitude dynamics [50]. The lift coefficient C_L enters as a direct control input, bypassing the need to compute the angle of attack α from aerodynamic databases or to propagate any rotational equations of motion. The physical justification for this simplification is a separation of timescales: the short-period and roll-subsidence modes of a typical fixed-wing aircraft decay in tenths of a second, whereas trajectory-level changes evolve over tens of seconds to minutes [44]. The optimizer specifies the quasi-steady lift level that an inner attitude control loop is assumed to track instantaneously; consequently, while this simplification remains to be validated against high-fidelity models, it possesses an inherent inner coherence as shown in the comparison between models done in Section 6.4.

The result is a compact formulation that is tractable for real-time or near-real-time mission planning, while retaining the physical accuracy needed to predict fuel consumption and satisfy hard operational constraints.

5.2.1 Reference Frames and Coordinate Systems

Two primary reference frames are used throughout the formulation [50], [54].

1. **Local NED frame** ($\mathcal{F}_{\text{NED}}$). A North-East-Down inertial frame fixed to the Earth's surface. The aircraft position is expressed as $(x, y, z)_{\text{NED}}$, where z is positive downward. Altitude follows as $h = -z$.
2. **Wind-axis frame** ($\mathcal{F}_W$). Centred at the centre of mass, with the x_w axis aligned with the velocity vector $\vec{V}$. In this frame, aerodynamic forces decompose cleanly: drag D acts along $-x_w$ and lift L acts along $-z_w$ (upward in steady level flight).

The orientation of the velocity vector in the NED frame is described by the flight-path angle γ (positive nose-up) and the heading angle ξ (clockwise from North). The bank angle μ rotates the lift vector within the plane perpendicular to $\vec{V}$, enabling lateral trajectory curvature.

Mission-level inputs (waypoints, airspace boundaries) are often specified in geodetic latitude, longitude, and altitude; the optimiser states (x, y, z) are Cartesian offsets in NED relative to a reference point on the WGS-84 ellipsoid. Figure 5.3 sketches the Earth-centred Earth-fixed (ECEF) frame and its relation to geodetic coordinates; Annex ?? documents the complete geodetic–ECEF–NED transformation chain, including practical curvature corrections for long horizontal legs.

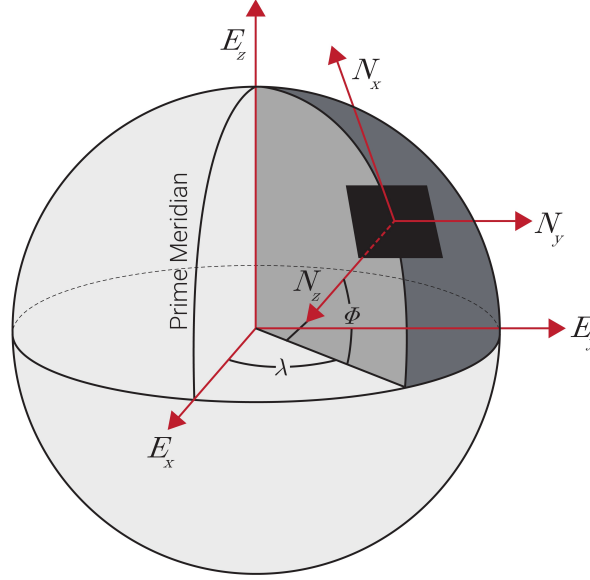


Figure 5.3: Earth-centred Earth-fixed (ECEF) Cartesian frame (X, Y, Z) and geodetic coordinates: latitude ϕ , longitude λ , and ellipsoidal height h along the ellipsoid normal. The local NED frame used in the trajectory model is obtained by projecting the ECEF displacement from a reference point into North, East, and Down axes fixed to that point (see Annex ??).

5.2.2 State and Control Vectors

The UC formulation is defined by a nine-dimensional state vector and a three-dimensional control vector:

$$\mathbf{z}(t) = [x, y, z, V, \mu, \gamma, \xi, m, C]^T \in \mathbb{R}^9 \quad (5.4)$$

$$\mathbf{u}(t) = [T, C_L, p]^T \in \mathbb{R}^3 \quad (5.5)$$

together with the free terminal time t_f as an additional scalar optimisation parameter. Tables 5.1 and 5.2 describe each variable, its physical meaning, and its operating domain.

Table 5.1: State variables of the Uncoupled formulation.

Symbol	Description	Units	Domain
x	Position along North axis (NED)	m	$\mathbb{R}$
y	Position along East axis (NED)	m	$\mathbb{R}$
z	Position along Down axis (NED), $h = -z$	m	$\mathbb{R}$
V	True airspeed (TAS)	m s^{-1}	$[V_{\text{stall}}, V_{\text{max}}]$
μ	Bank angle	rad	$[-\mu_{\text{max}}, \mu_{\text{max}}]$
γ	Flight-path angle	rad	$[-\gamma_{\text{max}}, \gamma_{\text{max}}]$
ξ	Heading angle	rad	$[0, 2\pi)$
m	Aircraft mass	kg	$[m_{\text{dry}}, m_0]$
C	Battery state of charge (electric a/c)	–	$[0, 1]$

Table 5.2: Control variables and optimisation parameter of the Uncoupled formulation.

Symbol	Description	Units	Domain
T	Engine thrust	N	$[0, T_{\text{max}}(h)]$
C_L	Lift coefficient (direct control input)	–	$[C_{L,\text{min}}, C_{L,\text{max}}]$
p	Roll rate (bank angle rate)	rad s^{-1}	$[p_{\text{min}}, p_{\text{max}}]$
t_f	Final time (free parameter)	s	$(0, +\infty)$

5.2.3 Equations of Motion

The equations of motion are derived by applying Newton's second law in the wind-axis frame and projecting the kinematic relations into the NED frame [44], [50]. The full system comprises nine equations, organised below by physical character.

Trajectory Kinematics

The rate of change of position in the NED frame follows from the definition of the velocity vector:

$$\begin{aligned}
 \dot{x} &= V \cos \gamma \cos \xi \\
 \dot{y} &= V \cos \gamma \sin \xi \\
 \dot{z} &= -V \sin \gamma
 \end{aligned} \tag{5.6}$$

These three relations are purely geometric; they hold regardless of the aerodynamic model.

Translational Dynamics

Projecting the forces along the three wind-axis directions and applying Newton's second law, the scalar equations for speed, flight-path angle, and heading angle are:

$$\dot{V} = \frac{T - D - mg \sin \gamma}{m} \quad (5.7)$$

$$\dot{\gamma} = \frac{L \cos \mu - mg \cos \gamma}{mV} \quad (5.8)$$

$$\dot{\xi} = \frac{L \sin \mu}{mV \cos \gamma} \quad (5.9)$$

Equation (5.7) governs the longitudinal energy balance: thrust accelerates the aircraft along the path while drag and the weight component along $\vec{V}$ (equal to $mg \sin \gamma$) oppose it. Equation (5.8) controls the vertical curvature of the path: the lift component $L \cos \mu$, directed normal to $\vec{V}$ within the vertical plane, changes the flight-path angle against the restoring effect of gravity. Equation (5.9) controls the horizontal curvature: the lateral component $L \sin \mu$ turns the heading. These three equations reflect the well-known result that in the absence of sideforce, a banked turn is the only way to change heading in coordinated flight. The full mathematical development of these force equations can be found in Annex ??

Bank Angle Kinematics

The bank angle evolves at the commanded roll rate:

$$\dot{\mu} = p \quad (5.10)$$

Within the uncoupled model, p is treated as a direct control input rather than as a state with its own moment equation. This is consistent with the decoupling assumption: the roll time constant is much shorter than the trajectory timescale, so the optimiser can specify p as if μ tracks any desired value without dynamic lag.

Mass and Energy Dynamics

For a fuel-burning aircraft, the mass decreases at a rate proportional to the propulsive power delivered by the engine. Expressing the specific fuel consumption in $\text{g kW}^{-1} \text{h}^{-1}$:

$$\dot{m} = -\frac{c_{\text{sfc}} \cdot P}{3.6 \times 10^6}, \quad P = \frac{T V}{\eta_p} \quad (5.11)$$

where P is the shaft power in watts and η_p is the propulsive efficiency. For an electrically powered aircraft, $\dot{m} = 0$ and the battery state of charge depletes as:

$$\dot{C} = -\frac{P}{U_{\text{batt}} \cdot 3.6} \quad (5.12)$$

where U_{batt} is the battery voltage in volts and C is in ampere-hours.

5.2.4 Aerodynamic Model

The lift L and drag D forces are computed from the standard aerodynamic definitions using dynamic pressure $q = \frac{1}{2}\rho V^2$:

$$L = q S C_L, \quad D = q S C_D \quad (5.13)$$

The drag coefficient follows the parabolic (quadratic) polar:

$$C_D = C_{D_0} + k C_L^2 \quad (5.14)$$

where C_{D_0} is the zero-lift parasite drag coefficient and k is the induced drag factor, related to the wing aspect ratio $\mathcal{A}$ and Oswald span efficiency e by $k = 1/(\pi e \mathcal{A})$ [43]. Both coefficients are taken from the aircraft aerodynamic database and held constant (no high-lift device corrections at this fidelity level). The air density ρ is computed from an ISA model as a function of altitude $h = -z$ [55].

Table 5.3: Aerodynamic polar parameters used in the Uncoupled formulation.

Parameter	Physical meaning	Typical range (fixed-wing UAV)
C_{D_0}	Zero-lift (parasite) drag coefficient	0.01 – 0.05
k	Induced drag factor	0.03 – 0.12
S	Wing reference area	aircraft-specific [m ²]
e	Oswald efficiency factor	0.70 – 0.95

The parabolic polar is exact for an elliptic lift distribution and remains a good approximation for most moderate-aspect-ratio wings at subsonic speed [43]. Its analytical simplicity is particularly valuable in

the context of trajectory optimisation, as it yields closed-form expressions for the lift-to-drag ratio and the fuel-optimal airspeed.

5.2.5 Summary of the Governing Equations

Table 5.4 collects the nine governing equations of the UC formulation.

Table 5.4: Nine governing equations of the Uncoupled formulation, grouped by physical category.

Category	#	Equation
Trajectory kinematics	1	$\dot{x} = V \cos \gamma \cos \xi$
	2	$\dot{y} = V \cos \gamma \sin \xi$
	3	$\dot{z} = -V \sin \gamma$
Force equations	4	$\dot{V} = (T - D - mg \sin \gamma)/m$
	5	$\dot{\gamma} = (L \cos \mu - mg \cos \gamma)/(mV)$
	6	$\dot{\xi} = L \sin \mu / (mV \cos \gamma)$
Mass evolution	7	$\dot{m} = -c_{\text{sfc}} \cdot P / (3.6 \times 10^6)$
Aerodynamic relations (algebraic)	8	$L = \frac{1}{2} \rho V^2 S C_L$
	9	$C_D = C_{D_0} + k C_L^2$

5.3 Full Control Formulation

The Full Control (FC) formulation extends the UC model by replacing the direct C_L input with physical control-surface deflections and by explicitly propagating both the translational and rotational dynamics in the *body-axis frame* [44], [50]. Unlike the UC model, which operates in the wind-axis frame and treats C_L as a quasi-steady control input, the FC formulation integrates Newton’s and Euler’s equations directly in body coordinates, computing the aerodynamic angles α and β algebraically from the instantaneous body velocities at every integration step. This added fidelity comes at the cost of a substantially larger state space and a tightly coupled system of equations, but it produces trajectories in which every control input corresponds to a physical actuator command that an autopilot can execute directly.

The FC formulation is best suited to situations where attitude dynamics matter — steeper climbs, coordinated turns at high bank angles, or tight manoeuvre sequences — and where a trajectory intended for direct upload to an autopilot must include explicit surface commands.

5.3.1 State and Control Vectors

The body-axis state vector comprises the NED position, the aircraft mass, the three body-axis velocity components, the three Euler orientation angles, the three body-axis angular rates, and the battery state of charge:

$$\mathbf{z}(t) = [x_e, y_e, z_e, m, u, v, w, \phi, \theta, \psi, p, q, r, C]^T \in \mathbb{R}^{14} \quad (5.15)$$

Here (u, v, w) are the projections of the velocity vector onto the body x_b, y_b, z_b axes respectively, and (ϕ, θ, ψ) are the classical roll–pitch–yaw Euler angles that orient the body frame relative to the NED frame. The yaw angle ψ plays the role of the heading ξ of the UC formulation, but is now a proper ODE state driven by the rotational equations rather than a wind-axis angle. The battery state C is omitted for fuel-burning aircraft (where $\dot{m} \neq 0$) and activated for electrically powered ones.

The control vector consists of the engine thrust and the three primary control-surface deflections:

$$\mathbf{u}(t) = [T, \delta_e, \delta_a, \delta_r]^T \in \mathbb{R}^4 \quad (5.16)$$

with t_f retained as a free scalar optimisation parameter, giving five optimisation controls in total.

Tables 5.5 and 5.6 list each variable with its physical meaning and operating domain.

Table 5.5: State variables of the Full Control formulation. All translational and rotational quantities are expressed in the body-axis frame; position is in the NED frame.

Symbol	Description	Units	Notes
x_e, y_e, z_e	NED position (North, East, Down)	m	Integration in NED
m	Aircraft mass	kg	Shared with UC
u	Body-axis forward velocity	m s^{-1}	Replaces V of UC
v	Body-axis lateral velocity	m s^{-1}	New in FC
w	Body-axis normal velocity	m s^{-1}	New in FC
ϕ	Euler roll angle	rad	Replaces μ of UC
θ	Euler pitch angle	rad	Replaces γ of UC
ψ	Euler yaw/heading angle	rad	Replaces ξ of UC
p	Body-axis roll rate	rad s^{-1}	New in FC
q	Body-axis pitch rate	rad s^{-1}	New in FC
r	Body-axis yaw rate	rad s^{-1}	New in FC
C	Battery charge (electric a/c)	Ah	Active only if $c_{\text{sfc}} = 0$

Table 5.6: Control variables and optimisation parameter of the Full Control formulation.

Symbol	Description	Units	Domain
T	Engine thrust	N	$[0, T_{\max}(h)]$
δ_e	Elevator deflection	rad	$[\delta_{e,\min}, \delta_{e,\max}]$
δ_a	Aileron deflection	rad	$[\delta_{a,\min}, \delta_{a,\max}]$
δ_r	Rudder deflection	rad	$[\delta_{r,\min}, \delta_{r,\max}]$
t_f	Final time (free param.)	s	$(0, +\infty)$

5.3.2 The Full-Control Coupled Resolution

The central challenge of the FC formulation — the *full-control coupled dilemma* — is that the aerodynamic forces depend on the instantaneous incidence angles α and β , which are themselves algebraic functions of the body velocities (u, v, w) , which in turn are the output of the translational dynamics driven by those same aerodynamic forces. In the UC formulation this circularity is broken by treating C_L as a free control input; in the FC formulation it is resolved step by step at each collocation node, evaluating the right-hand side of every differential equation in a fixed sequential order. The procedure is summarised in Algorithm 5.4 and detailed in the subsections that follow.

Algorithm 5.4: FC Right-Hand Side Evaluation at Node k

Input: state $\mathbf{z}_k = [x_e, y_e, z_e, m, u, v, w, \phi, \theta, \psi, p, q, r, C]_k$, control $\mathbf{u}_k = [T, \delta_e, \delta_a, \delta_r]_k$, wind $\mathbf{V}_w^{\text{NED}}$

1. **Build rotation matrix.** Compute $\mathbf{R}_{bn}(\phi, \theta, \psi)$ (NED $\rightarrow$ body) and its trigonometric scalars $\{c_\phi, s_\phi, c_\theta, s_\theta, c_\psi, s_\psi\}$.
2. **Rotate wind to body frame.** Project $\mathbf{V}_w^{\text{NED}}$ into body coordinates; compute wind-relative velocities $u_r = u - V_{wb,1}$, $v_r = v - V_{wb,2}$, $w_r = w - V_{wb,3}$ and aerodynamic speed $V_a = \sqrt{u_r^2 + v_r^2 + w_r^2}$.
3. **Compute aerodynamic angles.** $\alpha = \text{atan2}(w_r, u_r)$, $\beta = \arcsin(v_r/V_a)$.
4. **Evaluate aerodynamic coefficients.** Compute $C_L, C_D, C_Y, C_l, C_m, C_n$ from the linear stability-derivative model (Section 5.3.4).
5. **Compute wind-axis forces and moments.** L, D, Y from dynamic pressure; $\mathcal{L}_W, \mathcal{M}_W, \mathcal{N}_W$ from span and chord.
6. **Rotate to body axes.** Apply the (wind $\rightarrow$ body) rotation $\mathbf{R}(\alpha, \beta)$ to obtain body-axis force components $F_{x,a}, F_{y,a}, F_{z,a}$ and moment components L_b, M_b, N_b (including the thrust pitch moment $T d_{Tz}$).
7. **Translational dynamics.** Evaluate $(\dot{u}, \dot{v}, \dot{w})$ from Newton's law in body axes, including gravitational projections and Coriolis cross-products.
8. **Rotational dynamics.** Evaluate $(\dot{p}, \dot{q}, \dot{r})$ from the coupled Euler moment equations with inertial cross-coupling I_{xz} .
9. **Euler angle kinematics.** Evaluate $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ from the kinematic differential equations.
10. **Position kinematics.** Evaluate $(\dot{x}_e, \dot{y}_e, \dot{z}_e)$ by projecting (u, v, w) through $\mathbf{R}_{nb}$ into the NED frame.
11. **Energy/mass evolution.** Evaluate $\dot{m}$ (fuel) or $\dot{C}$ (electric).

Output: $\dot{\mathbf{z}}_k \in \mathbb{R}^{14}$

Figure 5.4: Sequential evaluation of the FC right-hand side at a single collocation node. Steps 2–6 break the aerodynamic coupling: α and β are obtained algebraically from the current body velocities before being fed into the coefficient model and the body-axis force rotation.

5.3.3 External Wind and Aerodynamic Angles

The aircraft's aerodynamic response depends not on the inertial body velocities (u, v, w) but on the velocity of the body *relative to the ambient air* [44]. Given a wind vector $\mathbf{V}_w^{\text{NED}} = (W_N, W_E, W_D)^T$ expressed in the NED frame, its body-axis projection is obtained through the direction-cosine matrix:

$$\begin{pmatrix} V_{wb,1} \\ V_{wb,2} \\ V_{wb,3} \end{pmatrix} = \mathbf{R}_{bn} \mathbf{V}_w^{\text{NED}} \quad (5.17)$$

where $\mathbf{R}_{bn}$ is the 3-2-1 Euler rotation matrix:

$$\mathbf{R}_{bn} = \begin{pmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ s_\phi s_\theta c_\psi - c_\phi s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi c_\theta \\ c_\phi s_\theta c_\psi + s_\phi s_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi c_\theta \end{pmatrix} \quad (5.18)$$

using the shorthand $c_\phi \equiv \cos \phi$, $s_\phi \equiv \sin \phi$, and likewise for θ and ψ . The wind-relative body velocities are then:

$$u_r = u - V_{wb,1}, \quad v_r = v - V_{wb,2}, \quad w_r = w - V_{wb,3} \quad (5.19)$$

from which the aerodynamic speed and incidence angles follow:

$$V_a = \sqrt{u_r^2 + v_r^2 + w_r^2} \quad (5.20)$$

$$\alpha = \text{atan2}(w_r, u_r), \quad \beta = \arcsin\left(\frac{v_r}{V_a}\right) \quad (5.21)$$

Equations (5.21) are purely *algebraic*: α and β are not ODE states — they have no associated differential equations — but are recomputed at every evaluation of the right-hand side from the current values of $(u, v, w, \phi, \theta, \psi)$. This is a key structural difference from some alternative formulations in which α and β are treated as additional dynamic states with their own kinematic differential equations (cf. Section 5.3.12).

5.3.4 Aerodynamic Coefficient Model

The aerodynamic forces and moments are expressed through six non-dimensional coefficients using the standard linear stability-and-control derivative model [44], [50]. Static terms, primary control-surface terms, damping derivatives with respect to non-dimensional angular rates, and a thrust-related drag increment are retained in the formulation below.

Introduce the non-dimensional angular rates (based on the aerodynamic speed V_a , mean aerodynamic

chord $\bar{c}$, and wing span b):

$$\hat{q} = \frac{\bar{c}}{2V_a} q, \quad \hat{p} = \frac{b}{2V_a} p, \quad \hat{r} = \frac{b}{2V_a} r \quad (5.22)$$

Dynamic pressure is

$$q_S = \frac{1}{2} \rho(z_e) V_a^2 S_w \quad (5.23)$$

with $\rho(z_e)$ the ISA air density at altitude $h = -z_e$ [55]. The coefficients are assembled in implementation order: C_L first, then C_D (which may depend on C_L , $\hat{q}$, and thrust T), then the remaining four.

$$C_L = C_{L_0} + C_{L_\alpha} \alpha + C_{L_{\delta_e}} \delta_e + C_{L_q} \hat{q} \quad (5.24a)$$

$$C_D = C_{D_0} + C_{D_{k1}} C_L + C_{D_{k2}} C_L^2 + C_{D_q} \hat{q} + C_{D_{CT}} \frac{T}{q_S} \quad (5.24b)$$

$$C_m = C_{m_0} + C_{m_\alpha} \alpha + C_{m_{\delta_e}} \delta_e + C_{m_q} \hat{q} \quad (5.24c)$$

$$C_Y = C_{Y_0} + C_{Y_\beta} \beta + C_{Y_{\delta_a}} \delta_a + C_{Y_{\delta_r}} \delta_r + C_{Y_p} \hat{p} + C_{Y_r} \hat{r} \quad (5.24d)$$

$$C_l = C_{l_0} + C_{l_\beta} \beta + C_{l_{\delta_a}} \delta_a + C_{l_{\delta_r}} \delta_r + C_{l_p} \hat{p} + C_{l_r} \hat{r} \quad (5.24e)$$

$$C_n = C_{n_0} + C_{n_\beta} \beta + C_{n_{\delta_a}} \delta_a + C_{n_{\delta_r}} \delta_r + C_{n_p} \hat{p} + C_{n_r} \hat{r} \quad (5.24f)$$

Implementation remark. The presence of the non-dimensional angular rates $\hat{p}$, $\hat{q}$, and $\hat{r}$ in Equations (5.24a)–(5.24f) reflects the general linear stability-derivative structure retained at the theoretical level. However, in the numerical examples and case studies reported in Section 6.4, all damping and unsteady derivatives associated with these rates are set to zero in the aerodynamic database following the quasi-steady stability-derivative practice catalogued in the USAF DATCOM [56]. In particular, the unsteady and speed-stability contributions that would be associated with $C_{m_{\dot{\alpha}}}$ and $C_{m_{\dot{u}}}$ are neglected in the implementation, so that the pitching moment reduces in practice to the quasi-steady combination of C_{m_0} , C_{m_α} , $C_{m_{\delta_e}}$, and C_{m_q} around the chosen trim condition.

The drag polar reduces to the UC parabolic form $C_D = C_{D_0} + k C_L^2$ when $C_{D_{k1}} = 0$, $C_{D_{k2}} = k$, and $C_{D_q} = C_{D_{CT}} = 0$. The wind-axis forces and aero moments are then:

$$L = q_S C_L, \quad D = q_S C_D, \quad Y = q_S C_Y \quad (5.25)$$

$$\mathcal{L}_W = q_S b C_l, \quad \mathcal{M}_W = q_S \bar{c} C_m, \quad \mathcal{N}_W = q_S b C_n \quad (5.26)$$

with b the wing span and $\bar{c}$ the mean aerodynamic chord. Table 5.7 lists the principal derivatives with their expected signs for a statically stable aircraft.

Table 5.7: Aerodynamic stability and control derivatives appearing in Equations (5.24a)–(5.24f). Signs in the last column refer to a statically stable, conventionally controlled aircraft.

Coefficient	Brief meaning	Sign
Lift		
C_{L_0}	Lift bias / trim intercept	varies
C_{L_α}	Lift curve slope ($\partial C_L / \partial \alpha$)	+
$C_{L_{\delta_e}}$	Elevator $\rightarrow$ lift	varies
C_{L_q}	Pitch-rate lift derivative	varies
Drag		
C_{D_0}	Zero-lift (parasite) drag	+
$C_{D_{k1}}$	Linear induced-drag term in C_L	+
$C_{D_{k2}}$	Quadratic induced-drag term in C_L^2	+
C_{D_q}	Pitch-rate drag increment	+
$C_{D_{CT}}$	Thrust-related drag increment	+
Side force		
C_{Y_0}	Sideforce bias / trim intercept	≈ 0
C_{Y_β}	Sideslip stability ($\partial C_Y / \partial \beta$)	–
$C_{Y_{\delta_a}}$	Aileron $\rightarrow$ sideforce	varies
$C_{Y_{\delta_r}}$	Rudder $\rightarrow$ sideforce	+
C_{Y_p}	Roll-rate sideforce derivative	varies
C_{Y_r}	Yaw-rate sideforce derivative	varies
Roll moment		
C_{l_0}	Roll-moment bias / trim intercept	≈ 0
C_{l_β}	Dihedral effect ($\partial C_l / \partial \beta$)	–
$C_{l_{\delta_a}}$	Aileron effectiveness in roll	+
$C_{l_{\delta_r}}$	Rudder $\rightarrow$ roll coupling	varies
C_{l_p}	Roll damping ($\partial C_l / \partial \hat{p}$)	–
C_{l_r}	Yaw-rate $\rightarrow$ roll coupling	varies
Pitch moment		
C_{m_0}	Pitch-moment bias / trim intercept	varies
C_{m_α}	Static longitudinal stability	–
$C_{m_{\delta_e}}$	Elevator effectiveness in pitch	–
C_{m_q}	Pitch damping ($\partial C_m / \partial \hat{q}$)	–
Yaw moment		
C_{n_0}	Yaw-moment bias / trim intercept	≈ 0
C_{n_β}	Weathercock stability ($\partial C_n / \partial \beta$)	+
$C_{n_{\delta_a}}$	Aileron adverse yaw	varies
$C_{n_{\delta_r}}$	Rudder effectiveness in yaw	+
C_{n_p}	Roll-rate $\rightarrow$ yaw coupling	varies
C_{n_r}	Yaw damping ($\partial C_n / \partial \hat{r}$)	–

5.3.5 Rotation of Forces and Moments to Body Axes

The aerodynamic forces (L, D, Y) and moments $(\mathcal{L}_W, \mathcal{M}_W, \mathcal{N}_W)$ of Equations (5.25)–(5.26) are expressed in the wind-axis frame. Before entering the body-axis equations of motion they must be projected into body coordinates through the wind→body rotation $\mathbf{R}(\alpha, \beta)$, parameterised by the same incidence angles computed in Section 5.3.3.

Using the shorthand $c_\alpha \equiv \cos \alpha$, $s_\alpha \equiv \sin \alpha$, $c_\beta \equiv \cos \beta$, $s_\beta \equiv \sin \beta$:

$$\begin{aligned} F_{x,a} &= -D c_\alpha c_\beta + L s_\alpha - Y c_\alpha s_\beta \\ F_{y,a} &= Y c_\beta - D s_\beta \\ F_{z,a} &= -D s_\alpha c_\beta - L c_\alpha - Y s_\alpha s_\beta \end{aligned} \quad (5.27)$$

$$\begin{aligned} L_b &= c_\alpha c_\beta \mathcal{L}_W - c_\alpha s_\beta \mathcal{M}_W - s_\alpha \mathcal{N}_W \\ M_b &= s_\beta \mathcal{L}_W + c_\beta \mathcal{M}_W + T d_{Tz} \\ N_b &= s_\alpha c_\beta \mathcal{L}_W - s_\alpha s_\beta \mathcal{M}_W + c_\alpha \mathcal{N}_W \end{aligned} \quad (5.28)$$

The term $T d_{Tz}$ in M_b accounts for the pitching moment generated when the thrust line passes at a vertical offset d_{Tz} from the centre of gravity. It is the only way thrust enters the rotational equations; in all other channels the engine contributes solely to the translational dynamics.

5.3.6 Translational Dynamics

With the body-axis forces known, Newton's second law in the rotating body frame gives the accelerations of each velocity component [50]. Three additional terms appear compared to the inertial-frame equations: the gravitational acceleration projected onto the body axes, and the Coriolis cross-product $\boldsymbol{\omega} \times \mathbf{v}$ arising from the non-inertial frame:

$$\begin{aligned} \dot{u} &= \frac{T + F_{x,a}}{m} - g \sin \theta - (qw - rv) \\ \dot{v} &= \frac{F_{y,a}}{m} + g \cos \theta \sin \phi - (ru - pw) \\ \dot{w} &= \frac{F_{z,a}}{m} + g \cos \theta \cos \phi - (pv - qu) \end{aligned} \quad (5.29)$$

The gravity projections $(-gs_\theta, gc_\theta s_\phi, gc_\theta c_\phi)$ are the components of the gravitational acceleration vector $\mathbf{g}^{\text{NED}} = (0, 0, g)^T$ resolved in the body frame through $\mathbf{R}_{bn}$. The Coriolis terms $(qw - rv)$, $(ru - pw)$, $(pv - qu)$ are the components of $\boldsymbol{\omega} \times \mathbf{v}$ and account for the apparent acceleration due to the rotation of the body frame.

5.3.7 Rotational Dynamics

The angular accelerations $(\dot{p}, \dot{q}, \dot{r})$ follow from Euler's equations for a rigid body [44], [50]. The implementation uses the general form that retains the I_{xz} product of inertia (non-zero for most real aircraft due to wing dihedral and structural asymmetries). Defining the augmented moment components:

$$\begin{aligned}\tilde{L} &= L_b + (I_{yy} - I_{zz}) q r + I_{xz} p q \\ \tilde{M} &= M_b - (I_{xx} - I_{zz}) p r - I_{xz} (p^2 - r^2) \\ \tilde{N} &= N_b + (I_{xx} - I_{yy}) p q - I_{xz} q r\end{aligned}\tag{5.30}$$

and the inertia determinant:

$$\Delta = I_{xx} I_{zz} - I_{xz}^2\tag{5.31}$$

the angular accelerations are obtained by inverting the 2×2 coupled system in $\dot{p}$ and $\dot{r}$:

$$\dot{p} = \frac{I_{zz}\tilde{L} + I_{xz}\tilde{N}}{\Delta}\tag{5.32a}$$

$$\dot{q} = \frac{\tilde{M}}{I_{yy}}\tag{5.32b}$$

$$\dot{r} = \frac{I_{xz}\tilde{L} + I_{xx}\tilde{N}}{\Delta}\tag{5.32c}$$

Note that if $I_{xz} = 0$ (perfectly symmetric aircraft), Equations (5.32a) and (5.32c) reduce to the familiar form $\dot{p} = \tilde{L}/I_{xx}$ and $\dot{r} = \tilde{N}/I_{zz}$. The general form is used here because although the aircraft models in the database have non-negligible I_{xz} , and omitting it would introduce a systematic bias in roll–yaw coupling, the aim of the thesis is to develop a generic optimizer for any kind of fixed-wing aircraft.

5.3.8 Euler Angle Kinematics

The Euler angles (ϕ, θ, ψ) are not force quantities; their evolution is governed by the purely kinematic relation between the body-axis angular rates (p, q, r) and the Euler angle rates $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ [50]:

$$\begin{aligned}\dot{\phi} &= p + \tan \theta (q \sin \phi + r \cos \phi) \\ \dot{\theta} &= q \cos \phi - r \sin \phi \\ \dot{\psi} &= \frac{q \sin \phi + r \cos \phi}{\cos \theta}\end{aligned}\tag{5.33}$$

These equations are the standard 3-2-1 Euler kinematic equations. They contain the well-known gimbal-lock singularity at $\theta = \pm 90^\circ$; in practice, a warning is issued when $|c_\theta| < 10^{-4}$ and the trajectory optimizer enforces a pitch angle path constraint that keeps the solution away from this singularity.

5.3.9 Position Kinematics

The NED position rates follow from projecting the body-axis velocity vector (u, v, w) into the NED frame through the transpose of $\mathbf{R}_{bn}$, i.e. through the body→NED rotation $\mathbf{R}_{nb} = \mathbf{R}_{bn}^T$:

$$\begin{aligned}\dot{x}_e &= c_\theta c_\psi u + (s_\phi s_\theta c_\psi - c_\phi s_\psi) v + (c_\phi s_\theta c_\psi + s_\phi s_\psi) w \\ \dot{y}_e &= c_\theta s_\psi u + (s_\phi s_\theta s_\psi + c_\phi c_\psi) v + (c_\phi s_\theta s_\psi - s_\phi c_\psi) w \\ \dot{z}_e &= -s_\theta u + s_\phi c_\theta v + c_\phi c_\theta w\end{aligned}\tag{5.34}$$

These three equations are the body-axis counterpart of the simple wind-axis kinematic relations $\dot{x} = V \cos \gamma \cos \xi$ of the UC model. They are more expensive to evaluate but are exact for any combination of (ϕ, θ, ψ) without requiring the auxiliary wind-axis angles γ and ξ .

5.3.10 Mass and Energy Evolution

The mass equation is identical in structure to that of the UC formulation. The propulsive power is now computed using the wind-relative axial velocity u_r rather than the inertial airspeed, because u_r is the velocity component that the engine thrust actually does work against:

$$P = \frac{T u_r}{\eta_p}\tag{5.35}$$

For a fuel-burning aircraft considering that the specific fuel consumption is given in g/KWh:

$$\dot{m} = -\frac{c_{\text{sfc}} \cdot P}{3.6 \times 10^6}\tag{5.36}$$

For an electrically powered aircraft ($c_{\text{sfc}} = 0$, $\dot{m} = 0$), the battery state of charge evolves as:

$$\dot{C} = -\frac{P}{U_{\text{batt}} \cdot 3.6}\tag{5.37}$$

where $U_{\text{batt}} = U_{b,\min} + (U_{b,\max} - U_{b,\min}) (C/C_{\text{nom}})$ is the state-of-charge-dependent terminal voltage and C is in ampere-hours. Note that the use of u_r rather than $V_a = \sqrt{u_r^2 + v_r^2 + w_r^2}$ is consistent with

the assumption that the propeller thrust acts primarily along the body x_b axis.

5.3.11 Summary of the Governing Equations

Table 5.9 organises the fourteen differential equations of the FC formulation by physical category.

Table 5.9: Fourteen governing differential equations of the FC formulation. The aerodynamic angles α and β are algebraic quantities; the six aerodynamic coefficient relations (5.24a)–(5.24f) are also algebraic once q_S and the non-dimensional rates (5.22) are evaluated. Including these algebraic relations raises the effective equation count to 22, but only 14 integration states are advanced per time step.

Category	#	Variables evolved
Position kinematics (NED)	3	$\dot{x}_e, \dot{y}_e, \dot{z}_e$
Translational dynamics	3	$\dot{u}, \dot{v}, \dot{w}$
Rotational dynamics	3	$\dot{p}, \dot{q}, \dot{r}$
Euler angle kinematics	3	$\dot{\phi}, \dot{\theta}, \dot{\psi}$
Mass / energy evolution	1	$\dot{m}$ (or $\dot{C}$)
Total (ODE)	14	
Algebraic closures	+8	$\alpha, \beta, C_L, C_D, C_Y, C_l, C_m, C_n$

5.3.12 Comparison of the Two Formulations

The UC and FC formulations represent different balances within the fidelity–complexity trade-off. Table 5.10 summarizes their key structural differences, while Annex ?? provides a complementary derivation-based comparison. This annex clearly demonstrates that the UC formulation is a direct simplification of the complete problem.

Table 5.10: Comparison of the Uncoupled and Full Control formulations. Both solve a Bolza OCP; the columns highlight where the two formulations differ structurally.

	Uncoupled (UC)	Full Control (FC)
Reference frame	Wind axis	Body axis
ODE states	9	14
State variables	$x, y, z, V, \mu, \gamma, \xi, m, C$	$x_e, y_e, z_e, m, u, v, w,$ $\phi, \theta, \psi, p, q, r, C$
Control variables	3+1 $(t_f, T, C_L, \text{auxiliary } p)$	4+1 $(t_f, T, \delta_e, \delta_a, \delta_r)$
Aero angles α, β	Not computed (absorbed into C_L)	Algebraic, from (u_r, v_r, w_r)
Inertia coupling	None (decoupled channels)	Full I_{xz} cross-coupling
Wind model	Implicit (speed V is TAS)	Explicit NED→body rotation
Path constraints*	6	8

* Actuator bounds on individual surfaces are not counted; the listed numbers refer to the operational path constraints (stall, load factor, thrust, sideslip, angular rates, and terrain clearance). The FC formulation adds the pitch singularity avoidance constraint and an explicit sideslip limit absent from the UC model.

The UC model offers a compact, fast-to-solve representation well suited to mission-level planning, warm starting, and analytical approximations (see Annex ??). The FC model resolves the full coupling between control surfaces, attitude, and path that is needed when trajectories are intended for direct autopilot upload or when attitude constraints are binding. In the numerical implementation of Chapter 6 (pipeline overview in Figure 6.17) the UC solution is used as the warm start for the FC optimisation: the body velocities (u, v, w) are initialised from the UC airspeed and heading, and the Euler angles from the UC flight-path and heading angles, through the analytical state projection introduced in Section 6.6.4 and validated in Section 6.4.

Chapter 6

Numerical Implementation of Optimal Trajectory Planning

The theoretical background developed in Chapter 5 established the continuous-time Optimal Control Problem (OCP) and the two flight-mechanics models governing the optimization: the nine-state Uncoupled Formulation (UF) and the fourteen-state Full-Control Formulation (FC). While these models provide a mathematically rigorous description of the vehicle dynamics, they are inherently infinite-dimensional. Because they describe the evolution of continuous functions $\mathbf{x}(t)$ and $\mathbf{u}(t)$ over a closed interval $[t_0, t_f]$, they exist in a functional space that standard numerical optimizers cannot navigate directly.

To bridge this gap, the continuous OCP must be transformed into a finite-dimensional Nonlinear Programming (NLP) problem. This process, known as transcription, involves discretizing the state and control trajectories at specific nodes. However, this transition is not merely a computational formality; it introduces a critical trade-off [49], [51] between:

- **Numerical Fidelity:** Ensuring the discretized solution accurately honors the underlying physics of the UF and FC models.
- **Computational Tractability:** Managing the “curse of dimensionality,” particularly for the 14-state FC model, to ensure the solver converges within a reasonable timeframe.

In this chapter, it is detailed the selection of the *Direct Collocation* method employed to solve the trajectory optimization problem. The chapter starts with the discretization schemes used to handle the differential constraints, followed by the formulation of the algebraic path constraints and objective functions. Finally, it is outlined the algorithmic structure of the NLP solver and the initialization strategies required to ensure robust convergence for both flight-mechanics formulations.

6.1 Numerical Transcription of the Continuous OCP

The continuous OCP defined in equation (5.1) is transcribed into a finite-dimensional NLP through a *Direct Collocation* method [21], [30], [35], [37], [53]. The time horizon $[0, t_f]$ is partitioned into $N - 1$ equal sub-intervals of duration $\Delta t = t_f / (N - 1)$; the continuous state and control histories are then represented by their values at the N resulting collocation nodes $\{t_k\}_{k=0}^{N-1}$. This choice converts the search for two infinite-dimensional functions into the search for a finite, structured decision vector and replaces the continuous-time dynamic constraint $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$ by an algebraic *defect* equation evaluated on each segment.

6.1.1 Decision-Vector Layout

For both formulations the decision vector $\mathbf{z}$ stacks the final time, the state samples, and the control samples in a state-then-control block layout:

$$\mathbf{z} = [t_f, \mathbf{x}_0^\top, \dots, \mathbf{x}_{N-1}^\top, \mathbf{u}_0^\top, \dots, \mathbf{u}_{N-1}^\top]^\top \in \mathbb{R}^{n_z}, \quad (6.1)$$

where the dimension of the state and control blocks differs between the two formulations. Table 6.1 summarises the resulting NLP size as a function of the node count N . The state-then-control ordering produces a block-banded sparsity pattern in the constraint Jacobian: the collocation rows couple only adjacent nodes, while every nodewise constraint (terrain clearance, performance bounds, box bounds) is block-diagonal in the node index. The free final time t_f enters as an additional scalar variable that allows both minimum-time and minimum-fuel objectives to be expressed within the same NLP template.

Table 6.1: Dimension of the decision vector (6.1) for the UF and FC formulations as a function of the node count N .

Formulation	States N_X	Controls N_U	Decision dim. n_z	Equality block
UF (point-mass)	9	3	$1 + 12N$	$9(N - 1)$
FC (rigid body)	14	4	$1 + 18N$	$14(N - 1)$

6.1.2 Trapezoidal Direct Collocation

On each sub-interval $[t_k, t_{k+1}]$ the state trajectory is approximated by a piecewise linear function and the time derivative $\dot{\mathbf{x}}$ by the trapezoidal rule. Requiring the resulting approximation to satisfy the dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ at both endpoints of every segment produces the *collocation defect*

$$\Delta_k = \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2} [\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1})] = \mathbf{0}, \quad k = 0, \dots, N - 2, \quad (6.2)$$

which appears in the NLP as a block of $N_X(N - 1)$ equality constraints. Each component of Δ_k corresponds to a single state variable: position, velocity, attitude, mass, or energy. The defect block (6.2) is structurally identical for the two formulations; what differs is the dynamics function $\mathbf{f}$, which is the nine-state point-mass field for UF and the fourteen-state rigid-body field for FC. The trapezoidal scheme is a $O(\Delta t^2)$ method whose accuracy is governed by the node density along the trajectory [30], [45]; the choice of N is therefore a free parameter of the implementation and is studied in Section 6.6.

6.1.3 Residual Normalisation

The constraint and objective rows of the Nonlinear Programming (NLP) problem combine quantities of heterogeneous physical dimensions, ranging from lengths in metres to force in Newtons and dimensionless angular ratios. Without appropriate preprocessing, the condition number of the constraint Jacobian becomes artificially large. This numerical ill-conditioning causes the descent directions computed by the interior-point algorithm to be poorly scaled, often leading to stalled convergence or termination at a spurious local minimum [34], [57]

To ensure numerical robustness, every residual produced by the implementation is divided by a representative reference scale before being inserted into the NLP. We define a normalised residual vector $\tilde{\mathbf{r}}_i$ as:

$$\tilde{\mathbf{r}}_i = \text{diag}(\mathbf{s}_{\text{ref}})^{-1} \mathbf{r}_i \quad (6.3)$$

where $\mathbf{s}_{\text{ref}}$ is a vector of scaling factors chosen such that every component of $\tilde{\mathbf{r}}_i$ is $O(1)$ irrespective of the underlying units. This implicit preconditioning is independent of the optimiser's internal scaling heuristics [19], [20] and is essential for the robust convergence observed in the case studies presented later.

The specific reference scales used for each state and control variable are derived from the aircraft's physical limits and mission characteristics, as summarised in Table 6.2.

6.2 Constraint Architecture of the Discretised NLP

The discretised optimal control problem solved in this chapter is the augmented Bolza form (extension from Equation (5.1)), in which the Mayer/Lagrange cost is supplemented by a container of *soft-penalty* terms that act on mission-level preferences (intermediate-point tracking, terminal shaping, slack relaxations):

$$J = \Phi(\mathbf{x}(t_f), t_f) + \int_0^{t_f} \mathcal{L}(\mathbf{x}, \mathbf{u}, t), dt + J_{\text{pen}} \quad (6.4)$$

Table 6.2: Reference scaling factors s_{ref} for NLP variables.

Category	Variable(s)	Reference Scale
Translational	Position (x, y, z)	L_{ref} (Total mission distance)
	Velocity (u, v, w)	V_{ref} (Cruise speed / V_{NE})
Attitude	Bank angle (ϕ)	ϕ_{max} (Active limit)
	Pitch angle (θ)	θ_{max}
	Yaw angle (ψ)	2π
Angular Rates	Roll/Yaw rate (p, r)	$2V_{\text{ref}}/b$ (Span-based rate)
	Pitch rate (q)	$2V_{\text{ref}}/\bar{c}$ (Chord-based rate)
Actuation	Control surfaces $(\delta_e, \delta_a, \delta_r)$	δ_{max} (Max deflection)
	Thrust (T)	T_{max}
	Mass (m)	m_{initial}
Temporal	Final time (t_f)	$L_{\text{ref}}/V_{\text{ref}}$

The grouped form of $\mathcal{L}$ and the explicit form of J_{pen} , together with the discretised regularisation weights actually used by the NLP solver, are introduced in Section 6.3.2; the heavy algebra is developed in Section 6.3.2.

At this point, however, the cost functional alone is insufficient to characterise a physically meaningful solution: an unconstrained NLP would freely exploit every degree of freedom in the decision space and produce trajectories that violate gravity, penetrate terrain, or demand thrust levels far beyond any realised powerplant. It is the constraint layer that transforms the discretised cost minimisation into a genuine flight-planning problem.

The constraint architecture is partitioned into four mathematical layers, ordered from the broadest spatial scale to the finest aerodynamic detail:

- **Domain and environment constraints** enforce geometric and map-based feasibility. They prevent the trajectory from intersecting the terrain, entering restricted airspace volumes, or leaving the authorised operational area, and they ensure that prescribed intermediate waypoints are visited in the correct order. These constraints depend only on the position state and are therefore structurally identical in UF and FC, differing only in the numerical values of certain safety margins.
- **Collocation and dynamic consistency constraints** constitute the equality backbone of the NLP. They force the sequence of decision variables to satisfy the equations of motion $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ on every time segment, coupling adjacent nodes and ensuring that the optimised trajectory is dynamically realisable.
- **Boundary equality constraints** anchor the beginning and end of the trajectory to the mission-

prescribed states. They encode the take-off conditions, the landing attitude and position, and optionally any intermediate state fixings required by the mission profile.

- **Performance constraints** confine the aircraft to its aerodynamic and propulsive operating envelope. Because UF and FC represent aircraft motion at fundamentally different fidelity levels, these constraints differ materially between the two formulations: UF constraints are expressed in terms of the scalar airspeed V , while FC constraints operate on the full wind-corrected aerodynamic velocity vector. A detailed comparison of the two sets is presented in Section 6.2.4.

These four layers map one-to-one onto the structural ingredients of the continuous OCP defined in Equation (5.1). The collocation defects (6.2) are the discretised counterpart of the governing ODE $\dot{\mathbf{x}} = \mathbf{f}$; the boundary equality constraints correspond to $\mathbf{x}(t_0) = \mathbf{x}_0$ and $\Psi(\mathbf{x}(t_f), t_f) = \mathbf{0}$; and the domain and performance constraints together populate the path constraint vector $\mathbf{h}$. Once written in finite-dimensional form they are incorporated into the NLP Lagrangian $\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = F(\mathbf{z}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{z}) + \boldsymbol{\nu}^\top \mathbf{h}(\mathbf{z})$ through dual variables $\boldsymbol{\nu} \geq \mathbf{0}$, and the KKT stationarity conditions $\nabla_{\mathbf{z}} \mathcal{L} = \mathbf{0}$ [29], [36] become the numerical target pursued by the interior-point algorithm [19], [38].

Throughout the section the residual-normalisation principle (6.3) is assumed to be applied to every constraint row before insertion into the NLP, with the appropriate reference scale (L_{ref} for positions, V_{ref} for speeds, Scale_ϕ for angles, T_{max} and δ_{max} for controls).

6.2.1 Domain and Environment Constraints

Domain and environment constraints define the spatial region within which a trajectory is admissible. They are assembled from geospatial data — digital elevation models (DEM), restricted-airspace polygons, operational geofences — and from mission waypoint specifications. All constraints in this category act *nodewise*: they are evaluated independently at each of the N collocation nodes $i = 0, 1, \dots, N - 1$, and no coupling between adjacent nodes is introduced. This structure preserves the block-diagonal sparsity pattern of the constraint Jacobian.

1) Terrain Clearance via Digital Elevation Model

The ground surface beneath the planned flight path is represented by a *Digital Elevation Model* (DEM) [1], a gridded raster dataset that associates a terrain height $h_{\text{DEM}}(x, y)$ with every horizontal coordinate pair in the mission area. At query points that fall between grid nodes the height is recovered by bilinear interpolation, providing a continuous and differentiable surface whose gradients are compatible with the analytic Jacobian assembly.

The terrain clearance constraint requires the geometric altitude of the aircraft z_i to exceed the interpolated DEM height by at least a prescribed safety buffer ϵ_{DEM} :

$$z_i \geq h_{\text{DEM}}(x_i, y_i) + \epsilon_{\text{DEM}}, \quad \epsilon_{\text{DEM}} = 5 \text{ m}, \quad \forall i = 0, \dots, N-1. \quad (6.5)$$

The 5 m safety buffer in (6.5) is selected to absorb both the finite resolution of the DEM (500 m voxel size in the use case considered here) and the short-period altitude excursions that may occur between collocation nodes where the dynamics are not explicitly enforced. Under the North-East-Down (NED) sign convention — in which altitude increases with decreasing z — the constraint is implemented in the standard inequality form

$$-z_i - h_{\text{DEM}}(x_i, y_i) \leq 0, \quad (6.6)$$

with one-sided bounds $(-\infty, 0]$.

2) No-Fly Zone Exclusion

No-fly zones (NFZs) represent volumes of airspace in which the operation of unmanned aerial vehicles is forbidden or restricted [2] — examples include airport terminal control areas, military exercise zones, and urban overflights. In the present implementation NFZs are modelled as *extruded polygon prisms*: a horizontal polygon footprint is defined by a set of vertices, and the zone is then extruded vertically between a lower altitude $z_j^{\min}$ and an upper altitude $z_j^{\max}$ to form a closed 3D exclusion volume.

For each zone $j = 1, \dots, N_{\text{NFZ}}$ and each outward-facing polygon face ℓ , an individual signed-distance inequality is generated:

$$d_{ij\ell}(x_i, y_i) \geq \epsilon_{\text{NFZ}}, \quad z_i \in [z_j^{\min}, z_j^{\max}], \quad \forall i, \quad (6.7)$$

where $d_{ij\ell}$ denotes the perpendicular distance from node i to face ℓ of zone j , measured positively outward. The altitude condition activates the constraint only when the node lies within the vertical extent of the zone, ensuring that the formulation is not penalised for flying over or under an NFZ that does not reach the current flight level.

The safety margin ϵ_{NFZ} is set in both formulations to

$$\epsilon_{\text{NFZ}}^{\text{UF}} = 25 \text{ m}, \quad \epsilon_{\text{NFZ}}^{\text{FC}} = 25 \text{ m}. \quad (6.8)$$

3) Geofence Containment

While NFZs exclude the trajectory from specific volumes, the *geofence* performs the complementary role of confining the trajectory to an authorised operational area. Structurally, the geofence is identical to an NFZ with reversed sign: rather than requiring the aircraft to remain *outside* a polygon, the aircraft must remain *inside* the geofence boundary at all times.

The feasibility condition is expressed as

$$\mathbf{r}_i \in \Omega_{\text{GEO}}, \quad i = 0, \dots, N - 1, \quad (6.9)$$

where Ω_{GEO} denotes the interior of the geofence polygon prism. The constraint reuses the polygon-distance infrastructure of (6.7) with reversed sign on each face. A common inward safety margin $\epsilon_{\text{GEO}} = 5$ m is applied in both UF and FC, ensuring that the trajectory does not graze the boundary.

4) Intermediate Waypoints

Many missions require the trajectory to pass through or over a series of prescribed intermediate points — inspection targets, relay stations, or airspace entry/exit gates — before reaching the terminal state. These conditions are encoded as *waypoint constraints*, which impose simultaneous requirements on position, attitude, and speed at specific points along the trajectory.

To avoid introducing non-smooth conditional logic into the NLP, each waypoint condition is enforced through an *auxiliary arc-length slack variable* $s_{wp,k} \geq 0$ that is appended to the optimisation vector. The three coupled conditions per waypoint k are:

1. **Position matching.** The Euclidean distance between the trajectory position and the prescribed waypoint location must not exceed a small tolerance:

$$\left\| \mathbf{r}(s_{wp,k}) - \mathbf{r}_{wp,k}^{\text{ref}} \right\|_2 \leq \epsilon_{wp}.$$

The arc-length parameter $s_{wp,k}$ is determined implicitly by the optimiser as the closest point on the trajectory to the waypoint.

2. **Attitude matching.** Heading, bank angle, and pitch are required to match the prescribed waypoint attitude, with residuals normalised by the respective attitude reference scale.
3. **Speed matching.** The local airspeed at the waypoint must equal the prescribed speed, with the residual normalised by V_{ref} in UF or by the corresponding body-velocity scale in FC.

The slack-variable formulation ensures that the waypoint constraint degrades gracefully under infeasibil-

ity: if the prescribed point cannot be reached exactly, the optimiser converges to a minimum-violation solution rather than declaring the problem infeasible outright.

6.2.2 Collocation and Dynamic Consistency Constraints

The domain constraints of the previous subsection confine the trajectory spatially, but they say nothing about *how* the aircraft moves between nodes. It is the collocation constraints that enforce the physical laws of motion along the entire trajectory.

The trapezoidal direct-collocation scheme introduced in (6.2) [30], [34] divides the time horizon $[0, t_f]$ into $N - 1$ equal segments of duration $\Delta t = t_f / (N - 1)$. On each segment $[t_k, t_{k+1}]$, the state trajectory is approximated by a piecewise linear function and the time derivative is approximated by the trapezoidal rule. Requiring this approximation to satisfy the equations of motion $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ at both endpoints of every segment gives rise to the defect vector

$$\Delta_k = \mathbf{x}_{k+1} - \mathbf{x}_k - \frac{\Delta t}{2} [\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1})] = \mathbf{0}, \quad k = 0, \dots, N - 2. \quad (6.10)$$

Each component of the defect Δ_k corresponds to one state variable: positions, velocities, angles, mass, and battery charge. Setting all components to zero — as required by the equality constraint with zero bounds — is the mechanism through which Newton’s laws, aerodynamic lift and drag, and propulsive thrust are all embedded into the optimisation. Without this constraint block, the decision variables at adjacent nodes would be completely decoupled.

The defect equation (6.10) is the same in structure for both formulations, but the dynamics function $\mathbf{f}$ differs: it is the nine-state point-mass field in UF and the fourteen-state rigid-body field in FC. Each residual component is normalised by its corresponding reference scale before insertion into the NLP, ensuring that the Jacobian rows associated with positional, translational and angular states are all of comparable magnitude.

6.2.3 Boundary Equality Constraints

The collocation constraints enforce internal consistency, but they do not by themselves specify where the trajectory starts or ends. Boundary equality constraints anchor the first and last nodes to the mission-prescribed initial and terminal conditions.

Initial State

At node $k = 0$, all state components are fixed to the take-off conditions specified in the mission file:

$$\mathbf{x}_0 = \mathbf{x}^{\text{init}}. \quad (6.11)$$

This is implemented as a block of equality constraints with zero-tolerance bounds $[0, 0]$ on each normalised residual, effectively removing the initial state from the optimisation and replacing it with a fixed parameter.

Final State

At the terminal node $k = N - 1$, the landing position and attitude are enforced as hard equalities:

$$\mathbf{r}_{N-1} = \mathbf{r}^{\text{end}}, \quad \boldsymbol{\eta}_{N-1} = \boldsymbol{\eta}^{\text{end}}, \quad (6.12)$$

where $\boldsymbol{\eta}$ represents the attitude sub-vector: (ϕ, θ, ψ) in FC or (μ, γ, ξ) in UF. Terminal speed and mass may optionally be fixed to prescribed values as well, depending on the mission configuration.

6.2.4 Performance Constraints

Performance constraints confine the aircraft to its aerodynamic and propulsive operating envelope. Unlike the spatial constraints of Section 6.2.1, which depend only on position, performance constraints couple speed, altitude, mass, and — in the rigid-body formulation — attitude and angular rates. Their mathematical form therefore differs between the two formulations.

In UF all performance limits are expressed in terms of the scalar airspeed V . In FC the governing quantity is the *wind-corrected aerodynamic velocity*

$$\mathbf{V}_{\text{aer},i} = \begin{pmatrix} u_i \\ v_i \\ w_i \end{pmatrix} - R_{EB}(\phi_i, \theta_i, \psi_i) \mathbf{V}_{\text{wind},i}, \quad V_{\text{aer},i} = \|\mathbf{V}_{\text{aer},i}\|, \quad (6.13)$$

where $R_{EB} \in SO(3)$ is the Earth-to-body rotation matrix and $U_{r,i} = \mathbf{V}_{\text{aer},i} \cdot \hat{\mathbf{e}}_x$ is the forward wind-relative component on the propeller axis. Using $\mathbf{V}_{\text{aer}}$ instead of the raw body velocity ensures that lift, drag, and thrust limits are evaluated in the correct aerodynamic reference frame regardless of the ambient wind.

Table 6.3 provides a side-by-side comparison of all performance constraints. A “—” entry indicates that the constraint is not applicable to that formulation.

Table 6.3: Performance constraints enforced at each collocation node ($i = 0, \dots, N - 1$) for the UF ($N_X = 9$) and FC ($N_X = 14$) formulations.

Constraint	UF — Point-Mass	FC — Rigid Body
Anti-Stall Margin	$\frac{V_i}{V_{\text{stall}}(h_i, m_i)} \geq 1$ Scalar airspeed bounded by mass-corrected stall speed.	$\frac{V_{\text{aer},i}}{V_{\text{stall}}(h_i, m_i)} \geq 1$ Wind-relative speed used to prevent aerodynamic stall.
Power Availability	$\frac{T_i V_i}{\eta_p P_{\text{max}}(h_i)} \leq 1$ Propulsive power limited by altitude-derated shaft power.	$\frac{T_i U_{r,i}}{\eta_p P_{\text{max}}(h_i)} \leq 1$ Power computed using forward body-velocity U_r.
Structural Limit	$V_i \leq V_{NE}$ Box bound on scalar airspeed.	$\frac{\ \mathbf{V}_{b,i}\ _2}{V_{NE}} \leq 1$ Norm of the full body-frame velocity vector.
C_L Saturation	<i>Implicit (Direct Control)</i> Enforced via box bounds on the C_L control variable.	$C_L(\alpha_i, \delta_{e,i}) \in [C_{L,\text{min}}, C_{L,\text{max}}]$ Nonlinear mapping of AoA and elevator deflection.
Min. Altitude	$\frac{-z_i}{h_{\text{min}}} \geq 1$ Absolute floor to avoid ground-effect or impact.	$\frac{-z_i}{h_{\text{min}}} \geq 1$ Absolute floor to avoid ground-effect or impact.
Control Surfaces	<i>N/A (Kinematic)</i> Roll rate is the primary control.	$ \delta_e , \delta_a , \delta_r \leq \delta_{\text{max}}$ Physical limits on aerodynamic actuators.
Angular Rates	$ p_{\text{rate},i} \leq p_{\text{max}}$ Bound on single-axis roll kinematics.	$ p_i , q_i , r_i \leq (\cdot)_{\text{max}}$ Individual bounds on all three body-frame rates.

6.2.5 Summary of the Constraint Architecture

The complete constraint architecture of the discretised NLP reflects a deliberate balance between physical fidelity and computational tractability. All residuals are normalised to $O(1)$ before being handed to the optimiser, and the constraint Jacobian inherits the block-sparse structure imposed by the nodewise and segmentwise formulations.

The two formulations share the domain constraints (terrain, NFZ, geofence, waypoints), the collocation

defect equations, and the boundary equalities. They diverge in the performance layer, where the rigid-body FC model requires wind-corrected speed definitions, an explicit C_L -saturation constraint, a minimum-altitude floor, and individual actuator bounds — none of which are necessary in the point-mass UF model.

Together, these four constraint layers populate the equality block $\mathbf{g}(\mathbf{z}) = \mathbf{0}$ (collocation defects, boundary equalities) and the inequality block $\mathbf{h}(\mathbf{z}) \leq \mathbf{0}$ (domain, box and performance constraints) of the discretised OCP. The two blocks, together with the cost functional defined in Section 6.4 for UF and Section 6.4 for FC, constitute the full mathematical specification of the NLP solved throughout the remainder of this chapter.

6.3 Cost Function Definitions

While a generalized NLP cost function was introduced in Section 6.2, both the Uncoupled Flight (UF) and Fully Constrained (FC) formulations require distinct modifications to tailor the objective to their specific kinematic and dynamic constraints. Despite these individual variations, both formulations preserve a core Bolza-form structure, augmented by the numerical penalties detailed in Section 6.3.2.

This shared Bolza-form objective consists of a Mayer term that penalizes the final time and a Lagrange term that integrates propulsive energy along the trajectory. The resulting template is flexibly adapted to represent two limiting operational regimes—the minimum-time and minimum-fuel objectives—by zeroing the complementary weighting factors. Accordingly, this section formalizes the mathematical framework for the base cost function (Section 6.3.1) and details the smoothness regularization used to stabilize the problem at the NLP level (Section 6.3.2).

6.3.1 Objective Function: The Time–Fuel Trade-off

The continuous-time Bolza objective function is transcribed into a discrete Non-Linear Programming (NLP) framework using the trapezoidal collocation grid. This mathematical architecture splits the optimization criteria into two distinct physical components:

- **The Mayer Term:** This term evaluates only the final snapshot of the trajectory. It isolates the total final time t_f , scaled by the weight w_{t_f} . Minimizing this term forces the solver to treat the flight path like a drag race.
- **The Lagrange Term:** This term accumulates across the entire flight duration. It integrates instantaneous propulsive power—calculated as the product of thrust and velocity ($T \cdot V$)—approximated via a discrete trapezoidal sum. Minimizing this integral directly optimizes fuel conservation.

To balance these competing operational demands, a scalar weighting parameter $\alpha \in [0, 1]$ is introduced

as a control dial. The mathematical actions and resulting aircraft behaviors under this parameterization are summarized in Table 6.5.

Table 6.5: Operational regimes driven by the scalar weight α .

Setting	Objective	Mathematical Action	Flight Behavior
$\alpha = 0$	Min-Time	Minimizes t_f ($w_{t_f} > 0$)	Max thrust ($T_{\max}$), shortest path
$\alpha = 1$	Min-Fuel	Minimizes power sum ($T \cdot V$)	Max efficiency (L/D), saves fuel
$0 < \alpha < 1$	Pareto	Blended compromise	Sweet spot: speed vs. throttle

It is critical to emphasize that both the Unconstrained Flight (UF) and Fully Constrained (FC) tracking formulations share an identical constraint set (e.g., identical obstacle boundaries and aerodynamic limits). Consequently, if the two formulations produce differing converged trajectories under the exact same value of α , the variance is not caused by differing physical laws. Instead, it is driven solely by the specific subset of decision variables that each formulation exposes to the NLP solver, with the UF model occasionally exploiting coordinate shortcuts hidden from its attitude-restricted counterpart.

Throughout this section the trapezoidal mid-node average and the forward increment are denoted

$$\bar{X}_k = \frac{1}{2}(X_k + X_{k+1}), \quad \Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k.$$

Both formulations are augmented by a smoothness-regularisation term that penalises node-to-node oscillations of the controls and of selected state components. This term is kept abstract as $J_{\text{pen}}^{\text{UF}}$ and $J_{\text{pen}}^{\text{FC}}$ in Sections 6.3.1–6.3.1 and is developed in vectorised form in Section 6.3.2. Its numerical motivation, the per-aircraft weight-selection protocol and the role of the gates β_c and β_s are covered in Section 6.3.2 and Annex ??.

Uncoupled formulation (UF)

The UF treats the aircraft as a point mass and uses the state and control vectors introduced in Section 5.2.2 of Chapter 5. The state $\mathbf{x}_{\text{UF}}$ in Equation (5.4) contains the position, the scalar airspeed V , the bank angle μ , the flight-path and heading angles (γ, ξ) , the mass m and the energy state C ; the control vector $\mathbf{u}_{\text{UF}}$ in Equation (5.5) collects the thrust T , the lift coefficient C_L and the bank-rate command p . The propulsive power $T V$ entering the Lagrange term is therefore evaluated with the *scalar airspeed* V taken directly from the UF state.

Putting Mayer, Lagrange and regularisation together, the UF cost reads

$$J_{\text{UF}} = w_{t_f} t_f + \sum_{k=0}^{N-2} \bar{T}_k \bar{V}_k \Delta t + J_{\text{pen}}^{\text{UF}}, \quad (6.14)$$

where the smoothness term $J_{\text{pen}}^{\text{UF}}$ is expanded in Equation (6.16) below.

Full-control formulation (FC)

The FC resolves the rigid-body dynamics introduced in Section 5.3 of Chapter 5. The state $\mathbf{x}_{\text{FC}}$ in Equation (5.15) expands the UF state with the body-frame velocities $\mathbf{V}_b = (u, v, w)$, the Euler angles (ϕ, θ, ψ) and the body-frame angular rates (p, q, r) , while the control vector $\mathbf{u}_{\text{FC}}$ in Equation (5.16) replaces the UF aerodynamic-input set by the thrust T and the three physical actuator deflections $(\delta_e, \delta_a, \delta_r)$.

The defining departure from the UF cost is the velocity used to compute the propulsive power. The FC state does not expose a scalar airspeed directly; the Lagrange term is therefore built from the *body-frame velocity norm* $\|\mathbf{V}_b\|_2 = \sqrt{u^2 + v^2 + w^2}$ that already drives the mass-flow equation of the FC (Equation (5.35)). In the no-wind validation suite (Section 6.4.1) $\|\mathbf{V}_b\|_2$ coincides numerically with the airspeed magnitude, so the two costs remain commensurable on a per-mission basis; algebraically, however, the input is distinct and reflects the level of fidelity at which each formulation samples the trajectory.

Carrying this substitution into the Bolza template gives

$$J_{\text{FC}} = w_{t_f} t_f + \sum_{k=0}^{N-2} \bar{T}_k \overline{\|\mathbf{V}_b\|_k} \Delta t + J_{\text{pen}}^{\text{FC}}, \quad (6.15)$$

where the overline on the norm denotes the same trapezoidal mid-node average as in the UF and $J_{\text{pen}}^{\text{FC}}$ is expanded in Equation (6.17) below.

6.3.2 Cost regularisation: smoothness penalties

The Bolza cost J in (6.4), introduced in Section 6.2, already exposes a family of soft-penalty terms J_{pen} . In the discretised NLP, these terms are joined by a second family of *cost-level regularisation* weights that act on first differences of the controls and on a small subset of the state. Their algebraic form is collected in Section 6.3.2 (Equations (6.16) and (6.17)); the present subsection is devoted to the rationale. The regularisation is not part of the underlying optimal control problem: it is a numerical device [36], [47] that compensates for two artefacts of direct collocation with a finite-difference Jacobian.

Numerical motivation

Two empirical phenomena motivate the introduction of J_{pen} :

- **Finite-difference Jacobian noise.** The pipeline computes the constraint Jacobian by forward differences. The truncation error scales as $\sqrt{\varepsilon_{\text{mach}}} \approx 10^{-8}$ in the best case, and in practice closer to 10^{-6} once the compound nonlinearities $\rho(h)$, $C_D(C_L)$ and $T \cdot V$ propagate [42], [58]. At the noise floor the KKT residual oscillates and IPOPT cannot certify convergence. The quadratic

increment penalties shift the gradient locally, smoothing the search direction enough for the line search to make progress.

- **High oscillating control chatter.** Without $w_{\Delta(\cdot)}$ the optimiser is free to switch the controls between adjacent bounds at adjacent nodes, since the underlying continuous OCP has no rate limit. This produces physically uninstantiable trajectories (pulse-like δ_e , saw-tooth thrust) that pass collocation but cannot be flown. The regularisation acts as an implicit rate limit, comparable to—but cheaper than—introducing $\dot{u}$ -bounds explicitly.

The regularisation block penalises inter-node differences (and, for γ , the absolute value) on the same uniform node grid used for the collocation defects (6.2); it does not replace any path inequality or mission-level soft constraint.

Weight selection procedure

The weights are tuned per aircraft following the standard practice in multi-objective NLP weighting [46]. The procedure, described in detail in Annex ??, is heuristic: starting from $w_{(\cdot)} = 0$, each weight is raised in geometric steps until two conditions hold simultaneously,

1. the total variation $\text{TV}(u) = \sum_k |\Delta u_k|$ of the control history drops below an acceptable threshold (typically an order of magnitude below the unregularised value); and
2. The regularisation contribution $P_c + P_s$ at the converged solution remains below $\sim 15\%$ of J_{total} , which ensures the solver converges to a similar local minimum. If the penalty term were too high relative to the base cost, it would dominate the objective function, introducing significant errors when minimising the primary cost functional (e.g., minimum time or minimum fuel consumption).

The nominal split of the objective into time, fuel, regularisation ($P_c + P_s$), and baseline terms at the converged iterate is the auditing quantity used to check condition (2).

Regularisation penalty terms

The placeholders $J_{\text{pen}}^{\text{UF}}$ and $J_{\text{pen}}^{\text{FC}}$ that close Equations (6.14) and (6.15) share a common architecture. Each is the sum of two families:

- a **control-increment family** that penalises node-to-node jumps of the control vector and is activated by a global gate $\beta_c \in [0, 1]$;
- a **state regularisation family** that penalises node-to-node jumps (and, for γ , the absolute value) of a small subset of the state and is activated by a second global gate $\beta_s \in [0, 1]$.

The two gates are convenience switches at the NLP level: the production configuration is $\beta_c = \beta_s = 1$,

while the ablation $\beta_c = \beta_s = 0$ recovers a pure Bolza objective and is used in the with-/without-regularisation comparison of Section 6.5. Mixed settings make either family inspectable in isolation.

UF formulation This formulation penalizes three control variables: thrust, lift coefficient (C_L), and roll rate (p). For the state variables, penalties are applied strictly to the derivatives of the flight-path angle and the bank angle. In this specific scenario, penalizing the control variable p is functionally equivalent to penalizing the derivative of the bank angle state. Although both approaches were evaluated, weighting the penalty of the finite difference terms was chosen as it is more straightforward to tune.

$$J_{\text{pen}}^{\text{UF}} = \beta_c \sum_{k=0}^{N-2} \|\xi_{c,k}^{\text{UF}}\|_{\mathbf{W}_c^{\text{UF}}}^2 + \beta_s \sum_{k=0}^{N-2} \|\xi_{s,k}^{\text{UF}}\|_{\mathbf{W}_s^{\text{UF}}}^2, \quad (6.16)$$

with the per-family regularisation vectors and their diagonal weight matrices given by

$$\xi_{c,k}^{\text{UF}} = \begin{bmatrix} \Delta T_k & \Delta C_{L,k} & \Delta p_k \end{bmatrix}^T,$$

$$\mathbf{W}_c^{\text{UF}} = \text{diag}(w_{\Delta T}, w_{\Delta C_L}, w_{\Delta p}),$$

$$\xi_{s,k}^{\text{UF}} = \begin{bmatrix} \Delta \gamma_k & \Delta \mu_k \end{bmatrix}^T,$$

$$\mathbf{W}_s^{\text{UF}} = \text{diag}(w_{\Delta \gamma}, w_{\Delta \mu}).$$

FC formulation In contrast, the full-control (FC) formulation penalizes a broader set of control variables, specifically the aerodynamic surface deflections (δ). Additionally, a penalty on the rudder deflection (δ_r) has been incorporated to prevent the optimization solver from converging on uncoordinated flight conditions. This uncoordinated tendency arises from the solver's increased degrees of freedom and must be restricted to ensure a physically natural trajectory and flight dynamics. Regarding the state variables, penalties were applied to ensure the smoothness of the equivalent aerodynamic and body Euler angles.

$$J_{\text{pen}}^{\text{FC}} = \beta_c \sum_{k=0}^{N-2} \|\xi_{c,k}^{\text{FC}}\|_{\mathbf{W}_c^{\text{FC}}}^2 + \beta_s \sum_{k=0}^{N-2} \|\xi_{s,k}^{\text{FC}}\|_{\mathbf{W}_s^{\text{FC}}}^2, \quad (6.17)$$

where the control-increment family targets the four FC actuators and the state family the Euler angles

tracked by the rigid-body attitude dynamics,

$$\boldsymbol{\xi}_{c,k}^{\text{FC}} = \begin{bmatrix} \Delta T_k & \Delta \delta_{e,k} & \Delta \delta_{a,k} & \Delta \delta_{r,k} \end{bmatrix}^{\text{T}},$$

$$\mathbf{W}_c^{\text{FC}} = \text{diag}(w_{\Delta T}, w_{\Delta \delta_e}, w_{\Delta \delta_a}, w_{\Delta \delta_r}),$$

$$\boldsymbol{\xi}_{s,k}^{\text{FC}} = \begin{bmatrix} \Delta \phi_k & \Delta \theta_k \end{bmatrix}^{\text{T}},$$

$$\mathbf{W}_s^{\text{FC}} = \text{diag}(w_{\Delta \phi}, w_{\Delta \theta}).$$

The tuning rationale of the weights $\{w(\cdot)\}$ are detailed in Annex ?? as well as the per-aircraft tuned values used in this work. With the base cost and its regularisation introduced in this compact form, the UF and FC objectives remain directly comparable across the minimum-time and minimum-fuel limits.

6.4 Validation Results

To evaluate the computational efficiency and functional capabilities of the developed software, verifying the initial optimization results against a suite of fundamental use cases is of paramount importance. This preliminary step ensures both the numerical robustness of the code and the accuracy of the trajectories generated by each framework. Accordingly, this section presents the numerical validation of the two discretized optimal-control formulations—the Uncoupled Formulation (UF) and the Full-Control Formulation (FC)—across the four reference scenarios established in Section 6.4.1.

6.4.1 Test Scenarios for Numerical Validation

The numerical implementation introduced in the previous section is tested using a structured set of scenarios defined in Table 6.6. This suite uses a 2×2 factorial design in empty airspace to isolate specific aircraft movements:

- **Lateral:** Straight flight versus a 180° heading reversal.
- **Longitudinal:** Level flight versus a 200 m climb between the entry and exit gates.

Aircraft MSN012, whose performance specifications and parameters are detailed in Annex ??, is used throughout this study. To isolate the longitudinal and lateral dynamics and their intrinsic coupling, environmental and mission-level constraints—including the digital elevation model (DEM), no-fly zones, geofencing, wind, and intermediate waypoints—were deactivated. This ensures that the validation focuses strictly on the underlying physics of the trajectory rather than map-based feasibility layers.

6.4.2 Scenario Definitions

The validation suite consists of four distinct flight profiles designed to stress-test the solver’s ability to handle vertical transitions, lateral maneuvers, and combined 3D trajectory optimization. These benchmarks provide the basis for the performance metrics analyzed in Chapter 6. Each scenario is evaluated under both Uncoupled formulation (UF) and Full Control formulation (FC) across the optimization spectrum, ranging from minimum-time ($\alpha = 0$) to minimum-fuel ($\alpha = 1$) objectives.

Table 6.6: Geometric and environmental configurations for validation benchmarks, specifying mission layout, vertical displacement, and trajectory complexity.

Label	Mission Geometry	Δ Alt. [m]	Dist. [km]	Wind	Primary Objective
SC-00	Straight-line, level flight	0	3.35	None	Baseline horizontal transit
SC-01	Straight-line, steady climb	+200	3.36	None	Vertical energy efficiency
SC-02	180° horizontal turn	0	4.09	None	Lateral maneuverability
SC-03	Combined climb-and-turn	+200	4.10	None	Full 3D trajectory coupling

6.4.3 Validation Metrics

To ensure the integrity of the numerical results, a converged solution is subjected to an *external* validation process. A trajectory is deemed valid only if the following metrics remain within their prescribed admissible ranges:

- **Maximum Normalized Defect:** The infinity norm of the state residuals, $\|\Delta\|_\infty \leq 10^{-4}$, is computed *a posteriori* using the converged decision vector. This ensures the collocation scheme is consistent with the underlying vehicle dynamics and that the discretization density is sufficient to suppress integration errors.
- **Constraint Feasibility Margins:** Every inequality constraint is mapped using the negative convention, requiring the condition $h_j(\mathbf{z}) \leq 0$ for compliance. For constraints expressed as dimensionless ratios—such as the aerodynamic stall limit ($1 - V/V_{\text{stall}} \leq 0$) or the power availability threshold—the constraint residual $h_j(\mathbf{z})$ directly quantifies the proximity of the trajectory to the edge of the flight envelope. A non-positive residual is strictly mandatory across all discrete nodes for the optimized flight path to be considered physically realizable.
- **Cross-Formulation Consistency:** This metric evaluates the deviation between the Unconstrained Flight (UF) and Full Control (FC) trajectories. The constrained solution must remain within an expected spatial tolerance of the baseline, proportional to the reference length and the number of nodes. Consistency across these formulations validates that the optimizer has reached a global or high-quality local optimum rather than an algorithmic artifact.
- **Minimum-Time versus Minimum-Fuel Performance:** A quantitative comparison across different optimization criteria, highlighting the fundamental variations in aircraft performance when prioritizing competing operational metrics (transit speed versus fuel conservation).
- **Influence of Regularization Penalties:** An analysis of how smoothing penalties regularize the NLP objective function. This comparison, which is expanded in Appendix ??, demonstrates the distinct numerical effects of applying regularization terms to either the control inputs or the state variables to suppress unphysical chattering.

These five key metrics—dynamic consistency (defects), constraint feasibility (margins), objective parameterization (time versus fuel), regularization effects (penalties), and formulation sensitivity—provide a comprehensive framework for the numerical validation presented in the following results.

6.4.4 Trajectory Visualisation

Both NLPs are solved across the four reference scenarios under both optimisation criteria, using the regularised production settings ($\beta_c = \beta_s = 1$). Figure 6.1 illustrates the planimetric ground tracks for all scenarios, overlaying the four computed solutions: UF (minimum-time), UF (minimum-fuel),

FC (minimum-time), and FC (minimum-fuel). The corresponding altitude profiles, which highlight the strategic vertical divergence between the minimum-time and minimum-fuel objectives for each formulation, are presented in Figures 6.2 and 6.3. Furthermore, Figures 6.8 through 6.11 directly cross-examine the altitude profiles of the UF and FC models scenario by scenario. Collectively, this visual suite serves to validate the numerical results following the qualitative-then-quantitative protocol advocated for direct-collocation solutions in [34], clearly delineating the performance trade-offs governed by the chosen cost functions alongside the inherent kinematic discrepancies between the point-mass and rigid-body formulations.

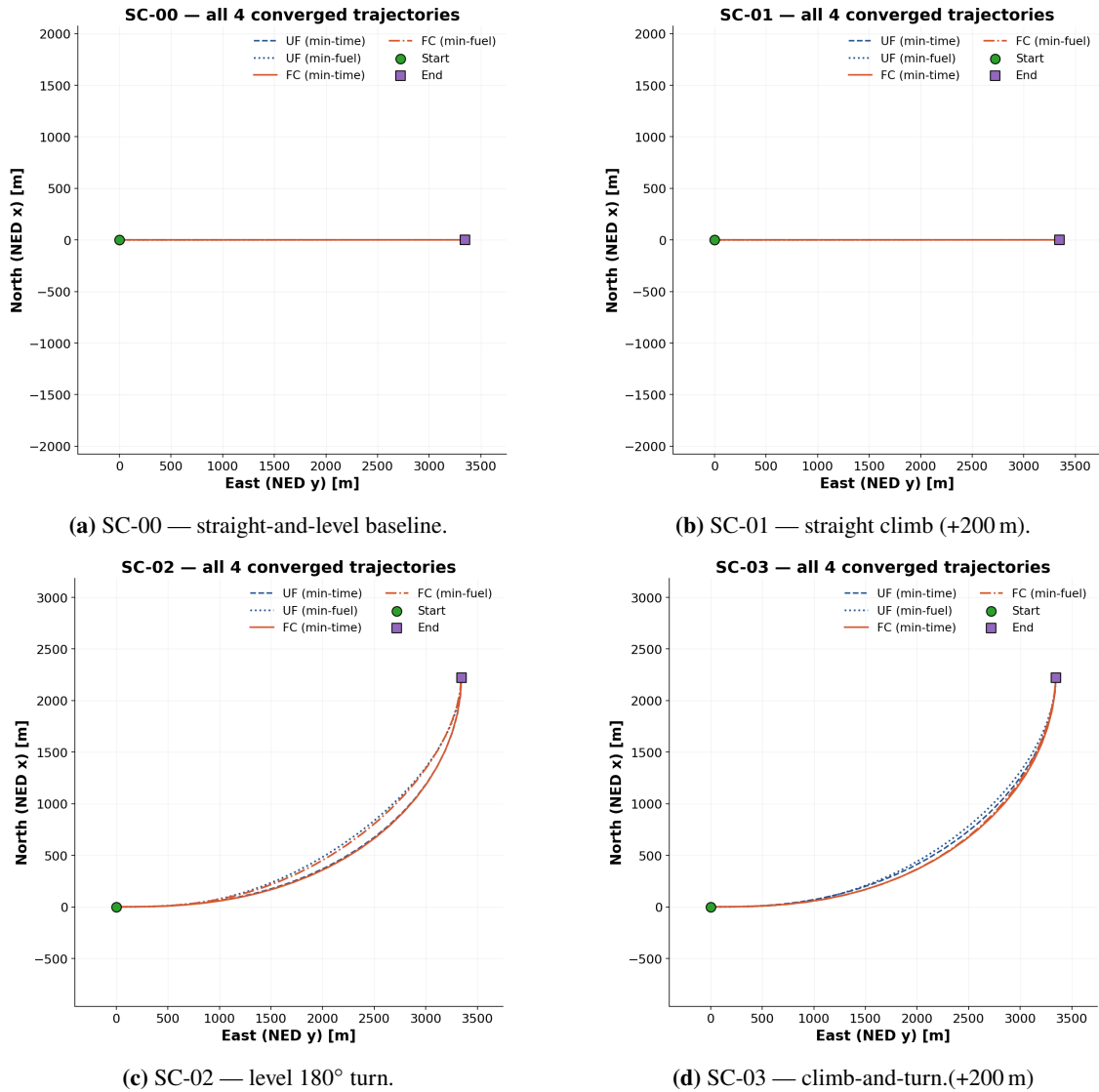


Figure 6.1: Top-view overlay of all four converged trajectories per scenario (UF and FC, min-time and min-fuel). All thumbnails are rendered at the same physical height so that the relative scale of each scenario's operational corridor is preserved across the grid. The geometric similarity across criteria confirms that the optimisation criterion modifies the flight profile, not the ground track.

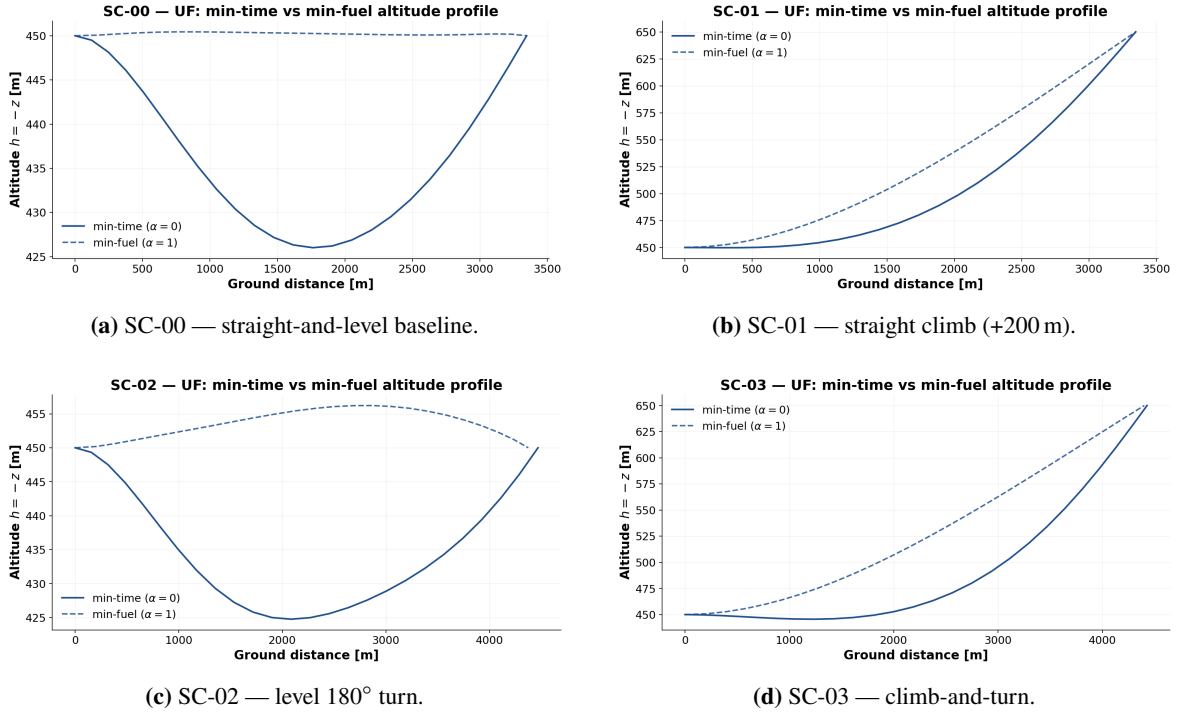


Figure 6.2: Altitude profiles of the converged UF solution on every reference scenario, overlaying the min-time ($\alpha = 0$) and min-fuel ($\alpha = 1$) criteria. SC-00 and SC-02 are level missions; SC-01 and SC-03 add the suite's +200 m terminal climb. SC-02 and SC-03 impose the 180° heading reversal.

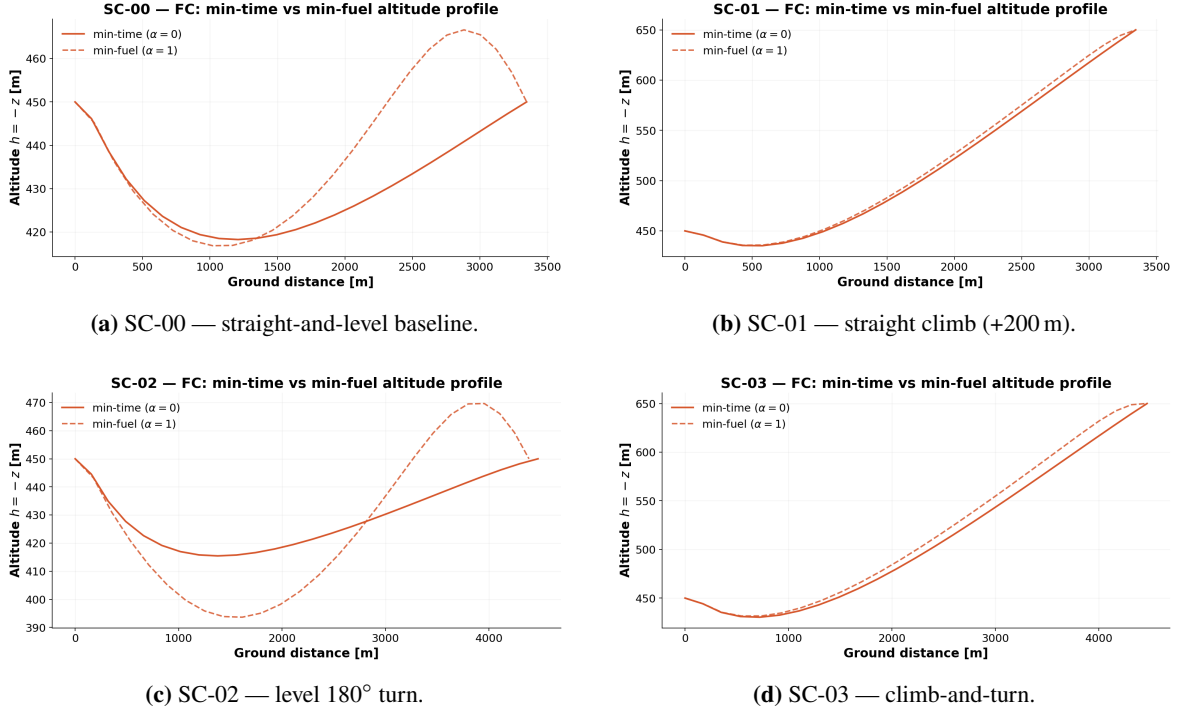


Figure 6.3: Altitude profiles of the converged FC solution on every reference scenario, overlaying the min-time ($\alpha = 0$) and min-fuel ($\alpha = 1$) criteria.

Under the minimum-time criterion, the four ground tracks are nearly coincident within each reference scenario. This spatial agreement confirms that the optimization criterion primarily dictates the velocity scheduling and thrust allocation profiles rather than altering the horizontal path geometry. While the straight baselines (SC-00, SC-01) maintain an almost invariant ground track, the heading-reversal maneuvers (SC-02, SC-03) exhibit modest path-length variations (under 2.5% across the entire suite) as the optimizer trades spatial extension against transient kinetic energy.

In contrast, the altitude and airspeed profiles reveal that the choice of objective function fundamentally reshapes the vertical flight envelope. This minimum-time versus minimum-fuel divergence is most pronounced during the level missions (SC-00 and SC-02), where the optimizer has greater vertical freedom to select an altitude profile that strictly minimizes the cost function. Conversely, the climbing scenarios (SC-01 and SC-03) provide less room for vertical optimization due to the spatial geometry of the problem; the aircraft is forced to satisfy strict altitude boundary constraints at the final node, which restricts the solver's ability to purely prioritize thrust economy or speed.

Furthermore, the altitude overlays between the Uncoupled Formulation (UF) and Full-Control Formulation (FC) demonstrate that resolving rigid-body dynamics refines the vertical and velocity schedules without significantly shifting the horizontal corridor dictated by the spatial boundary conditions. As detailed in Section 6.4, the discrepancies between the two formulations are non significant (although

not identical) on constrained climbing paths. However, these differences expand significantly when the spatial geometry affords the IPOPT solver greater freedom to modulate the altitude profile to minimize the objective cost. For both criteria, the most efficient strategy to optimize the cost function is to intricately couple the velocity and altitude profiles. By intelligently leveraging gravitational potential energy to smoothly accumulate or shed airspeed, the solver regulates kinetic energy organically, thereby avoiding abrupt, highly penalized transients in engine thrust.

Crucially, it is a mathematical misconception to assume that the UF and FC models should converge to identical trajectories. The UF model utilizes a simplified, three-degrees-of-freedom (3-DOF) point-mass approximation restricted entirely to the aerodynamic wind frame. Because it abstracts away the aircraft's physical orientation, it inherently omits the kinematic coupling and inertial transport terms required to rotate the vehicle body. Conversely, the FC model operates within an expanded state-space that explicitly tracks rigid-body attitude dynamics. This added fidelity equips the FC optimizer with additional degrees of freedom, allowing it to coordinate transient attitude adjustments that exploit subtle aerodynamic and propulsive couplings hidden from the simplified UF equations. Consequently, the FC solver navigates a fundamentally different, more expressive solution space, naturally leading to distinct altitude and velocity profiles during highly coupled maneuvers.

This underlying structural divergence complicates the calibration of the regularization framework. Although the smoothing penalties described in Section 6.3.2 were heuristically tuned to promote similar qualitative trajectories, achieving exact numerical equivalence between the formulations is practically impossible. Because the regularization terms penalize fundamentally distinct physical variables across the two formulations—such as wind-frame steering rates in the UF versus body-axis angular rates and actuator deflections in the FC—there is no direct mathematical mapping between their respective penalty weights. The resulting differences in the converged optima (as seen, for example, in the velocity-altitude trade-offs in Table ??) are therefore a logical consequence of both formulation expressiveness and the asymmetric impact of numerical regularization [47].

6.4.5 Constraint Satisfaction Verification

To ensure the physical and mathematical validity of the optimized trajectories, every converged solution is subjected to a three-tier verification pipeline. While the comprehensive data for every run is archived in Annex ??, this section provides a condensed summary based on the maximum residuals within each category:

- **Tier A: Decision-Variable Box Bounds** — search-space limits imposed on the individual components of the state and control vectors. The specific non-dimensional bounds are listed in Tables ?? and ??.
- **Tier B: Hard Path Constraints** — physical and operational envelopes (e.g., structural limits

or safety margins) enforced during the optimisation. The limits for UF and FC are detailed in Tables ?? and ??.

- **Tier C: Collocation Defect Residuals** — dynamic-fidelity residuals quantifying how exactly the trapezoidal collocation scheme satisfies the underlying differential equations. By construction these equality residuals are satisfied to within the solver tolerance and are reported as absolute magnitudes rather than margins.

For Tiers A and B, the pipeline reports a *worst-case margin*. To provide a uniform metric for both upper and lower limits, the margin is computed as

$$M = \begin{cases} \text{value} - \bar{x} & (\text{Upper bounds}) \\ \underline{x} - \text{value} & (\text{Lower bounds}) \end{cases} \quad (6.18)$$

Under this convention, a **negative margin (OK)** indicates that the constraint was satisfied with a remaining slack equal to $|M|$, while a **positive margin (ACTIVE)** indicates that the constraint was either exactly met or slightly violated within the allowed numerical tolerance. Tier C residuals are always reported as OK due to their nature as satisfied equality constraints (interior-point solvers, whenever they converge, satisfy equality constraints) and are detailed in the per-run breakdowns of Annex ??.

Furthermore, it is important to note that in Figures 6.4 and 6.5 the boundaries have been renormalised with respect to the margin distribution itself, providing a common criterion of comparison and easing the visual assessment of the constraint-satisfaction validation. Although Section 6.1.3 already establishes that the constraints are scaled internally according to the criteria listed in Table 6.2, this additional output-level renormalisation has been considered useful purely as a post-processing step for data visualisation.

Under this max–min renormalisation the bars are confined to $[-1, 0]$ and should be read against three reference points: -1 denotes a wide absolute margin (constraint far from binding, no concern), 0 denotes a near-violation (solver operating at the boundary), and an intermediate value of ≈ -0.5 is *not* a tight margin but the signature of a set of variables whose absolute margins are uniformly small (close to zero in most cases), which the renormalisation simply collapses around the midpoint of the dynamic range.

Uncoupled Formulation (UF)

Figure 6.4 reports the cross-scenario UF verdicts for the four reference cases under both criteria; the corresponding numerical worst-case margins are gathered in Annex ??.

Every bar in Figure 6.4 is strictly non-positive, confirming that the solver successfully satisfies all

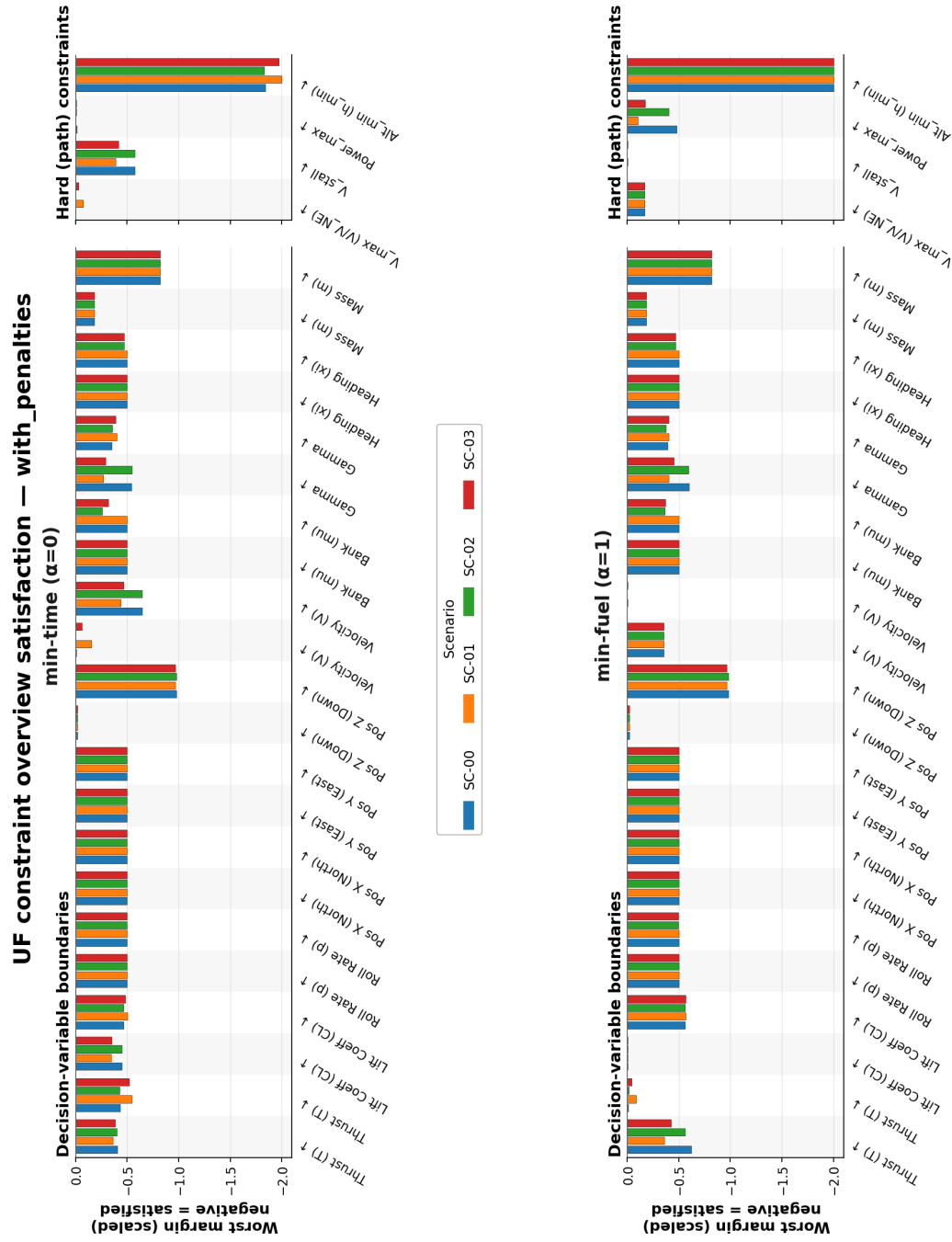


Figure 6.4: Normalised worst-case constraint margins for the Uncoupled Formulation (UF) across the four reference scenarios under the minimum-time and minimum-fuel objectives. Bars are renormalised to $[-1, 0]$ for visualisation purposes.

decision-variable bounds and physical path constraints across all four scenarios. Rather than examining the entire constraint matrix, the distinct operational strategies of the two cost functions can be understood by observing how specific state and control variables—namely velocity (V), lift coefficient (C_L), flight path angle (γ), and bank angle (μ)—react to the different trajectory geometries: straight level flight (SC-00), straight climbs (SC-01), turning level flight (SC-02), and climbing turns (SC-03).

- **Minimum-Time (The High-Speed Boundary):** Optimizing for speed naturally drives the aircraft to the upper limits of its kinetic envelope. The solver pushes the airspeed (V) directly against its upper bounds, especially during level missions (narrowing the margin to -0.0088 on SC-00 and -0.0042 on SC-02). Because the aircraft maintains such high dynamic pressure, it requires very little lift to stay aloft, keeping the C_L margins wide (≈ -0.45).

The geometric demands of the scenarios heavily influence the angular states. In the climbing scenarios (SC-01 and SC-03), the flight path angle (γ) pushes much closer to its upper boundary (tightening to -0.2705 on SC-01) compared to the level scenarios. Similarly, the turning scenarios (SC-02 and SC-03) require aggressive maneuvering to minimize time, forcing the bank angle (μ) tightly against its bounds (reaching -0.2567 on SC-02), whereas μ remains completely inactive on the straight paths.

- **Minimum-Fuel (The High-Efficiency Boundary):** Shifting to a fuel-conservation objective forces the active constraints into the low-speed, high-lift corner of the envelope. To maximize aerodynamic efficiency, the optimizer drastically reduces V , collapsing the velocity lower bound and V_{stall} margins to the order of $O(10^{-4})$ across all scenarios (e.g., dropping to -5.59×10^{-4} on SC-00). To sustain flight at these minimal speeds, the lift coefficient (C_L) margin shrinks tightly against its upper bound (reaching -0.0024 on the SC-02 turn).

This low-speed strategy fundamentally alters the spatial variables. During the climbs (SC-01 and SC-03), the γ margins remain noticeably wider (-0.4016 on SC-01) than in the minimum-time formulation, reflecting a shallower, more economically optimal ascent. Furthermore, executing the turns (SC-02 and SC-03) at low speeds requires less aggressive roll rates to achieve the necessary heading change, resulting in gentler bank angles (μ) with wider margins (-0.3639 on SC-02) than their minimum-time counterparts.

Finally, the numerical robustness of the discretisation is excellent in both cases. Discretisation residuals (detailed in Annex ??) remain completely negligible, never exceeding 1.66×10^{-5} . However, defect norms shrink even further (to $\leq 2.2 \times 10^{-6}$) under the minimum-fuel criterion, which is consistent with the fuel objective producing naturally smoother control histories than the aggressive bang-bang tendencies of a minimum-time formulation.

Full-Control Formulation (FC)

The same three-tier verification, applied to the FC NLP, is reported in Figure 6.5. Each bar gives the worst-case margin of a single (variable, side) – including the FC-only attitudes, body-axis velocities, control surfaces and body rates – as well as the rigid-body path constraints ($V_{\max}$, V_{stall} , $C_{L,\max}$, $P_{\max}$, $h_{\min}$); the exact numerical values are gathered in Annex ??.

As with the point-mass model, every bar in Figure 6.5 is strictly non-positive, verifying that the 6-DOF rigid-body solver strictly respects all state boundaries, actuator limits, and physical path constraints. Crucially, the FC results exhibit the exact same macroscopic optimization trends as the Uncoupled Formulation (UF) analyzed earlier, adapting to the chosen objective by pushing against specific edges of the flight envelope:

- **Minimum-Time (The High-Speed Boundary):** Mirroring the UF strategy, the FC solver minimizes transit time by driving the aircraft to the upper limits of its kinetic envelope. The forward body velocity (U) and structural airspeed (V/V_{NE}) margins shrink to a mere -0.0026 on the planar turn (SC-02). To sustain this high-kinetic state, the engine operates near maximum continuous power, coming within -0.0015 of saturation during the climbing turn (SC-03). Consequently, the high dynamic pressure keeps the aircraft safely distant from aerodynamic stall boundaries (maintaining a wide stall margin of -0.5765 on level flights). Furthermore, the geometric demands of minimum-time turns force the roll angle (ϕ) tightly against its bounds (-0.2661 on SC-02).
- **Minimum-Fuel (The High-Efficiency Boundary):** Consistent with the UF findings, optimizing for fuel economy shifts the strategy toward maximum aerodynamic efficiency. The solver drastically reduces the commanded airspeed, which correspondingly plummets the required propulsive effort. During the level SC-00 transit, the engine practically idles, with the thrust lower bound margin collapsing to $O(10^{-4})$ (specifically -8.65×10^{-4}). As observed in the point-mass model, executing maneuvers at these lower speeds requires less aggressive spatial reorientations, resulting in gentler bank angles during turns (the ϕ margin widens to -0.3423 on SC-02 compared to its minimum-time counterpart).

Beyond confirming that the macroscopic trajectory strategies match the UF, the expanded state-space of the FC formulation proves to be dynamically well-behaved. The angular rate margins (roll p , pitch q , yaw r) and the aerodynamic control surface deflections (elevator δ_e , aileron δ_a , rudder δ_r) remain comfortably far from their saturation limits across all four scenarios. This confirms that the rigid-body optimizer successfully achieves the global objectives without resorting to aggressive, physically unrealizable actuator commands. This is due, in part, to the penalties applied. Figures 6.15 and 6.16 illustrate the transition to heavy penalization of the control variables.

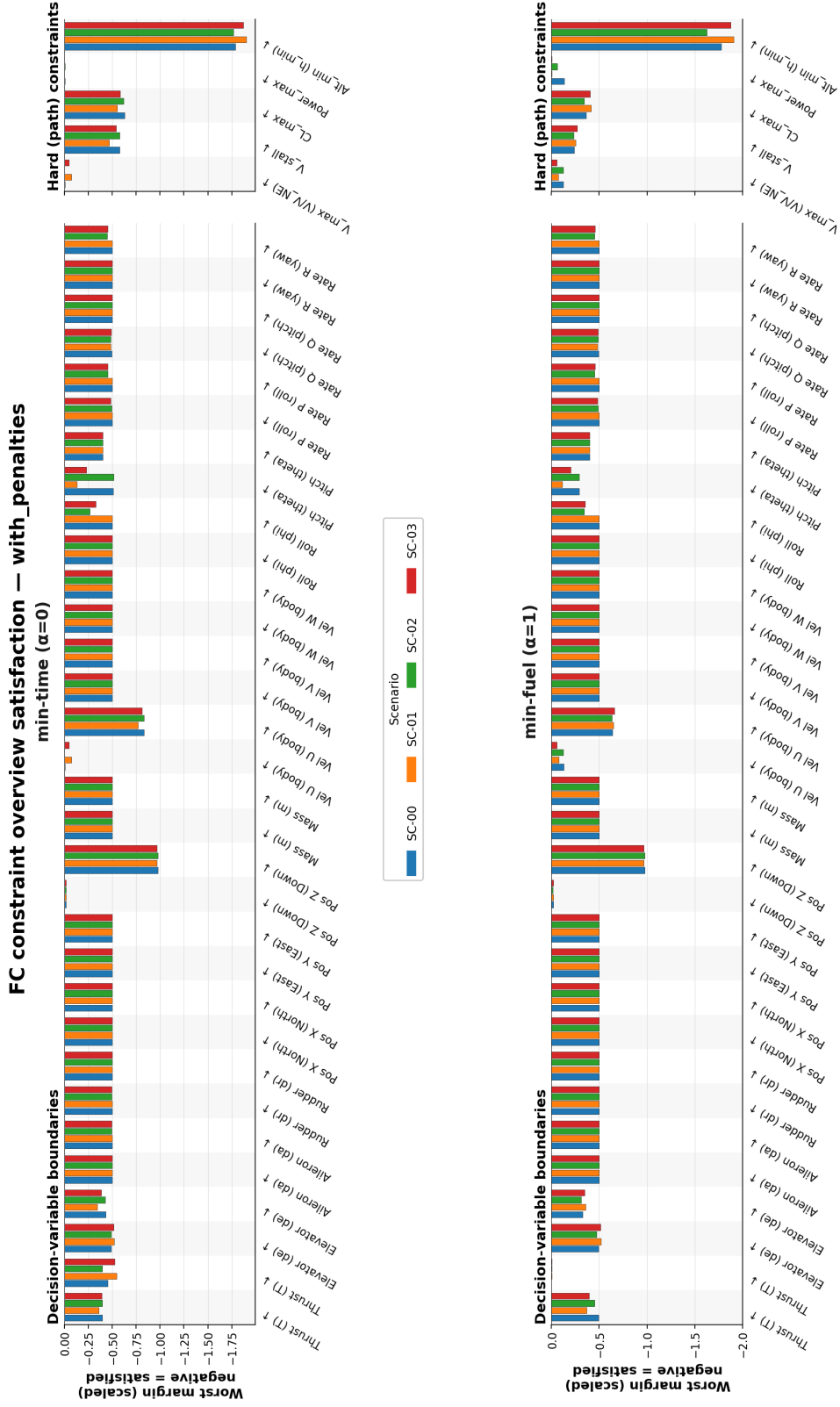


Figure 6.5: Normalised worst-case constraint margins for the Full-Control Formulation (FC) across the four reference scenarios under the minimum-time and minimum-fuel objectives. Bars are renormalised to $[-1, 0]$ for visualisation purposes.

6.4.6 Min-Time vs Min-Fuel Trade-off

The cost functional (6.14) interpolates between a pure min-time objective ($\alpha = 0$) and a pure min-fuel objective ($\alpha = 1$) through the scalar weighting parameter $\alpha = w_{\text{fuel,time}}$. Figures 6.6 and 6.7 condense the per-formulation trade between the two extremes into a single bar-chart panel per formulation, with the four reference scenarios on the horizontal axis of every metric. The corresponding per-scenario numerical tables are gathered in Annex ?? (Sections ??–??).

Uncoupled Formulation (UF)

Min-time vs Min-fuel across validation scenarios — UF

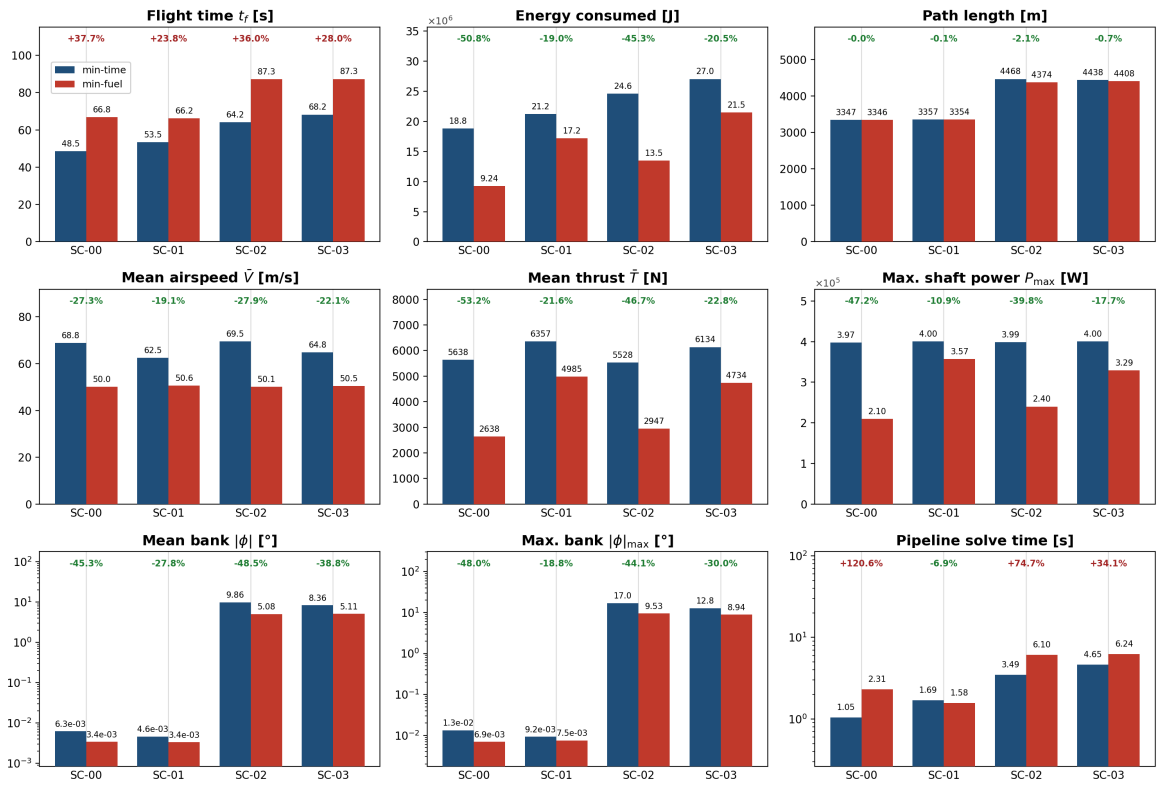


Figure 6.6: Global comparison of the minimum-time and minimum-fuel solutions across the four reference scenarios for the Uncoupled Formulation (UF). Each panel reports a key performance, kinematic, propulsive or computational metric; bars in blue and red correspond to $\alpha = 0$ and $\alpha = 1$ respectively.

In general, shifting from a minimum-time to a minimum-fuel strategy yields substantial energy savings at the direct cost of an increased flight time, as evidenced in Figure 6.6. However, the magnitude of this trade-off depends heavily on the specific manoeuvre. In purely level flight—whether straight or turning—the energy savings are most pronounced because the aircraft can significantly reduce its airspeed and adopt gentler bank angles (Tables ?? and ?? in Annex ??). Conversely, when a manoeuvre requires an increase in altitude (Tables ?? and ??), the inherent energy cost of climbing restricts the

amount of fuel that can be saved simply by slowing down. Across all scenarios, fuel-minimising profiles consistently favour slower mean airspeeds and less aggressive manoeuvring, such as shallower bank angles during turns. Furthermore, the results dispel the intuitive assumption that a minimum-time trajectory naturally equates to the shortest physical path (minimum-distance): time-minimising paths frequently cover longer distances, since the optimiser prioritises altitude profiles that facilitate greater acceleration and higher sustained speeds (as also visible in Figures 6.8 and 6.10), ultimately minimising total flight time despite the longer route.

Full-Control Formulation (FC)

Min-time vs Min-fuel across validation scenarios — FC

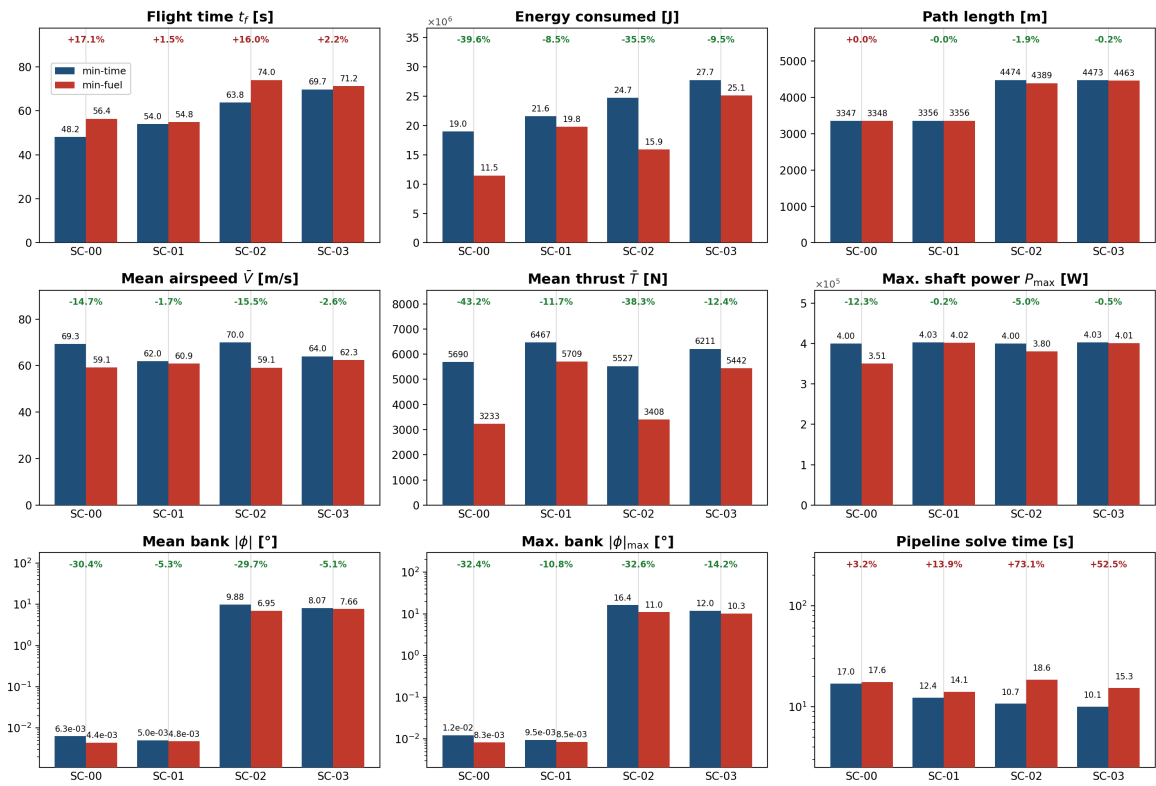


Figure 6.7: Global comparison of the minimum-time and minimum-fuel solutions across the four reference scenarios for the Full-Control Formulation (FC). Panel layout and colour convention are the same as in Figure 6.6.

The qualitative tendencies highlighted for the UF formulation —fuel-minimising profiles favouring longer flight times and shallower bank angles— carry over to the Full-Control formulation in Figure 6.7: the same blue-to-red shift is observed across all four scenarios in the flight-time, energy and peak-thrust panels. The detailed per-scenario figures are reported in Tables ??–?? in Annex ??.

A direct comparison of the two formulations, however, exposes a significant quantitative divergence.

Under the minimum-time criterion, the UF and FC trajectories remain tightly clustered, showing exceptional agreement across all scenarios with total flight times t_f deviating by less than 2.5 %. In this high-power regime, the aircraft operates near its performance limits, meaning the optimal trajectory is strictly constrained by boundary conditions and maximum available thrust, leaving little room for structural variations between the models (as observed in the tightly overlaid profiles of Figures 6.8, 6.9, 6.10 and 6.11). In stark contrast, the minimum-fuel objective exposes a massive divergence between the two formulations. This mismatch is clearly visible not only in the integrated metrics of Figures 6.6 and 6.7 but also in the underlying altitude and velocity profiles. Under the fuel-optimisation branch, the UF formulation yields an overly optimistic Pareto front—for instance, buying a 20.5 % energy reduction on SC-03 at the cost of a 28.0 % time extension. The FC model, however, produces a significantly compressed trade-off space, yielding only a 9.5 % energy saving for a modest 2.2 % increase in time, an effect strongly illustrated by the major vertical path splits between the formulations in the min-fuel cases (Figures 6.8–6.11).

The root cause of this optimization divergence lies in how aerodynamic trim is achieved. The simplified UF framework assumes the aircraft can instantly balance its flight path solely through a reduction in aerodynamic speed $\bar{V}$. The FC formulation, however, respects full rigid-body dynamic restrictions. To maintain trim, the optimizer must explicitly account for elevator deflection δ_e , pitch attitude, and angle of attack. Crucially, the FC formulation penalizes all control states and actuator rates to ensure physical viability. Because generating lift at lower airspeeds requires larger angles of attack and specific elevator trim configurations—which in turn alter the pitch-moment balance and drag profile—the FC optimizer cannot down-scale its airspeed profile as aggressively as the UF model without incurring severe penalty costs elsewhere. Consequently, the multi-variable trim requirements bound the FC trajectory to a much tighter, more realistic envelope, leading to the noticeably more constrained vertical profiles shown across all scenarios (Figs. 6.8, 6.9, 6.10, and 6.11).

When evaluating lateral-directional performance as seen in turning scenarios, care must be taken when comparing the bank angles directly because the two variables are fundamentally distinct. The UF formulation evaluates an aerodynamic bank angle abstraction, typically the velocity-frame roll angle μ or wind-relative bank, whereas the FC formulation evaluates the true rigid-body Euler roll angle ϕ . Because the Euler roll angle explicitly captures the kinematic coupling of the aircraft’s physical orientations during a turn, it cannot be mapped one-to-one to the aerodynamic flight-path bank. The data reflects this structural difference; the rigid-body trim restrictions prevent the FC bank angle from shrinking as aggressively as the UF abstraction under the fuel objective, leading to higher mean and maximum bank angles for the FC formulation, which directly correlates to the distinct vertical lifting strategies seen in the turn-heavy altitude profiles (Figs. 6.10 and 6.11).

The high physical fidelity and multi-variable coupling of the FC formulation come at a steep computational premium. As indicated by the pipeline solve times across all test scenarios, the FC

model requires significantly longer to converge than the UF counterpart. This massive increase in execution time—peaking at a +919.6% overhead during the SC-01 climb—highlights the mathematical difficulty of solving the highly non-linear Differential-Algebraic Equations (DAEs) associated with rigid-body trim, validating the sequential UF $\rightarrow$ FC implementation pipeline as a necessary practical compromise for trajectory generation.

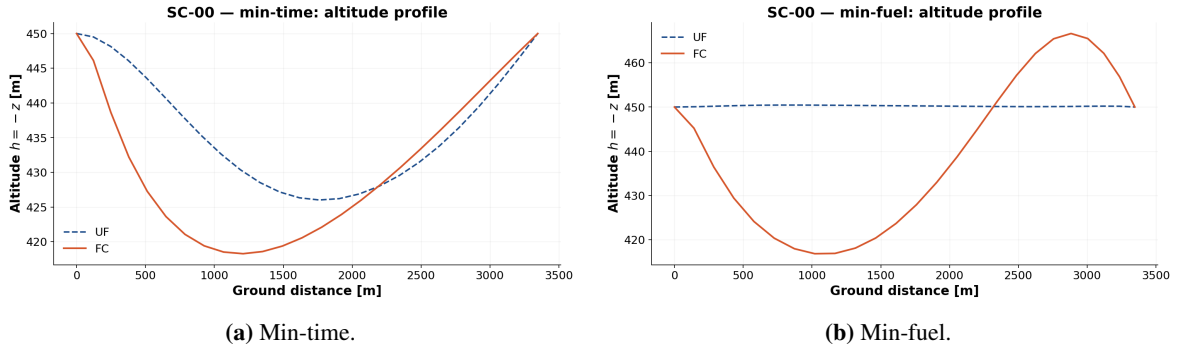


Figure 6.8: UF and FC altitude traces overlaid on SC-00 (straight-and-level baseline) under both criteria.

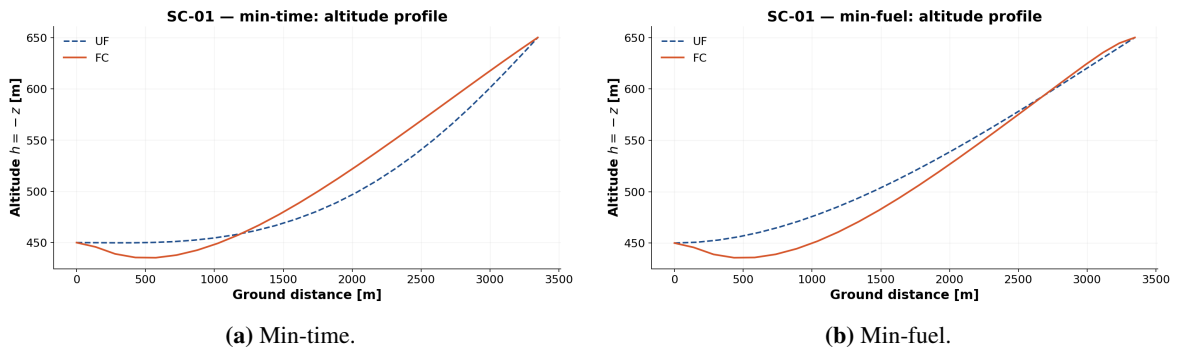


Figure 6.9: UF and FC altitude traces overlaid on SC-01 (straight climb) under both criteria.

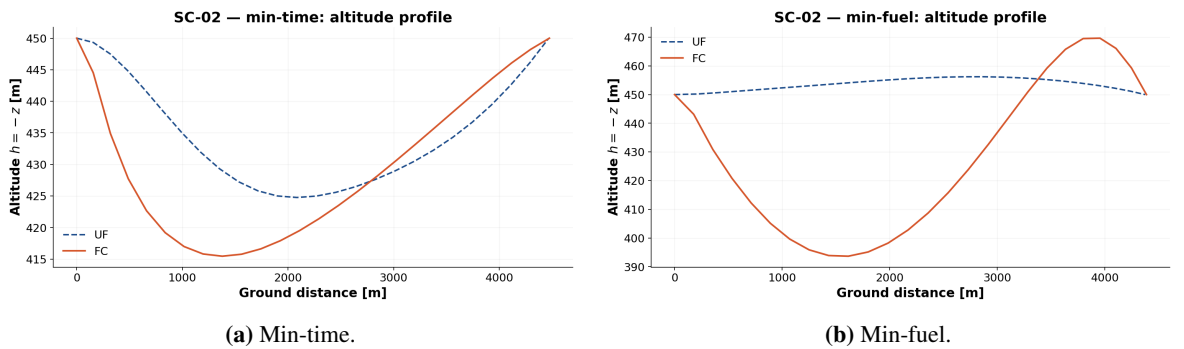


Figure 6.10: UF and FC altitude traces overlaid on SC-02 (level 180° turn) under both criteria.

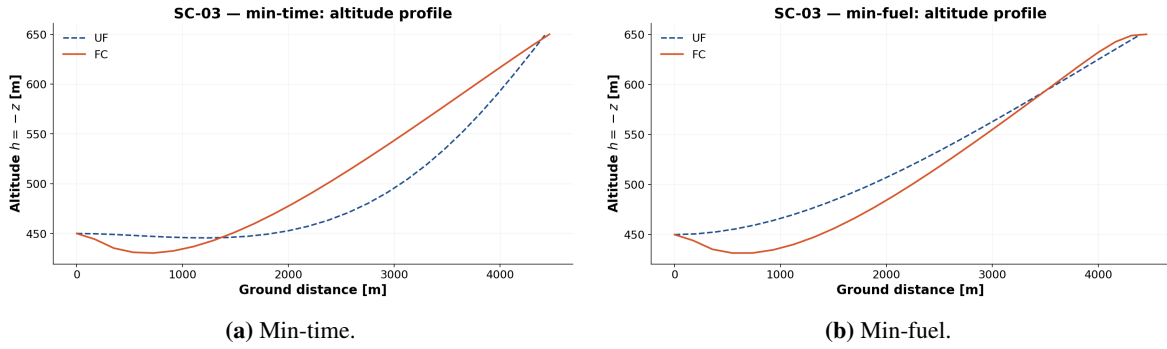


Figure 6.11: UF and FC altitude traces overlaid on SC-03 (climb-and-turn) under both criteria.

6.5 Impact of cost regularisation on the optimum

To verify that the regularisation introduced in Section 6.3.2 acts mainly on smoothness rather than on the location of the constrained optimum, every benchmark scenario was solved twice: once with the nominal gates $\beta_c = \beta_s = 1$ and once with the ablation $\beta_c = \beta_s = 0$. All other numerical settings, including the warm-start strategy and the interior-point tolerances, were held fixed. The discussion below uses scenario SC-01 under the minimum-fuel criterion as a highly explicit representative case; the dashboards, tables and per-state defect breakdowns for the remaining three scenarios are reported scenario by scenario in Annex ??, with the SC-01 entries collected in Section ??.

6.5.1 Headline Metrics

Table 6.7: SC-01 (straight climb), min-fuel ($\alpha = 1$): comparison of global quantities between the regularised production run ($\beta_c = \beta_s = 1$) and the unregularised ablation ($\beta_c = \beta_s = 0$). The entries reproduce the minimum-fuel ($\alpha = 1$) columns of the uncoupled and full-control Pareto summaries for this scenario.

Form.	Metric	With reg.	No reg.	Δ [%]
UF	Flight time t_f [s]	66.15	66.66	+0.77
UF	Energy [J]	1.72×10^7	1.70×10^7	-1.16
UF	Path length [m]	3353.6	3354.4	+0.024
UF	Mean thrust [N]	4964.4	4843.9	-2.43
FC	Flight time t_f [s]	54.81	66.50	+21.3
FC	Energy [J]	1.98×10^7	1.71×10^7	-13.6
FC	Path length [m]	3355.8	3355.5	-0.009
FC	Mean thrust [N]	5711.5	4818.5	-15.6

The regularisation and control smoothing, however, come at the expense of solution optimality. Specifically, achieving a smooth optimal solution proves significantly more challenging with the FC

formulation than with the UF formulation, often resulting in a higher degree of suboptimality. This behaviour is reflected in the key metrics of Table 6.7:

- **Global Cost.** For the SC-01 minimum-fuel scenario, the UF mission metrics shift by at most a few percent, in line with the expectation that regularisation primarily smooths the control profiles. Conversely, the FC formulation exhibits a larger variation in flight time; while both configurations remain feasible, the unregularised formulation allows the NLP solver to converge to a different time horizon while maintaining a similar path length.
- **Control Total Variation.** As recorded in the SC-01 diagnostic of Annex ?? (and gathered cross-scenario in Section ??), the total variation $TV(u)$ drops sharply across all axes once the increment penalties are activated, successfully suppressing high-frequency bang-bang chatter.
- **Collocation Defect.** The per-state defect norms reported in the same annex section remain below the prescribed external tolerance for both formulations. The unregularised runs simply exhibit more aggressive profile oscillations without violating the defect bounds.

The comprehensive, per-scenario data for all four benchmarks is reported in Annex ??.

6.5.2 Qualitative impact on the trajectory

Figures 6.12 and 6.13 illustrate, for scenario SC-01 under the minimum-fuel criterion, the trajectory dashboards obtained with the Uncoupled and Full-Control formulations, respectively. The corresponding control-surface deflection histories for the rigid-body model are displayed in Figure 6.14, and the aileron, rudder and elevator pair across the regularised and unregularised runs is isolated in Figures 6.15 and 6.16.

The plots highlight the close agreement between both formulations when regularisation is active, as well as the successful suppression of high-frequency oscillations. The two models track the same mission geometry while differing in the rigidity of the attitude dynamics, as discussed in Section 6.4. The quantitative comparison of regularised and unregularised runs (cost summary, total variation, defect breakdown) is given in the SC-01 block of Annex ??.

UF penalty impact dashboard — SC-01 · min-fuel

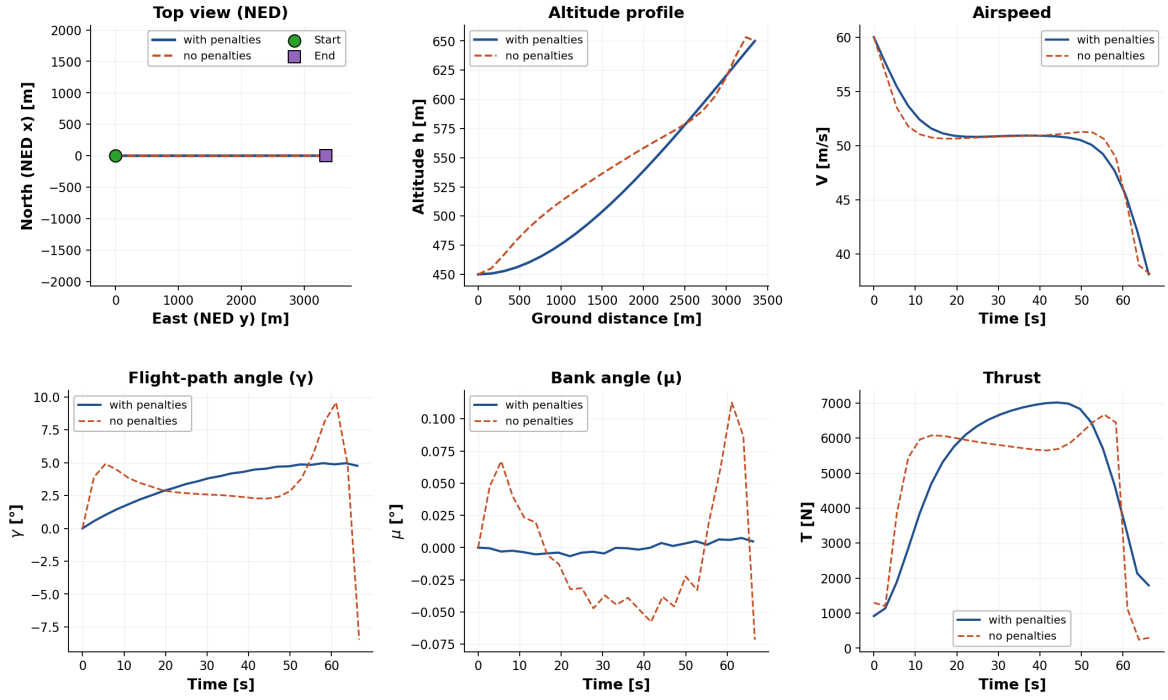


Figure 6.12: Minimum-fuel trajectory for Scenario SC-01 (straight climb) using the Uncoupled Formulation (UF): comparison of the solution with active penalty regularisation ($\beta_c = \beta_s = 1$) versus the unregularised ablation ($\beta_c = \beta_s = 0$). The panels display the altitude profile, the bank angle, the flight-path angle and the airspeed.

FC penalty impact dashboard — SC-01 · min-fuel

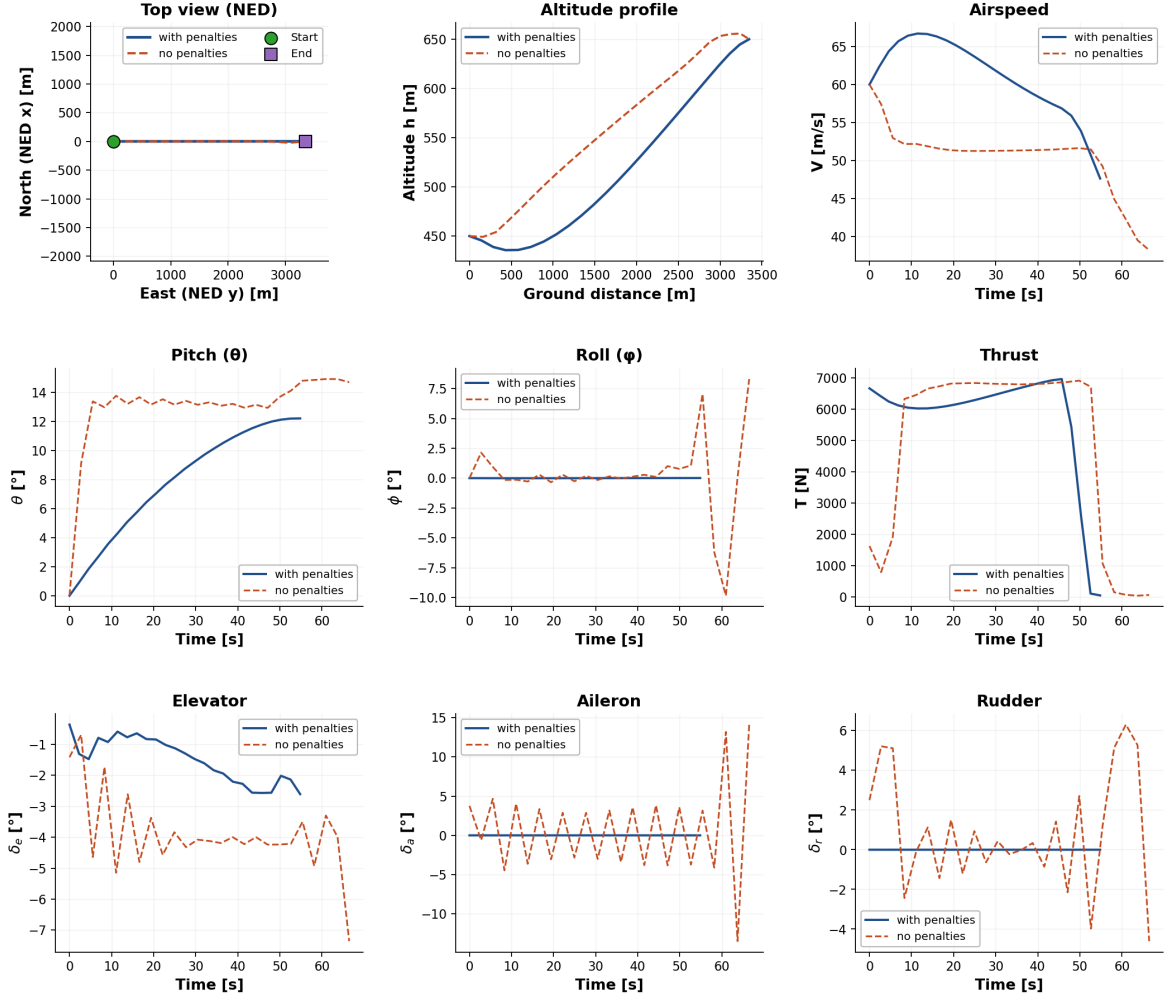


Figure 6.13: Minimum-fuel trajectory for Scenario SC-01 (straight climb) using the Full-Control Formulation (FC), under the same conventions as Figure 6.12. The rigid-body model tracks the same global trajectory; the discriminator is the bank-angle panel, where the unregularised run exhibits the chatter signature that the regularisation suppresses.

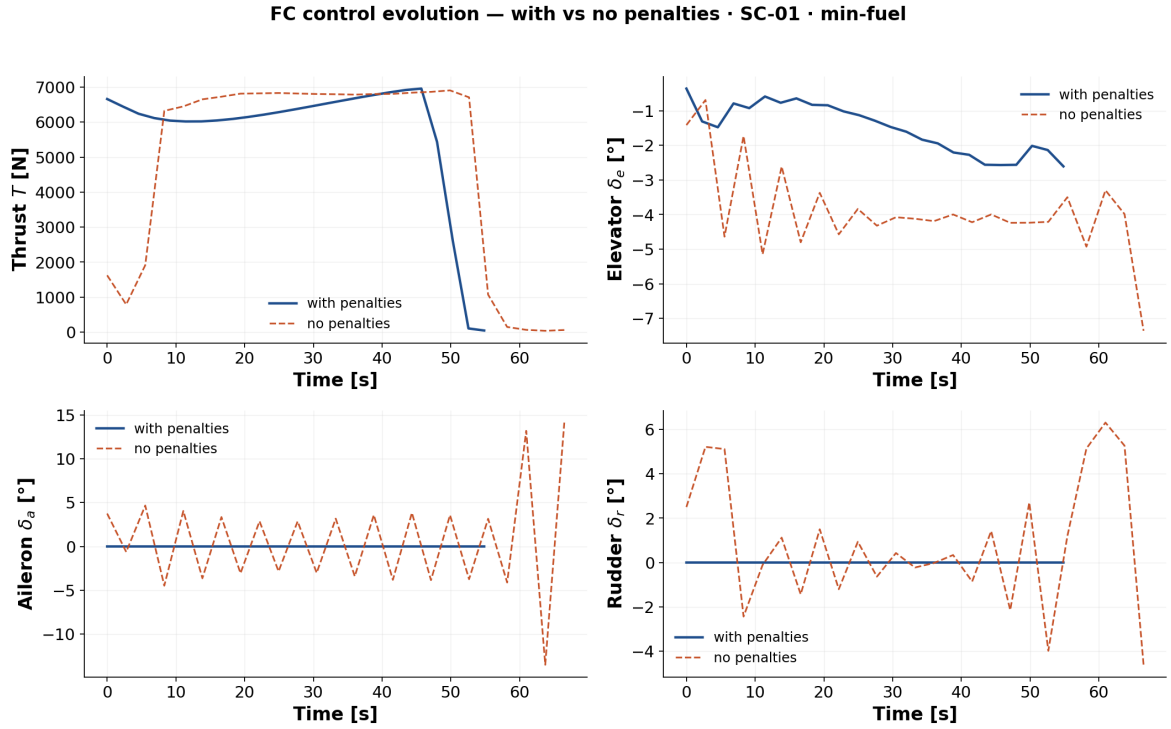


Figure 6.14: Control variable time histories for the Full-Control (FC) formulation in Scenario SC-01 (straight climb): comparison highlighting the effect of active regularisation penalties ($\beta_c = \beta_s = 1$) in suppressing high-frequency bang-bang chatter against the unregularised controls ($\beta_c = \beta_s = 0$).

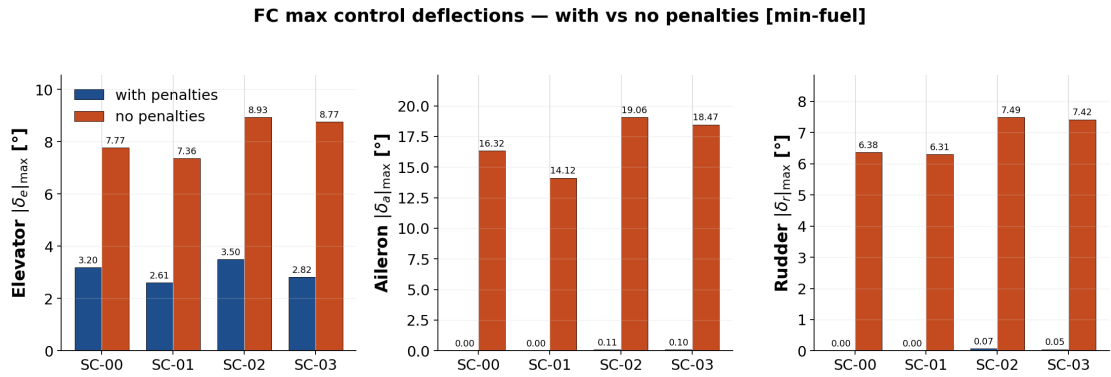


Figure 6.15: Aerodynamic control-surface deflections ($\delta_e, \delta_a, \delta_r$) of the FC formulation under the minimum-fuel criterion, comparing the regularised production setting ($\beta_c = \beta_s = 1$) against the unregularised ablation ($\beta_c = \beta_s = 0$). Solid blue: regularised; dashed red: unregularised. The aileron and rudder axes concentrate the chatter that the smoothness penalties suppress.

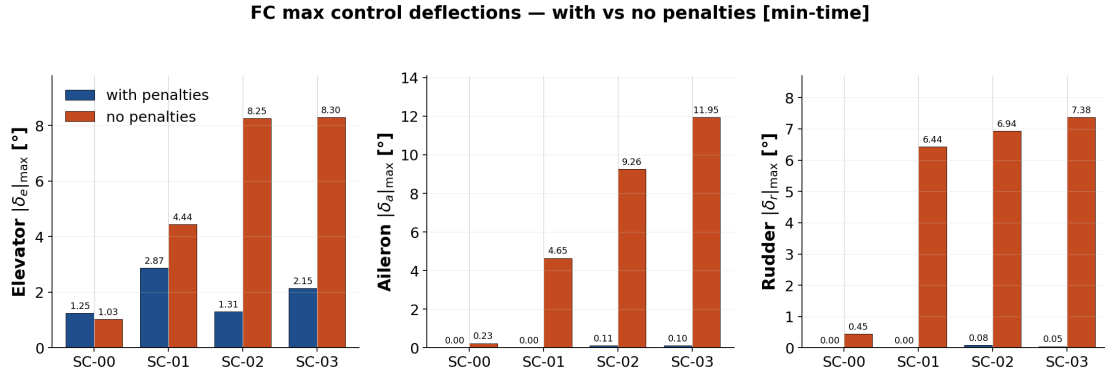


Figure 6.16: Counterpart of Figure 6.15 under the minimum-time criterion. The same regularisation gates are compared on the three actuator axes; despite the very different operating point (high-power, high-speed) the unregularised aileron and rudder still display the bang-bang behaviour that the penalties cure.

Of the regularisation weights defined in Equations (6.16) and (6.17), the controls contributing most to the total penalty mass at the converged solution are $w_{\Delta\delta_e}$ and $w_{\Delta T}$ for the FC formulation, and $w_{\Delta C_L}$ for the UF formulation. These specific axes correspond to the channels where the unregularised discrete optimum exhibits its largest oscillation amplitudes, aligning with the diagnosis in Section 6.3.2.

Mitigating these high-frequency control oscillations is paramount for practical engineering applications. In an unregularised formulation, the mathematical solver frequently introduces rapid, non-physical control chattering (bang-bang behaviour) to squeeze out marginal numerical gains (especially seen in the deflection controls of the aircraft in Figure 6.14). By penalising these sharp variations, the regularisation framework ensures smooth control profiles that avoid unrealistic actuator fatigue and thermal stress. Consequently, this yields physically realisable flight paths and significantly more realistic trajectory profiles that conform to true aircraft operational limits.

6.6 Computational Acceleration via Warm-Start Strategy

The numerical implementation developed in Sections 6.1–6.4 is mathematically complete: any feasible initial guess together with a sufficiently small barrier parameter would in principle drive the interior-point algorithm to a KKT point of the discretised NLP. In practice, however, the convergence rate of an interior-point method is highly sensitive to the quality of the initial iterate $\mathbf{z}_0$, and the cost of each linear sub-problem grows with the dimension and the conditioning of the constraint Jacobian. This section presents the *warm-start* chain employed to construct a near-feasible initial guess prior to any NLP call, together with the additional algorithmic choices that reduce the per-iteration cost of the optimiser. The section closes with the quantitative analysis of how these choices affect convergence and wall-clock time on the four reference scenarios of Section 6.4.1.

6.6.1 Role of the Initial Guess in Interior-Point Optimisation

Interior-point methods transform the constrained NLP into a sequence of unconstrained barrier sub-problems parameterised by a decreasing barrier parameter $\mu > 0$. For a given μ , the algorithm seeks a stationary point of the log-barrier augmented Lagrangian

$$\mathcal{L}_\mu(\mathbf{z}, \boldsymbol{\lambda}) = F(\mathbf{z}) - \mu \sum_{j=1}^{N_I} \ln(s_j(\mathbf{z})) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{z}), \quad (6.19)$$

which balances three core mathematical components: the primary cost function $F(\mathbf{z})$ to be minimised; a logarithmic barrier term weighted by μ that utilises positive slack variables $s_j(\mathbf{z}) > 0$ to penalise proximity to inequality boundaries; and a vector of equality constraints $\mathbf{g}(\mathbf{z})$ scaled by the Lagrange multipliers $\boldsymbol{\lambda}$. As an iterate approaches a boundary, the slacks s_j approach zero, driving the logarithmic penalty to infinity and forcing the primal variables $\mathbf{z}$ to remain strictly inside the feasible region. As $\mu \rightarrow 0$, the influence of this barrier vanishes, and the solution sequence of (6.19) converges to a KKT point of the original problem [19], [39], [40].

The convergence rate of this sequence depends critically on how close the initial point $\mathbf{z}_0$ is to the primal-dual feasible manifold. If $\mathbf{z}_0$ violates many constraints simultaneously, the optimiser must first drive the slacks away from zero before it can begin reducing the objective, consuming many iterations in a recovery phase. For the trajectory NLP in particular — where the dominant equality block is the $(N - 1) \times N_X$ collocation defect (6.10) — a random or linear initial guess typically produces defect norms $\|\Delta_k\|$ several orders of magnitude above the solver tolerance, triggering this expensive recovery.

A warm start addresses this by constructing $\mathbf{z}_0$ such that the defect norms and the key inequality violations are already small, placing the iterate in the neighbourhood of the feasible manifold before the first NLP call. The strategy adopted here builds this initial guess through two sequential layers: a

geometric path-planning stage in $\mathbb{R}^2$, followed by an inverse-dynamics trajectory-reconstruction stage that lifts the geometric path into a physically consistent state-control sequence.

6.6.2 Hybrid A* Path Planner

The first layer of the warm-start chain is a Hybrid A* search [48], whose objective is to deliver, as cheaply as possible, a kinematically feasible two-dimensional centre-line in the horizontal plane that connects the mission start and goal poses while avoiding all declared no-fly zones. The output is a sequence of poses (x_k, y_k, ψ_k) in the $\mathbb{R}^2$ configuration space [59], with the heading ψ retained as a third search coordinate; altitude, speed, attitude and control-surface commands are deliberately left to the inverse-dynamics layer and to the NLP. Restricting the search to a planar graph keeps it cheap enough to be invoked at the very top of the warm-start chain while replacing the piecewise-linear, infinite-curvature corners returned by classical grid A* [7] by a path that already respects the minimum-turn-radius constraint of the aircraft [15], [48], [60].

The search domain is a 2-D occupancy grid built from the bounding box of the start, goal and NFZ vertices, expanded by $1.5 R_{\min}$ and discretised with 25 m cells. The minimum turn radius itself is fixed by the active bank-angle limit,

$$R_{\min} = \frac{V_{\text{cruise}}^2}{g \tan \phi_{\text{active}}}, \quad (6.20)$$

and is enforced at every node expansion through the unicycle kinematic model

$$\dot{x} = V \cos \psi, \quad \dot{y} = V \sin \psi, \quad \dot{\psi} = V/R, \quad (6.21)$$

integrated in closed form over a fixed arc length Δs for $R \in \{-R_{\min}, \infty, +R_{\min}\}$ (left turn, straight, right turn). The resulting successor of a node $n_k = (x_k, y_k, \psi_k)$ is

$$\begin{aligned} x_{k+1} &= x_k + R [\sin(\psi_k + \Delta s/R) - \sin \psi_k], \\ y_{k+1} &= y_k - R [\cos(\psi_k + \Delta s/R) - \cos \psi_k], \\ \psi_{k+1} &= \psi_k + \Delta s/R, \end{aligned} \quad (6.22)$$

so that the children of n_k live in the continuous plane and only their containing cell is used for closed-set bookkeeping and collision queries. Successors falling inside an obstructed cell are discarded, which enforces collision-free feasibility at every step. Nodes in the open set are then ranked by the standard best-first estimate $f(n) = g(n) + h(n)$, with $g(n)$ the arc length already travelled along the integrated primitives (plus a small additive penalty whenever the turning command changes sign) and $h(n)$ an admissible lower bound on the remaining cost to the goal; the search terminates as soon as a node falls within the tolerance region of the goal pose.

$$f(n) = g(n) + h(n). \quad (6.23)$$

The heuristic $h(n)$ is taken as the maximum of two admissible lower-bound estimates: (i) the holonomic Euclidean distance from n to the goal, which ignores both obstacles and curvature; and (ii) the non-holonomic Dubins lower bound, defined as the length of the shortest curve of bounded curvature $1/R_{\min}$ joining the current pose (x, y, ψ) to the goal pose (x_g, y_g, ψ_g) [61]. Dubins proved that this minimiser is always one of six closed-form {LSL, LSR, RSL, RSR, LRL, RLR} primitives composed of circular arcs of radius $R_{\min}$ (L/R) and straight segments (S), so its evaluation is essentially free compared to an expansion and injects the minimum-turn-radius constraint of the fixed-wing aircraft [26] directly into the priority ordering; Reeds–Shepp paths [27] are not considered because the aircraft cannot fly backwards. In practice the Dubins branch dominates near the goal, where heading alignment becomes constraining, whereas the Euclidean branch dominates in the free-space cruise portion of long missions. Taking the maximum of the two is admissible and consistent, so the search expands nodes in optimal order while pruning dead-ends early.

6.6.3 Inverse-Dynamics Trajectory Reconstruction

The Hybrid A* output is a sequence of (x_k, y_k, ψ_k) triples that is kinematically consistent but carries no information about speed, altitude, aerodynamic forces or mass. The inverse-dynamics layer [3] closes this gap by deriving a full state–control trajectory that approximately satisfies the aircraft equations of motion at every node. The variable-density Hybrid A* waypoints are first resampled to exactly N arc-length-spaced nodes by linear interpolation,

$$\mathbf{r}(s) = \mathbf{r}_k + \frac{s - \ell_k}{\ell_{k+1} - \ell_k} (\mathbf{r}_{k+1} - \mathbf{r}_k), \quad s \in [\ell_k, \ell_{k+1}], \quad (6.24)$$

with ℓ_k the cumulative arc length up to node k , and the time step is fixed at $\Delta t = L_{\text{total}}/(V_0(N - 1))$ under a constant-speed assumption $V_0 = \|\mathbf{V}_{\text{gnd},0}\|$. The heading χ_k and the flight-path angle γ_k are then recovered by finite differences of the resampled positions (forward at the first node, backward at the last, central elsewhere).

The bank angle and the three body angular rates follow analytically under the coordinated-turn assumption (no sideslip, wings level with respect to the local lift vector):

$$\phi_k = \arctan\left(\frac{V_0 \dot{\chi}_k}{g}\right), \quad q_k = \dot{\gamma}_k, \quad r_k = \dot{\chi}_k \cos \gamma_k, \quad p_k = \dot{\phi}_k - r_k \tan \gamma_k, \quad (6.25)$$

with $\dot{\chi}_k$ and $\dot{\gamma}_k$ obtained by finite differences of the extracted angles. All computed rates are subsequently clamped to the structural limits $(p_{\max}, q_{\max}, r_{\max})$ so that the warm-start trajectory enters the NLP already inside the rigid-body box constraints listed in Section 6.2.4.

With the kinematic state $(\mathbf{r}_k, V_0, \phi_k, \gamma_k, \chi_k, p_k, q_k, r_k)$ known at every node, a Newton-based 6-DoF trim solver is invoked sequentially for $k = 0, \dots, N - 1$ [50] to find the thrust T_k , the elevator deflection $\delta_{e,k}$ and the body-frame velocity components (u_k, v_k, w_k) that satisfy the rigid-body force and moment

balance,

$$\mathbf{f}_{\text{trim}}(T_k, \delta_{e,k}, u_k, v_k, w_k; \phi_k, \gamma_k, \chi_k, p_k, q_k, r_k, m_k) = \mathbf{0}, \quad (6.26)$$

while the mass is updated by integrating the propulsive fuel-burn model forward in time,

$$m_{k+1} = m_k - \frac{T_k}{\eta_p V_0 c_{\text{sfc}}} \Delta t, \quad (6.27)$$

with c_{sfc} the specific fuel consumption coefficient. Each node is trimmed in isolation and the result is therefore not globally optimal, but its collocation defect is typically two to three orders of magnitude smaller than that of a linear-interpolation initial guess, which is precisely the property required from a warm start.

6.6.4 UF $\rightarrow$ FC State Projection

After the UF NLP converges, its solution must be mapped into the fourteen-dimensional FC state space before the rigid-body NLP can be initialised. A direct linear interpolation of the nine UF states would leave the body angular rates, the cross-track velocities and the control-surface deflections undefined, producing a large initial defect in the FC collocation constraints (6.10). The projection used here performs this mapping analytically at every FC node.

For each node i the UF state $(V_i, \mu_i, \gamma_i, \xi_i)$ is used to reconstruct the aerodynamic angle of attack from the linearised lift relation:

$$\alpha_i = \theta_i - \gamma_i, \quad \theta_i \approx \gamma_i + \frac{C_{L_0} + C_{L_\alpha} \alpha_i - C_{L,\text{eq}}}{C_{L_\alpha}}, \quad C_{L,\text{eq}} = \frac{2m_i g}{\rho V_i^2 S_w}, \quad (6.28)$$

where $C_{L,\text{eq}}$ is the equilibrium lift coefficient required to balance weight at the current speed. The Euler angles are then assigned as $(\phi_i, \theta_i, \psi_i) = (\mu_i, \theta_i, \xi_i)$; the body velocities (u_i, v_i, w_i) follow from the standard aerodynamic-to-body rotation; the angular rates are derived from the coordinated-turn relations (6.25); and the elevator deflection is computed from the longitudinal trim condition. This analytical projection places the FC initial guess in the near-feasible region of the rigid-body state space, so that the FC NLP requires far fewer iterations than a cold start would.

6.6.5 Numerical Pipeline Overview

The numerical solution of the discretised OCP follows a sequential pipeline that exploits the increasing fidelity of the two flight models as a built-in initial-guess refinement. Figure 6.17 summarises the four numerical layers involved.

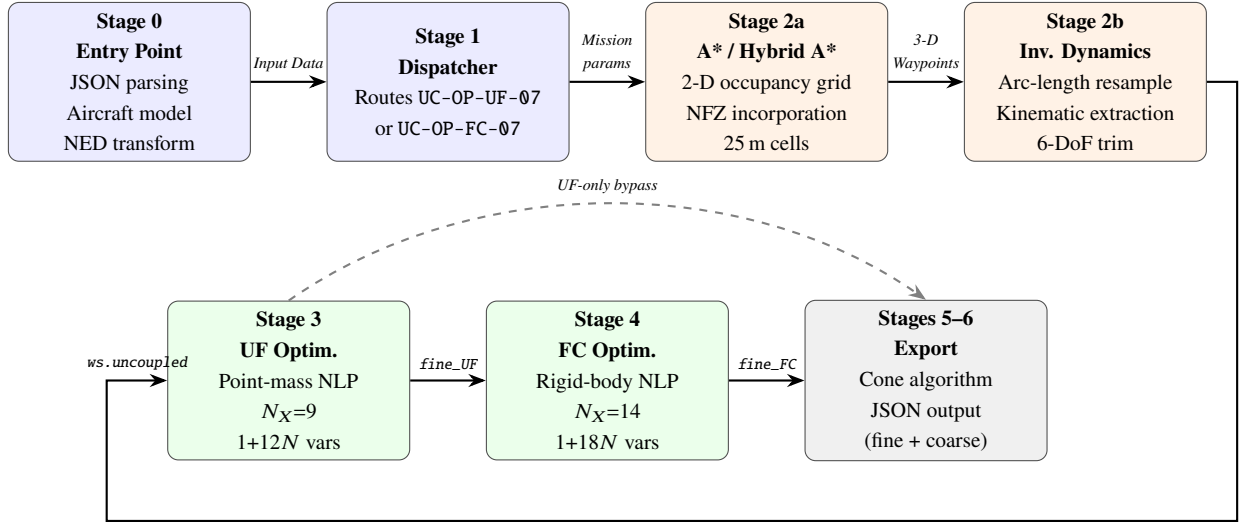


Figure 6.17: Schematic of the numerical pipeline used to transcribe and solve the continuous OCP. The geometric layer and inverse-dynamics stage produce a near-feasible initial guess that warm-starts the UF NLP; the converged UF solution, after a state-space projection, in turn warm-starts the FC NLP. The dashed arrow indicates the lower-fidelity UF-only output used in scenarios that do not require rigid-body validation. The mathematical content of each block is detailed in the corresponding section of this chapter.

The four layers correspond to four distinct mathematical problems of increasing fidelity: (i) the geometric layer searches a kinematically feasible centre-line in $\mathbb{R}^2$ subject to the minimum turn radius; (ii) the inverse-dynamics stage solves a sequence of 6-DoF nodewise trim problems whose union approximately satisfies the rigid-body equations of motion; (iii) the UF NLP optimises the discretised point-mass OCP with all path and performance constraints active; and (iv) the FC NLP refines the converged UF solution to satisfy the additional rigid-body constraints. The mathematical formulation of layers (iii) and (iv) is the subject of Sections 6.4–6.4; the geometric layer and inverse-dynamics stage are formalised as the warm-start strategy in Section 6.6.

6.6.6 Warm-Start Performance Results

This subsection quantifies the impact of the warm-start architecture on the convergence, the computational cost and the optimality of the recovered trajectories. The central claim is that a physics-informed initial guess, generated by a kinodynamic Hybrid A* planner and lifted to a dynamically consistent state by the inverse-dynamics layer, removes the cold-start pathologies of interior-point optimisation and, when chained through the reduced (UF) formulation before the full-control (FC) solve, conditions the harder coupled problem so that it converges faster and to a more optimal solution. The benchmark exercises eight scenarios — the four validation cases of Section 6.4 plus four additional missions with non-trivial geometry — grouped into four families: `empty` (scenarios 1–4), `nfz_only` (scenarios 5–6), `waypoint_only` (scenario 7) and `nfz_waypoint` (scenario 8). Each scenario is solved under two objectives, `min_time` and `min_fuel`, as two independent comparisons: a UF comparison that isolates

the value of the warm start (WS-ON, the Hybrid A* guess) against the cold linear interpolation (WS-OFF), and an FC comparison that isolates the value of the intermediate UF stage, contrasting the full pipeline WS→UF→FC (FC seeded by the UF optimum) against WS→FC (FC seeded directly by the inverse-dynamics trajectory, skipping the UF solve).

Reading the tables. Throughout this subsection a *positive* percentage denotes an improvement: a positive *speedup* means the indicated pipeline is faster, a positive iteration figure means it uses fewer IPOPT iterations. Four conventions fix the meaning of the numbers. First, every scenario and variant was run repeatedly and the reported computational times are the *mean over repetitions*, whereas the IPOPT iteration counts and the compared trajectory are taken from the *fastest* repetition. Second, the IPOPT iteration counts are *whole-pipeline totals*: for WS→UF→FC the count is the sum of the UF-stage and the FC-stage iterations, while for WS→FC it is the single FC solve alone. Third, a cell shown as “–” marks a run that did not converge or that saturated the computational-time cap (overtime); such a run is retained as a row but is *excluded from every mean*, so that the aggregate figures compare only those scenarios in which both sides produced a usable solution. Fourth, *feasibility is decided by the final optimisation stage* of each run — the UF solve for a UF comparison, the FC solve for an FC comparison — so that a UF failure inside an FC run is only a diagnostic on the seed and does not by itself make the FC result infeasible. The aggregate of all four (objective, formulation) pairs is consolidated in Table 6.8.

Table 6.8: Aggregate means across all scenarios. UF rows: warm-start vs. no-warm-start. FC rows: WS→UF→FC vs. WS→FC (positive ⇒ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Objective	Form.	Mean Δ time	Mean Δ iters	Feasible
min_time	UF	-8.6%	-0.7%	5/8
min_time	FC	41.8%	-38.3%	7/8
min_fuel	UF	9.0%	18.5%	5/8
min_fuel	FC	49.1%	-21.4%	7/8

The Hybrid A* Guess Generator

The warm start replaces the geometry-blind linear interpolation with a guess that already encodes the topology of the mission environment. The Hybrid A* planner performs a kinodynamic graph expansion in which every node transition is a short motion primitive whose curvature is bounded by the aircraft minimum turning radius $R_{\min}$, so the resulting path is flyable by construction and never demands a turn sharper than the airframe can execute. The search is carried out on a discretised representation of the mission area in which the no-fly zones (NFZs) are encoded as blocked regions: the expansion therefore *avoids the NFZs explicitly* rather than discovering them as constraint violations during optimisation, and it threads the mission waypoints and special geometric features in the prescribed order. The

discrete path is then handed to the inverse-dynamics stage (Section 6.6.3), which recovers a physically consistent 15-degree-of-freedom trajectory along it, supplying the optimiser with a complete state and control history rather than a bare position curve. The product is a guess that lies in the correct homotopy class — it passes on the admissible side of every NFZ and honours every waypoint — so the optimiser starts inside the basin of attraction of a feasible local minimum instead of having to discover the environment from scratch. The cost of producing this guess is modest: across the eight `min_time` runs the Hybrid A* preplan takes between 41 and 193 ms and the inverse-dynamics lift a further 1.6 to 22.9 ms, a fixed overhead that the following sections weigh against the optimisation savings it unlocks.

Failure of the Cold Start

The cold start interpolates linearly between the start and end points only. This straight segment ignores the environment entirely: it makes no reference to the NFZs and no reference to the mission geometry, and in any scenario with an active spatial constraint it is born infeasible, placing the optimiser in a region from which the interior-point method must recover before it can even begin to optimise. Table 6.9 reports, for the `min_time` objective and the UF formulation, the wall-clock total time, the IPOPT iteration count and the feasibility outcome under both start strategies.

Table 6.9: Computational and iteration overview (UF, `min_time`): warm-start (WS) vs. no-warm-start (No-WS). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	Total time [ms]			IPOPT iterations			Dev. [%]
	WS	No-WS	Speedup	WS	No-WS	Speedup	
scenario 1	525	452	-16.2%	13	13	0.0%	0.2
scenario 2	737	636	-15.9%	17	17	0.0%	0.2
scenario 3	1352	1269	-6.5%	30	28	-7.1%	0.0
scenario 4	1233	1025	-20.3%	25	22	-13.6%	0.1
scenario 5	2393	–	–	39	–	–	–
scenario 6	2225	–	–	33	–	–	–
scenario 7	2935	–	–	39	–	–	–
scenario 8	2338	2783	16.0%	38	46	17.4%	13.6
Mean			-8.6%			-0.7%	

The decisive effect of the geometry-aware guess is feasibility, not raw speed. In the three constrained scenarios the cold start does not converge at all: scenario 5 (`nfz_only`) terminates with `Restoration_Failed`, unable to recover feasibility, while scenarios 6 (`nfz_only`) and 7 (`waypoint_only`) return `Infeasible_Problem_Detected`. Each of these cold-start cells is therefore shown as “–” and excluded from the means, whereas the warm-started UF solve converges in all three (39, 33 and 39 iterations, in 2.39, 2.22 and 2.94 s respectively). Counting feasibility by the final UF stage, the cold start succeeds in only 5/8 scenarios against 8/8 for the warm start: the warm start is the only strategy that solves the constrained missions at all, which is precisely the regime an

operational planner must handle.

Where both strategies do converge — the unconstrained `empty` family and the `nfz_waypoint` scenario 8 — the mean tells the complementary half of the story. The mean UF speedup over those five comparable scenarios is in fact *negative*, at -8.6% in time and -0.7% in iterations, because the comparable scenarios are exactly the easy ones: in scenarios 1–4 the linear interpolant already lies inside the trust region of these short missions, the trajectory deviations collapse below 0.3% (both solvers reach the same optimum), and the fixed Hybrid A* + inverse-dynamics overhead is not amortised by an NLP saving that is essentially absent, so the warm start regresses by 16.2% , 15.9% , 6.5% and 20.3% . The single comparable scenario in which the warm start is faster is scenario 8, at $+16.0\%$ time and $+17.4\%$ iterations, where the NFZ-and-waypoint geometry is non-trivial enough for the better-conditioned guess to pay off. The conclusion is that the warm start is not a uniform accelerator on the UF stage; its value is concentrated where the cold start fails outright, and that value is invisible in any mean that can only average the scenarios where both succeed.

Figures 6.18 and 6.19 make the mechanism visible for two representative cases. Each is a top-down (East $\times$ North) view overlaying the Hybrid A* raw path, the guess produced by the Hybrid A* + inverse-dynamics chain, the warm-started UF optimum and the cold-started UF optimum, together with the NFZ polygons and the mission waypoints.

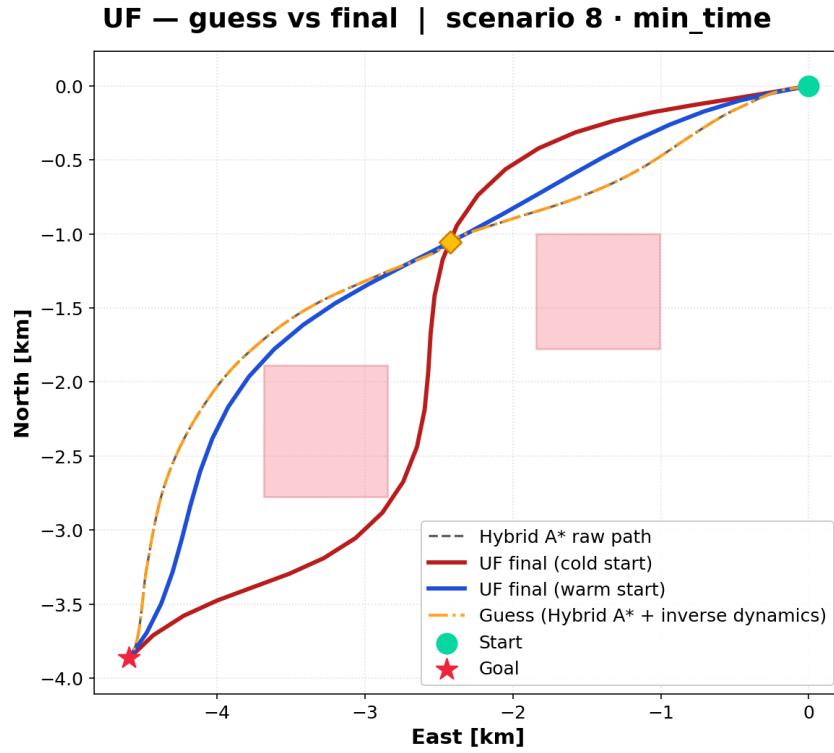


Figure 6.18: UF warm-start vs cold-start trajectory comparison for scenario 8 (nfz_waypoint, min_time), top-down East $\times$ North. The plot overlays the Hybrid A* raw path, the guess (Hybrid A* + inverse dynamics), the UF final warm-started trajectory and the UF final cold-started trajectory, with the NFZ polygons and the mission waypoints. The warm-started UF tracks the geometry-aware guess around the NFZ and through the waypoints, whereas the cold-started UF starts from the straight start-to-end segment.

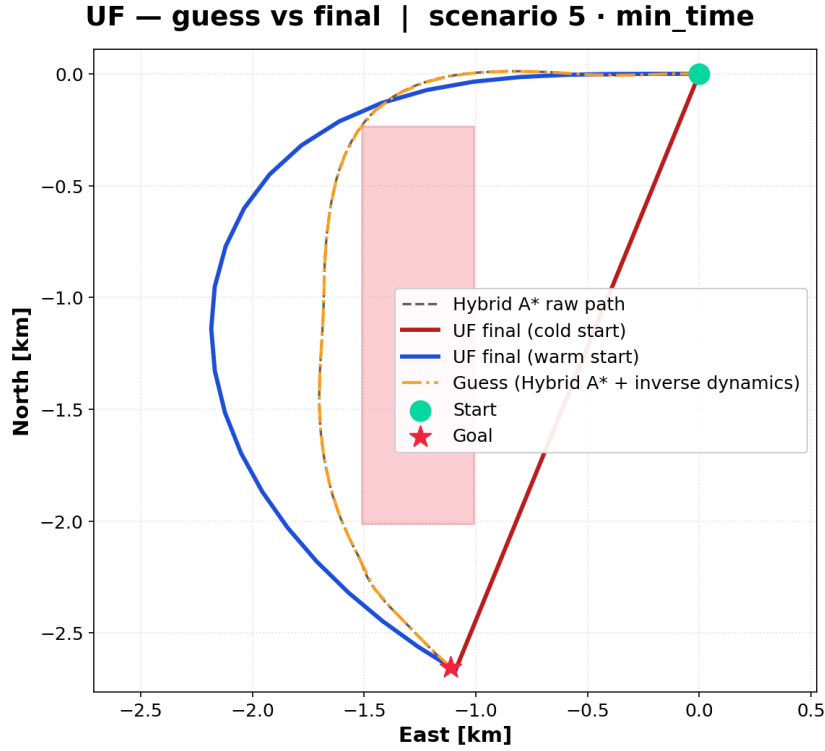


Figure 6.19: UF warm-start vs cold-start trajectory comparison for scenario 5 (`nfz_only`, `min_time`), top-down East $\times$ North, with the same curve set as Figure 6.18. The warm-started UF follows the Hybrid A* + inverse-dynamics guess around the NFZ; the cold-started UF, seeded by a straight segment that cuts through the forbidden region, fails to recover feasibility (`Restoration_Failed`) and is reported as “-” in Table 6.9.

In scenario 8 (Figure 6.18) both solvers converge, but the warm-started UF (following the orange dash-dotted guess) bends smoothly around the NFZ and threads the waypoints, while the cold-started UF must deform the straight start-to-end segment into a feasible path during the solve. In scenario 5 (Figure 6.19) the contrast is sharper: the straight cold-start guess passes directly through the NFZ, the optimiser cannot recover a feasible trajectory from it, and the run fails, whereas the warm start — already routed around the zone by Hybrid A* — converges without difficulty. These two figures are the visual counterpart of the “-” rows in Table 6.9.

Importance of the Full Pipeline

A natural question is whether the UF solve is necessary at all, given that the inverse-dynamics stage already returns a detailed 15-degree-of-freedom trajectory that could seed the FC solver directly. The answer is no, and the reason is the nature of the inverse-dynamics layer. It is a *quasi-stationary*, *trim-based* reconstruction: it recovers a dynamically consistent state along the Hybrid A* path under a sequence of local trim assumptions, and it therefore carries the modelling error of that quasi-stationary approximation and, crucially, *does not optimise the trajectory*. Seeding the FC solver directly from it

(WS→FC) hands the hardest problem in the pipeline — the fully coupled 15-DOF formulation, which converges poorly from a weak guess — a start that is feasible in geometry but neither optimal nor fully consistent with the coupled dynamics. The alternative is to first condition the guess through the reduced UF optimisation, whose optimum is both dynamically cleaner and already close to the cost-optimal trajectory, and to use that optimum to seed FC (WS→UF→FC). Table 6.10 contrasts the two for the `min_time` objective.

Table 6.10: Computational and iteration overview (FC, `min_time`): WS→UF→FC (baseline) vs. WS→FC (skip the UF solve). Iteration counts are whole-pipeline totals; WS→UF→FC sums the UF and FC solves. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	Total time [ms]			IPOPT iterations			Dev. [%]
	WS→UF→FC	WS→FC	$\Delta\%$	WS→UF→FC	WS→FC	$\Delta\%$	
scenario 1	4485	12104	62.9%	48	51	5.9%	3.8
scenario 2	5243	8163	35.8%	54	34	-58.8%	3.5
scenario 3	8457	12916	34.5%	75	48	-56.2%	4.5
scenario 4	10260	13950	26.5%	78	44	-77.3%	4.5
scenario 5	10460	12392	15.6%	93	51	-82.4%	3.7
scenario 6	12863	22669	43.3%	79	63	-25.4%	3.4
scenario 7	11608	–	–	78	–	–	–
scenario 8	10806	41419	73.9%	82	111	26.1%	3.3
Mean			41.8%			-38.3%	

The intermediate UF stage improves the FC convergence in every comparable scenario: WS→UF→FC is faster than WS→FC in all seven scenarios where both converge, by a mean of 41.8% and by as much as 73.9% (scenario 8) and 62.9% (scenario 1). The starkest case is scenario 7, where seeding FC directly from the inverse-dynamics trajectory saturates the computational-time cap (TIMEOUT, `Maximum_CpuTime_Exceeded`) and is reported as “–”, whereas routing the guess through the UF stage first brings the same problem to convergence in 11.6 s. Counting feasibility by the final FC stage, WS→UF→FC converges in all 8/8 scenarios against 7/8 for the direct WS→FC seed. The small FC deviations (all between 3.3% and 4.5%) confirm that, where both variants converge, they land on essentially the same trajectory: the UF stage does not change *which* solution FC finds, it changes how cheaply and how reliably FC reaches it. This matters precisely because FC is the harder, higher-fidelity problem; the weaker the seed, the more punishing its poor convergence, and the UF solve is the cheapest available way to strengthen that seed.

Iterations and Computational Cost

Iteration counts and wall-clock time tell complementary stories, and reading them together is essential to interpret the pipeline correctly. Table 6.11 lines up the two comparisons side by side: the UF warm-start speedup and iteration reduction, and the FC time and iteration differences between WS→UF→FC and

WS→FC.

Table 6.11: Per-scenario relative differences (min_time). UF: warm-start speedup / iteration reduction (positive ⇒ warm start cheaper). FC: WS→UF→FC vs. WS→FC (positive ⇒ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	UF (WS vs No-WS)		FC (WS→UF→FC vs WS→FC)	
	Speedup	Iter. redux	Δ time	Δ iters
scenario 1	-16.2%	0.0%	62.9%	5.9%
scenario 2	-15.9%	0.0%	35.8%	-58.8%
scenario 3	-6.5%	-7.1%	34.5%	-56.2%
scenario 4	-20.3%	-13.6%	26.5%	-77.3%
scenario 5	–	–	15.6%	-82.4%
scenario 6	–	–	43.3%	-25.4%
scenario 7	–	–	–	–
scenario 8	16.0%	17.4%	73.9%	26.1%

On the UF side, in the comparable scenarios time and iterations move together: where the warm start helps it helps on both metrics (scenario 8 at +16.0% time / +17.4% iterations) and where it hurts it hurts on both (the short empty missions), because the fixed Hybrid A* + inverse-dynamics overhead and the per-iteration cost are simply added to the same NLP; the three constrained scenarios contribute no UF gain figure because their cold-start baseline did not converge. The FC comparison, by contrast, exhibits a deliberate and informative *divergence* between the two metrics. In most scenarios WS→UF→FC records *more* total pipeline iterations yet *less* wall-clock time: scenario 5 takes 93 pipeline iterations against 51 for WS→FC (–82.4% on the iteration count) but still finishes 15.6% faster; scenario 4 shows 78 against 44 iterations (–77.3%) yet is 26.5% faster; and scenario 3 shows 75 against 48 (–56.2%) while being 34.5% faster. Two effects combine here. First, the WS→UF→FC count includes the cheap UF iterations, which inflate the total without costing much wall-clock. Second, and more importantly, the iterations that WS→FC does spend are individually very expensive: starting FC from the rough, non-optimised inverse-dynamics trajectory forces IPOPT into costly restoration steps, so each of its iterations carries a far larger per-iteration cost than the iterations of an FC solve seeded by the conditioned UF optimum. Scenario 8 makes the point numerically: the WS→FC solver spends 41.3 s over its 111 iterations (≈ 0.37 s per iteration), whereas the whole WS→UF→FC pipeline — UF solve included — completes in 10.8 s. The aggregate FC iteration figure is therefore negative (–38.3% mean, i.e. more iterations for the full pipeline) while the aggregate time is strongly positive (+41.8%): fewer but expensive restoration iterations lose to more but cheaper conditioned iterations. Only where the direct seed is so poor that it also takes more iterations — scenarios 1 and 8, at +5.9% and +26.1% iteration reductions *in favour* of the pipeline — do both metrics agree and the pipeline win on each.

Optimality of the Recovered Solutions

The final question is whether the warm start preserves, or even improves, the optimality of the recovered trajectory. Table 6.12 reports the achieved flight time — the quantity the `min_time` objective minimises — for both comparisons across all scenarios.

Table 6.12: Trajectory flight time [s] (`min_time`). UF: warm-start (WS) vs. no-warm-start (No-WS). FC: WS→UF→FC vs. WS→FC. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	UF [s]			FC [s]		
	WS	No-WS	Δ	WS→UF→FC	WS→FC	Δ
scenario 1	51.1	51.4	-0.3	52.8	53.2	-0.3
scenario 2	54.6	54.9	-0.3	55.3	56.5	-1.2
scenario 3	84.6	84.6	+0.0	87.3	87.4	-0.1
scenario 4	89.5	89.4	+0.1	87.3	87.4	-0.1
scenario 5	75.4	–	–	72.3	73.3	-1.0
scenario 6	94.7	–	–	86.2	90.3	-4.1
scenario 7	283.1	–	–	288.0	–	–
scenario 8	101.5	109.5	-8.0	105.1	107.8	-2.7
Mean Δ			-1.7			-1.4

For the FC comparison the recovered flight time is lower (more optimal) for the WS→UF→FC pipeline in every comparable scenario: the Δ column is negative throughout, with a mean of -1.4 s. The clearest evidence is the two constrained-geometry cases singled out above. In scenario 5 (`nfz_only`) the full pipeline converges to a 72.3 s trajectory against 73.3 s for the direct seed, a 1.0 s improvement; in scenario 8 (`nfz_waypoint`) it reaches 105.1 s against 107.8 s, a 2.7 s improvement. In both, the warm-started guess conditioned through UF converges to the genuinely minimum-time solution, whereas the weaker direct seed settles on a slightly slower trajectory. Scenario 6 shows the same effect at 86.2 s versus 90.3 s (-4.1 s); scenario 7 provides no comparison because the WS→FC run timed out and is shown as “–”.

The UF comparison must be read with the feasibility outcomes of Table 6.9 in mind. In the `empty` family the warm- and cold-started UF flight times agree to within 0.3 s, confirming that both reach the same optimum where the geometry is trivial, and in scenario 8 the warm start even shortens the flight time by 8.0 s (101.5 against 109.5 s). The three constrained scenarios 5–7 carry no UF flight-time comparison at all: their cold-start runs are infeasible and therefore shown as “–”, so the warm-started values (75.4, 94.7 and 283.1 s) are the only feasible minima available. Read together, the FC and UF metric columns show that the warm start does not trade optimality for robustness: it delivers the feasible minimum-time trajectory, and the UF→FC chain converges to a flight time at least as good as, and in the constrained cases strictly better than, the direct seed.

Minimum-Fuel Objective

The same trends hold under the `min_fuel` objective, for which the headline metric is the trajectory end mass (a higher end mass means less fuel burnt). The UF warm start again rescues the three constrained scenarios, where the cold start fails (`Restoration_Failed` on scenario 5, `Infeasible_Problem_Detected` on scenarios 6 and 7) and is excluded from the means, leaving feasibility at 5/8 for the cold start against 8/8 for the warm start; here the comparable-scenario mean UF speedup is even positive (+9.0% time, +18.5% iterations). The FC pipeline `WS→UF→FC` remains faster than the direct seed in every comparable scenario (mean +49.1%, with the same `WS→FC TIMEOUT` on scenario 7), and among the converged FC runs the two variants reach essentially the same end mass (differences below 0.1 kg), confirming that the value of the intermediate UF stage on this objective lies in convergence and cost rather than in a different optimum. The full per-scenario `min_fuel` tables (Tables A.1, A.2 and A.4) are collected in Annex A.

Chapter 7

Conclusions

7.1 Summary of Contributions

This thesis has developed an end-to-end pipeline for 3D minimum-time and minimum-fuel trajectory planning of a fixed-wing aircraft, covering the climb, cruise and descent phases and supporting both a point-mass (UC) and a full 6-DoF rigid-body (FC) formulation. The implementation has been validated on a suite of four reference scenarios and the impact of the regularisation penalties on the optimum has been characterised quantitatively.

7.2 Fulfilment of Requirements

7.3 Limitations

7.4 Future Work

- Closed-loop integration with an MPC layer using the offline trajectory as a reference.
- Stochastic wind modelling and chance-constrained reformulation of the NFZ and altitude constraints.
- Multi-aircraft / multi-mission coordination with shared airspace constraints.
- Replacement of the analytic aerodynamic model by a data-driven surrogate trained on flight-test data.

References

- [1] NASA Earth Science Data Systems. “Digital elevation / terrain model (dem),” National Aeronautics and Space Administration (NASA). (2024), [Online]. Available: <https://www.earthdata.nasa.gov/topics/land-surface/digital-elevation-terrain-model-dem> (visited on Mar. 3, 2026).
- [2] European Union Aviation Safety Agency. “Geo zones – know where to fly your drone.” Official source for aviation safety information for the general public., EASA Light. (2024), [Online]. Available: <https://www.easa.europa.eu/en/light/topics/geo-zones-know-where-fly-your-drone> (visited on Mar. 3, 2025).
- [3] P. Lu, “Inverse dynamics approach to trajectory optimization for an aerospace plane,” *Journal of Guidance, Control, and Dynamics*, vol. 16, no. 4, pp. 726–732, 1993. doi: [10.2514/3.21073](https://doi.org/10.2514/3.21073). [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/3.21073> (visited on Mar. 12, 2026).
- [4] SESAR Joint Undertaking, “U-space blueprint and european drone outlook study,” European Commission, Brussels, Belgium, Technical Report, 2023. [Online]. Available: <https://www.sesarju.eu/> (visited on May 6, 2026).
- [5] Federal Aviation Administration, “Faa aerospace forecast fiscal years 2024–2044,” U.S. Department of Transportation, Washington, D.C., Forecast Report, 2024. [Online]. Available: https://www.faa.gov/data_research/aviation/aerospace_forecasts/unmanned_aircraft_systems.pdf (visited on May 7, 2026).
- [6] A. Nuić, D. Poles, and V. Mouillet, “BADA: An advanced aircraft performance model for present and future ATM systems,” *International Journal of Adaptive Control and Signal Processing*, vol. 24, no. 10, pp. 850–866, 2010, Published online in Wiley InterScience. doi: [10.1002/acs.1176](https://doi.org/10.1002/acs.1176). [Online]. Available: <https://www.eurocontrol.int/sites/default/files/2019-03/overview-bada-apm.pdf> (visited on Mar. 12, 2026).
- [7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. doi: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136). [Online]. Available: <https://doi.org/10.1109/TSSC.1968.300136> (visited on May 4, 2026).

- [8] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Technical Report TR 98-11, 1998. [Online]. Available: <https://msl.cs.illinois.edu/~lavalle/papers/Lav98c.pdf> (visited on Mar. 10, 2026).
- [9] Singular Aircraft. “Flyox i: Technical specifications and mission configurations.” (2024), [Online]. Available: <https://singularaircraft.com/flyox> (visited on Mar. 10, 2026).
- [10] Singular Aircraft. “Flyox i: The largest amphibious unmanned aerial platform.” (2024), [Online]. Available: <https://singularaircraft.com/> (visited on Mar. 13, 2026).
- [11] F. Santoso, M. A. Garratt, and S. G. Anavatti, “State-of-the-art integrated guidance and control systems in unmanned vehicles: A review,” *IEEE Systems Journal*, vol. 15, no. 3, pp. 3312–3323, 2021. doi: [10.1109/JSYST.2020.3007428](https://doi.org/10.1109/JSYST.2020.3007428). [Online]. Available: <https://doi.org/10.1109/JSYST.2020.3007428> (visited on May 5, 2026).
- [12] T. A. Johansen, “Introduction to nonlinear model predictive control and moving horizon estimation,” Norwegian University of Science and Technology (NTNU), Technical Publication, 2011. [Online]. Available: <https://torarnj.folk.ntnu.no/nonlinear.pdf> (visited on Mar. 13, 2026).
- [13] Y. Zhang, H. Liang, and J. Li, “Research on UAV path planning based on improved Dijkstra’s algorithm,” *International Journal for Housing Science and Its Applications*, vol. 46, no. 3, pp. 7473–7482, Aug. 2025. doi: [10.70517/ijhsa463641](https://doi.org/10.70517/ijhsa463641). [Online]. Available: <https://doi.org/10.70517/ijhsa463641> (visited on Mar. 21, 2026).
- [14] ScienceDirect, *Trajectory planning - an overview*, Computer Science Topics, 2024. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/trajectory-planning> (visited on Mar. 21, 2026).
- [15] J. Li, C. Yu, Z. Zhang, Z. Sheng, Z. Yan, X. Wu, *et al.*, “Improved A-Star path planning algorithm in obstacle avoidance for the fixed-wing aircraft,” *Electronics*, vol. 12, no. 24, p. 5047, 2023. doi: [10.3390/electronics12245047](https://doi.org/10.3390/electronics12245047). [Online]. Available: <https://doi.org/10.3390/electronics12245047> (visited on Mar. 22, 2026).
- [16] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, L. E. Kavraki, and S. Thrun, *Probabilistic roadmap path planning*, Lecture notes based on *Principles of Robot Motion*, MIT Press., Columbia University, Department of Computer Science, 2015. [Online]. Available: <https://www.cs.columbia.edu/~allen/F15/NOTES/Probabilisticpath.pdf> (visited on Mar. 22, 2026).
- [17] G. Noh, J. Park, D. Han, *et al.*, “Selective goal aiming rapidly exploring random tree path planning for UAVs,” *International Journal of Aeronautical and Space Sciences*, vol. 22, no. 6, pp. 1397–1412, 2021. doi: [10.1007/s42405-021-00406-7](https://doi.org/10.1007/s42405-021-00406-7). [Online]. Available: <https://doi.org/10.1007/s42405-021-00406-7> (visited on Mar. 22, 2026).
- [18] S. P. Bradley, A. C. Hax, and J. F. Shapiro, “Applied mathematical programming,” in Reading, Mass.: Addison-Wesley, 1977, ch. 13: Non-linear Programming, Available at MIT OpenCourse-

- Ware. [Online]. Available: <https://web.mit.edu/15.053/www/AMP-Chapter-13.pdf> (visited on Mar. 22, 2026).
- [19] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006. doi: [10.1007/s10107-004-0559-y](https://doi.org/10.1007/s10107-004-0559-y). [Online]. Available: <https://doi.org/10.1007/s10107-004-0559-y> (visited on May 8, 2026).
- [20] A. Wächter, C. Laird, and COIN-OR Project, *Ipopt (interior point optimizer) documentation*, <https://coin-or.github.io/Ipopt/>, 2026. (visited on Mar. 22, 2026).
- [21] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi – A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019. doi: [10.1007/s12532-018-0139-4](https://doi.org/10.1007/s12532-018-0139-4). [Online]. Available: <https://doi.org/10.1007/s12532-018-0139-4> (visited on May 7, 2026).
- [22] H. Chibueze, *Symbolic regression with expression trees*, Kaggle Code Repository, 2023. [Online]. Available: <https://www.kaggle.com/code/henrychibueze/symbolic-regression-with-expression-trees> (visited on Mar. 22, 2026).
- [23] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd. New York: Springer, 2011, Reference for deterministic execution and memory constraints., ISBN: 978-1-4419-8237-7. doi: [10.1007/978-1-4419-8237-7](https://doi.org/10.1007/978-1-4419-8237-7). [Online]. Available: <https://doi.org/10.1007/978-1-4419-8237-7> (visited on May 8, 2026).
- [24] R. Sambharya, G. Hall, B. Amos, and B. Stellato, “Learning to warm-start fixed-point optimization algorithms,” *arXiv preprint*, vol. abs/2309.07835, 2023. arXiv: [2309.07835 \[math.OC\]](https://arxiv.org/abs/2309.07835). [Online]. Available: <https://arxiv.org/abs/2309.07835> (visited on May 9, 2026).
- [25] Emergent Mind. “Warm-start initialization,” Emergent Mind. (2026), [Online]. Available: <https://www.emergentmind.com/topics/warm-start-initialization> (visited on Mar. 22, 2026).
- [26] I. Lugo-Cardenas, G. Flores, S. Salazar, and R. Lozano, “Dubins path generation for a fixed wing UAV,” in *2014 International Conference on Unmanned Aircraft Systems (ICUAS)*, IEEE, 2014, pp. 339–346. doi: [10.1109/ICUAS.2014.6842272](https://doi.org/10.1109/ICUAS.2014.6842272). (visited on May 5, 2026).
- [27] T. Fraichard and A. Scheuer, “From Reeds and Shepp’s to continuous-curvature paths,” *IEEE Transactions on Robotics*, vol. 20, no. 6, pp. 1025–1035, Dec. 2004. doi: [10.1109/TRO.2004.833789](https://doi.org/10.1109/TRO.2004.833789). [Online]. Available: <https://inria.hal.science/inria-00000009> (visited on May 10, 2026).
- [28] J. Li, J. Lou, Y. Li, S. Pan, and Y. Xu, “Trajectory tracking of autonomous vehicle using Clothoid curve,” *Applied Sciences*, vol. 13, no. 4, p. 2733, 2023. doi: [10.3390/app13042733](https://doi.org/10.3390/app13042733). [Online]. Available: <https://www.mdpi.com/2076-3417/13/4/2733> (visited on May 9, 2026).

- [29] M. Davenport, “The Karush–Kuhn–Tucker (KKT) conditions,” *Georgia Tech ECE 3803 Course Notes*, Nov. 2021. [Online]. Available: <https://mdav.ece.gatech.edu/ece-3803-fall2021/notes/17-notes-3803-f21.pdf> (visited on Mar. 22, 2026).
- [30] A. Albert, L. Imsland, and J. Haugen, “Numerical optimal control mixing collocation with single shooting: A case study,” *IFAC-PapersOnLine*, vol. 49, no. 7, pp. 290–295, 2016, 11th IFAC Symposium on Dynamics and Control of Process Systems (DYCOPS-CAB). doi: [10.1016/j.ifacol.2016.07.307](https://doi.org/10.1016/j.ifacol.2016.07.307). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896316305146> (visited on Mar. 22, 2026).
- [31] S. Li, Z. Wang, A. Narayan, R. Kirby, and S. Zhe, *Meta-learning with adjoint methods*, arXiv preprint., 2021. arXiv: [2110.08432](https://arxiv.org/abs/2110.08432) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2110.08432> (visited on May 6, 2026).
- [32] A. M. Bradley, “PDE-constrained optimization and the adjoint method,” Stanford University, Technical Tutorial, 2010. [Online]. Available: https://cs.stanford.edu/~ambrad/adjoint_tutorial.pdf (visited on Mar. 22, 2026).
- [33] H. C. Elman and G. H. Golub, “Inexact and preconditioned Uzawa algorithms for saddle point problems,” *SIAM Journal on Numerical Analysis*, vol. 31, no. 6, pp. 1645–1673, 1994, Dedicated to Seymour Parter. doi: [10.1137/0731084](https://doi.org/10.1137/0731084). [Online]. Available: <https://www.cs.umd.edu/~elman/papers/inexact-uzawa.pdf> (visited on May 10, 2026).
- [34] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming* (Advances in Design and Control 19), 2nd. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2010, ISBN: 978-0-89871-688-7. doi: [10.1137/1.9780898718577](https://doi.org/10.1137/1.9780898718577). (visited on May 16, 2026).
- [35] C. R. Hargraves and S. W. Paris, “Direct trajectory optimization using nonlinear programming and collocation,” *Journal of Guidance, Control, and Dynamics*, vol. 10, no. 4, pp. 338–342, 1987. doi: [10.2514/3.20223](https://doi.org/10.2514/3.20223). (visited on May 14, 2026).
- [36] J. Nocedal and S. J. Wright, *Numerical Optimization* (Springer Series in Operations Research and Financial Engineering), 2nd. New York: Springer, 2006, ISBN: 978-0-387-30303-1. doi: [10.1007/978-0-387-40065-5](https://doi.org/10.1007/978-0-387-40065-5). (visited on May 13, 2026).
- [37] L. T. Biegler, *Nonlinear Programming: Concepts, Algorithms, and Applications to Chemical Processes* (MOS-SIAM Series on Optimization). Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2010, ISBN: 978-0-89871-702-0. doi: [10.1137/1.9780898719383](https://doi.org/10.1137/1.9780898719383). (visited on May 13, 2026).
- [38] A. Forsgren, P. E. Gill, and M. H. Wright, “Interior methods for nonlinear optimization,” *SIAM Review*, vol. 44, no. 4, pp. 525–597, 2002. doi: [10.1137/S0036144502414942](https://doi.org/10.1137/S0036144502414942). (visited on May 14, 2026).
- [39] H. Y. Benson, D. F. Shanno, and R. J. Vanderbei, “A comparative study of large-scale nonlinear optimization algorithms,” in *High Performance Algorithms and Software for Nonlinear*

- Optimization*, G. Di Pillo and A. Murli, Eds., Boston, MA: Kluwer Academic Publishers, 2003, pp. 95–127. DOI: [10.1007/978-1-4613-0241-4_5](https://doi.org/10.1007/978-1-4613-0241-4_5). (visited on May 16, 2026).
- [40] E. A. Yildirim and S. J. Wright, “Warm-start strategies in interior-point methods for linear programming,” *SIAM Journal on Optimization*, vol. 12, no. 3, pp. 782–810, 2002. DOI: [10.1137/S1052623400369235](https://doi.org/10.1137/S1052623400369235). (visited on May 17, 2026).
- [41] D. C. Liu and J. Nocedal, “On the limited memory BFGS method for large scale optimization,” *Mathematical Programming*, vol. 45, no. 1–3, pp. 503–528, 1989, Seminal reference for the L-BFGS quasi-Newton update used by interior-point solvers when exact Hessians are unavailable. DOI: [10.1007/BF01589116](https://doi.org/10.1007/BF01589116). [Online]. Available: <https://doi.org/10.1007/BF01589116> (visited on May 11, 2026).
- [42] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2008, ISBN: 978-0-89871-659-7. DOI: [10.1137/1.9780898717761](https://doi.org/10.1137/1.9780898717761). (visited on May 15, 2026).
- [43] J. D. Anderson, *Aircraft Performance and Design*. Boston: McGraw-Hill, 1999, Standard reference for the parabolic drag polar, Oswald efficiency factor and point-mass performance equations., ISBN: 978-0-070-01971-3.
- [44] B. Etkin and L. D. Reid, *Dynamics of Flight: Stability and Control*, 3rd. New York: John Wiley & Sons, 1996, Canonical reference for stability and control derivatives, body- and wind-axis derivations, and aircraft Newton–Euler equations of motion., ISBN: 978-0-471-03418-6.
- [45] A. V. Rao, “A survey of numerical methods for optimal control,” *Advances in the Astronautical Sciences*, vol. 135, no. 1, pp. 497–528, 2009. [Online]. Available: <http://www.anilvrao.com/Publications/ConferencePublications/trajectorySurveyAAS.pdf> (visited on May 13, 2026).
- [46] R. T. Marler and J. S. Arora, “Survey of multi-objective optimization methods for engineering,” *Structural and Multidisciplinary Optimization*, vol. 26, no. 6, pp. 369–395, 2004. DOI: [10.1007/s00158-003-0368-6](https://doi.org/10.1007/s00158-003-0368-6). (visited on May 17, 2026).
- [47] A. N. Tikhonov and V. Y. Arsenin, *Solutions of Ill-Posed Problems*. Washington, DC: V. H. Winston & Sons, 1977, Translated by Fritz John from the 1974 Russian edition., ISBN: 978-0-470-99124-4.
- [48] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, “Path planning for autonomous vehicles in unknown semi-structured environments,” *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010, Introduces the Hybrid-state A* algorithm with kinematic constraints. DOI: [10.1177/0278364909359210](https://doi.org/10.1177/0278364909359210). [Online]. Available: <https://doi.org/10.1177/0278364909359210> (visited on May 11, 2026).
- [49] P. Sharma, J. Y. M. Goh, B. Açıkmeşe, and F. Djeumou. “Transcription-induced failure modes in 6-DOF rocket landing trajectory optimization.” arXiv: [2605.08420 \[math.OC\]](https://arxiv.org/abs/2605.08420). (May 8, 2026), [Online]. Available: <https://arxiv.org/abs/2605.08420> (visited on May 16, 2026), preprint.

- [50] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*, 3rd. Hoboken, NJ: John Wiley & Sons, 2015, ISBN: 978-1-118-87098-3. DOI: [10.1002/9781119174882](https://doi.org/10.1002/9781119174882). [Online]. Available: <https://doi.org/10.1002/9781119174882> (visited on May 2, 2026).
- [51] M. P. Kelly, “An introduction to trajectory optimization: How to do your own direct collocation,” *SIAM Review*, vol. 59, no. 4, pp. 849–904, 2017. DOI: [10.1137/16M1062569](https://doi.org/10.1137/16M1062569). [Online]. Available: https://www.matthewpeterkelly.com/research/MatthewKelly_IntroTrajectoryOptimization_SIAM_Review_2017.pdf (visited on May 16, 2026).
- [52] C. Goerzen, Z. Kong, and B. Mettler, “A survey of motion planning algorithms from the controls, robotics, and cognitive science communities,” *Journal of Intelligent & Robotic Systems*, vol. 57, no. 1–4, pp. 65–100, 2010. DOI: [10.1007/s10846-009-9383-1](https://doi.org/10.1007/s10846-009-9383-1). (visited on May 4, 2026).
- [53] M. Diehl. “Direct single and direct multiple shooting methods,” University of Freiburg, Department of Microsystems Engineering (IMTEK). (2015), [Online]. Available: <https://www.imtek.de/professuren/systemtheorie/events/dateien/directshootingmethods> (visited on Apr. 10, 2026).
- [54] VectorNav Technologies. “Inertial navigation primer: Reference frames,” VectorNav Technologies. (2024), [Online]. Available: <https://www.vectornav.com/resources/inertial-navigation-primer/math-fundamentals/math-refframes> (visited on Apr. 12, 2026).
- [55] International Organization for Standardization, *ISO 2533:1975 — standard atmosphere*, International Standard, Defines the International Standard Atmosphere (ISA) used to compute air density as a function of geometric altitude., Geneva, Switzerland, 1975. [Online]. Available: <https://www.iso.org/standard/7472.html> (visited on May 3, 2026).
- [56] D. E. Hoak and R. D. Finck, “USAF stability and control DATCOM,” Air Force Wright Aeronautical Laboratories, Wright-Patterson AFB, Tech. Rep. AFFDL-TR-79-3032, 1978, Revised. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADB072483> (visited on May 18, 2026).
- [57] P. E. Gill, W. Murray, and M. H. Wright, *Practical Optimization*. London: Academic Press, 1981, ISBN: 978-0-12-283950-4. [Online]. Available: <https://archive.org/details/practicaloptimiz0000gill> (visited on May 15, 2026).
- [58] A. R. Curtis, M. J. D. Powell, and J. K. Reid, “On the estimation of sparse Jacobian matrices,” *Journal of the Institute of Mathematics and Its Applications*, vol. 13, pp. 117–119, 1974. [Online]. Available: <https://academic.oup.com/imamat/article-abstract/13/1/117/719659> (visited on May 16, 2026).
- [59] S. M. LaValle, *Planning Algorithms*. Cambridge: Cambridge University Press, 2006, ISBN: 978-0-521-86205-9. DOI: [10.1017/CB09780511546877](https://doi.org/10.1017/CB09780511546877). [Online]. Available: <https://lavalle.pl/planning/> (visited on May 18, 2026).
- [60] S. Zhang, Q. Guo, W. Liu, H. Kang, D. Wu, and H. Zhou, “Hybrid A-Star path planning method based on directional guidance and segment-based heuristic weighting,” *Applied Sciences*,

- vol. 15, no. 22, p. 12 298, 2025. doi: [10.3390/app152212298](https://doi.org/10.3390/app152212298). [Online]. Available: <https://doi.org/10.3390/app152212298> (visited on May 25, 2026).
- [61] L. E. Dubins, “On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents,” *American Journal of Mathematics*, vol. 79, no. 3, pp. 497–516, 1957. doi: [10.2307/2372560](https://doi.org/10.2307/2372560). (visited on May 19, 2026).

Appendix A

Warm-start benchmark: minimum-fuel detail

This annex backs Section 6.6.6 with the complete per-scenario tables for the `min_fuel` objective, whose trends are summarised in the body but whose full detail is deferred here. The benchmark runs every scenario under both objective criteria (`min_time`, `min_fuel`) and both formulations (UF, FC), as the two independent comparisons described in the body: the UF comparison contrasts the Hybrid A* + inverse-dynamics warm start (WS-ON) with the cold linear-interpolation start (WS-OFF), and the FC comparison contrasts the full pipeline WS→UF→FC with the direct seed WS→FC that skips the UF solve. The aggregate of all four (objective, formulation) pairs is given in the body by Table 6.8.

Conventions. As in the body, a positive percentage denotes an improvement (a positive speedup is faster, a positive iteration figure uses fewer IPOPT iterations). Reported computational times are the mean over repetitions, while iteration counts and the compared solution come from the fastest repetition; iteration counts in the FC comparison are whole-pipeline totals, with the WS→UF→FC column summing the UF and FC solves. A cell shown as “-” is a run that did not converge or saturated the computational-time cap (overtime); it is retained as a row but excluded from every mean. Feasibility is decided by the final optimisation stage of each run. The deviation column Δ reports the relative trajectory deviation between the two compared solutions.

A.1 Minimum-fuel criterion: per-scenario detail

Tables A.1 and A.2 report the computational time, the IPOPT iteration count and the feasibility outcome for the UF and FC comparisons under the `min_fuel` objective; Table A.3 lines the two comparisons up side by side, and Table A.4 reports the achieved trajectory end mass (the quantity the objective

maximises).

Table A.1: Computational and iteration overview (UF, min_fuel): warm-start (WS) vs. no-warm-start (No-WS). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	Total time [ms]			IPOPT iterations			Dev. [%]
	WS	No-WS	Speedup	WS	No-WS	Speedup	
scenario 1	957	1562	38.8%	16	18	11.1%	0.1
scenario 2	510	669	23.7%	11	18	38.9%	0.7
scenario 3	1145	1697	32.5%	23	36	36.1%	2.8
scenario 4	1669	1472	-13.4%	26	27	3.7%	2.9
scenario 5	2274	–	–	47	–	–	–
scenario 6	2553	–	–	32	–	–	–
scenario 7	2823	–	–	36	–	–	–
scenario 8	3048	2227	-36.9%	38	39	2.6%	13.3
Mean			9.0%			18.5%	

Table A.2: Computational and iteration overview (FC, min_fuel): WS→UF→FC (baseline) vs. WS→FC (skip the UF solve). Iteration counts are whole-pipeline totals; WS→UF→FC sums the UF and FC solves. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	Total time [ms]			IPOPT iterations			Dev. [%]
	WS→UF→FC	WS→FC	Δ%	WS→UF→FC	WS→FC	Δ%	
scenario 1	6653	11233	40.8%	55	44	-25.0%	4.2
scenario 2	5205	12073	56.9%	48	47	-2.1%	4.0
scenario 3	11333	21330	46.9%	83	59	-40.7%	4.4
scenario 4	12217	17139	28.7%	86	63	-36.5%	4.2
scenario 5	12763	20638	38.2%	111	69	-60.9%	3.9
scenario 6	9444	20952	54.9%	74	63	-17.5%	3.5
scenario 7	11913	–	–	72	–	–	–
scenario 8	11100	48824	77.3%	79	118	33.1%	3.7
Mean			49.1%			-21.4%	

The pattern mirrors the min_time analysis of the body. On the UF comparison the warm start is decisive in the three constrained scenarios: the cold start fails on scenarios 5, 6 and 7 (Restoration_Failed on scenario 5, Infeasible_Problem_Detected on scenarios 6 and 7), so those cells are shown as “–” and excluded from the means, while the warm-started solve reaches a feasible optimum in each. Over the five comparable scenarios the mean UF speedup is +9.0% in time and +18.5% in iterations, with the largest single gains on the empty scenarios 1–3 (+38.8%, +23.7% and +32.5% in time); scenarios 4 and 8 regress (–13.4% and –36.9%), exactly as the fixed warm-start overhead is or is not amortised by the NLP saving. Feasibility is 5/8 for the cold start against 8/8 for the warm start.

Table A.3: Per-scenario relative differences (min_fuel). UF: warm-start speedup / iteration reduction (positive $\Rightarrow$ warm start cheaper). FC: WS $\rightarrow$ UF $\rightarrow$ FC vs. WS $\rightarrow$ FC (positive $\Rightarrow$ keeping the UF stage is cheaper). Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	UF (WS vs No-WS)		FC (WS $\rightarrow$ UF $\rightarrow$ FC vs WS $\rightarrow$ FC)	
	Speedup	Iter. redux	Δ time	Δ iters
scenario 1	38.8%	11.1%	40.8%	-25.0%
scenario 2	23.7%	38.9%	56.9%	-2.1%
scenario 3	32.5%	36.1%	46.9%	-40.7%
scenario 4	-13.4%	3.7%	28.7%	-36.5%
scenario 5	–	–	38.2%	-60.9%
scenario 6	–	–	54.9%	-17.5%
scenario 7	–	–	–	–
scenario 8	-36.9%	2.6%	77.3%	33.1%

Table A.4: Trajectory end mass [kg] (min_fuel). UF: warm-start (WS) vs. no-warm-start (No-WS). FC: WS $\rightarrow$ UF $\rightarrow$ FC vs. WS $\rightarrow$ FC. Runs that did not converge or saturated the computational-time cap (overtime) are shown as – and are excluded from the means.

Scenario	UF [kg]			FC [kg]		
	WS	No-WS	Δ	WS $\rightarrow$ UF $\rightarrow$ FC	WS $\rightarrow$ FC	Δ
scenario 1	3999.3	3999.3	-0.0	3999.2	3999.2	+0.0
scenario 2	3998.7	3998.7	-0.0	3998.6	3998.6	+0.0
scenario 3	3998.7	3998.7	-0.1	3998.4	3998.4	+0.0
scenario 4	3998.1	3998.1	-0.1	3998.1	3998.0	+0.0
scenario 5	3998.9	–	–	3998.8	3998.8	+0.0
scenario 6	3998.6	–	–	3998.4	3998.4	-0.0
scenario 7	3994.7	–	–	3994.0	–	–
scenario 8	3998.4	3998.3	+0.1	3998.3	3998.3	+0.0
Mean Δ			-0.0			0.0

On the FC comparison the full pipeline WS→UF→FC is faster than the direct WS→FC seed in all seven comparable scenarios (mean +49.1%, up to +77.3% on scenario 8), and the direct seed times out on scenario 7 (Maximum_CpuTime_Exceeded, shown as “-”), giving 7/8 feasibility against 8/8 for the pipeline. As under `min_time`, the FC iteration totals are mostly larger for the pipeline (mean -21.4%) while its wall-clock cost is markedly lower, the signature of the expensive restoration iterations incurred by the rough direct seed. The end-mass column confirms that the value of the intermediate UF stage on this objective lies in convergence and cost rather than optimality: among the converged FC runs the two variants agree to within 0.1 kg.